Reference textbook: Computer Organization and Design, Revised Fourth Edition
David A. Patterson, John L. Hennessy

# Computer Organization

## Full course handout

A4 · Two slides per page · Lin Cing

---

## Contents

1. Abstractions, instructions, and arithmetic

Chapter 1 · Computer abstractions and performance 2
Chapter 2 · Instructions and program execution 22
Chapter 3 · Computer arithmetic 49

2. The processor and memory hierarchy

Chapter 4 · Processor datapaths and pipelines 83
Chapter 5 · Memory hierarchy 110

3. Storage, I/O, and parallel systems

Chapter 6. Storage and I/O 145
Chapter 7 Multicores, multiprocessors, and clusters 166

Reference appendices: GPUs and the program toolchain

Appendix A. Graphics and computing GPUs 185
Appendix B. Assemblers, linkers, and execution environments 206

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The chapter question

Between `sum += a[i]` and transistor switching lie several contracts. This chapter gives each contract a boundary that can be described and measured.

MIPS32 is the teaching instruction set architecture, or ISA. Its use does not imply that current processors all execute MIPS. ARM and x86 enter as comparative cases.

Ask which layer determines whether multiplication is an instruction, then which layer determines its cycle count. The first question concerns the ISA. The second concerns its implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Prerequisites and outcomes

The mathematical prerequisites are binary notation, scientific notation, ratios, and weighted averages. You should be able to predict execution time with consistent units and state the conditions under which the prediction fails.

Every performance claim needs a workload. A processor being "fast" without an input, operation, and measurement method is an incomplete statement.

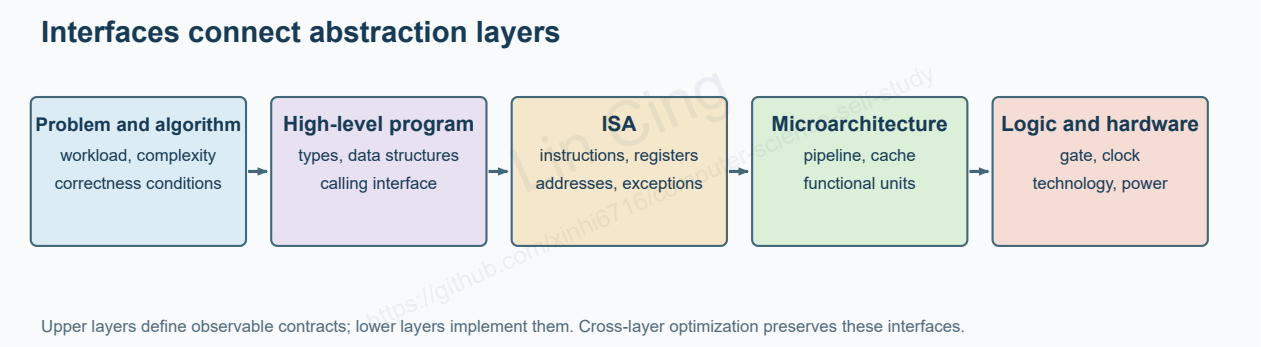
Repair a processor advertisement by supplying the workload, time unit, and method that would make its claim testable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 1.1 Computer abstraction layers

Applications use libraries and system calls. Compilers generate ISA instructions. Microarchitectures implement these instructions as datapath operations. Logic and physical technology implement registers, adders, and memory.

An interface specifies observable behavior, so the upper layer need not describe every transistor. Information still crosses layers during optimization. Data locality, for example, affects both compiler transformations and cache behavior.




An abstraction does not conceal everything. Instructions, addresses, exceptions, and performance counters expose selected lower-level behavior.



## ISA and microarchitecture

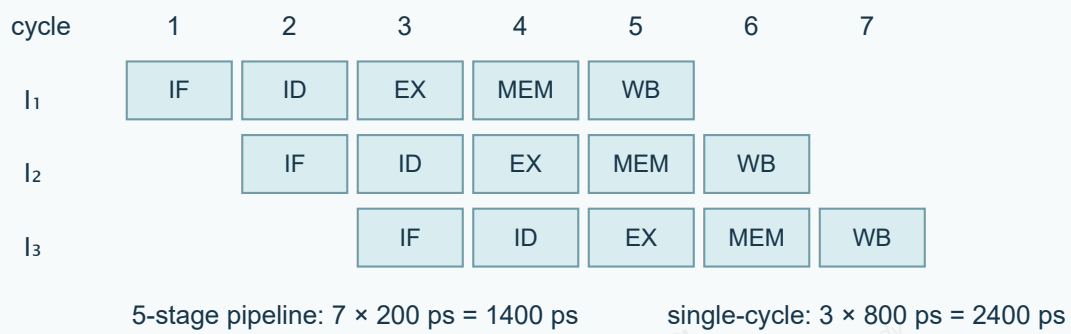
The ISA defines software-visible instructions, registers, data types, addressing, privilege, and exceptions. Processors with the same ISA may have different pipeline depths, caches, and execution units.

The microarchitecture defines how that contract is implemented. It determines the work in each clock cycle and the number of instructions that can remain in flight.

Prediction, caching, and reordering are valid only when committed behavior satisfies the architectural contract. Memory ordering rules constrain observations that concurrent software may make.

The same function interface can admit different internal algorithms. Interface compatibility alone says nothing about their costs.

## One ISA, two implementations: timing three instructions




Assume the same executable and input, three independent instructions, correct ISA implementations, no stalls or exceptions, and a MEM stage even for non-memory instructions. The single-cycle design uses one 800 ps cycle per instruction, finishing in $3 \times 800 = 2400$ ps.

The five-stage design uses 200 ps cycles and takes seven cycles from the first IF through the final WB: $7 \times 200 = 1400$ ps. Its speedup is $2400/1400 = 12/7$. The same instruction results do not imply the same execution time.



## Application classes and design tradeoffs

Personal computers, servers, embedded controllers, and mobile devices face different constraints. Servers often value throughput and availability. Mobile systems have battery and thermal limits. Embedded control may require a bounded worst-case response time.

A larger cache can improve a workload while consuming area and leakage power. Deep sleep can save energy while increasing wake-up latency. A useful design comparison includes performance, cost, power, dependability, and programmability.

Rank the constraints for a data-center CPU and a pacemaker. The appropriate optimization objective changes with the application.

## Course map

Chapter 2 establishes the ISA. Chapter 3 develops integer and floating-point arithmetic. Chapter 4 builds a datapath and pipeline. Chapter 5 introduces caches and virtual memory. Chapter 6 covers storage and I/O. Chapter 7 connects these ideas to multicore processors, multiprocessors, and clusters.

Dimensional analysis, comparisons against an explicit baseline, and workload-based validation recur throughout. For every new mechanism, identify the resource and time component it changes.

| Thread | Initial model | Mechanisms developed later |
|---|---|---|
| Data | Operands, results, and instruction count | Registers/addresses (2) → arithmetic (3) → datapath (4) → cache (5) |
| Control | ISA state and execution time | Branch/call (2) → hazards/exceptions (4) → I/O completion (6) |
| Communication | Bytes moved and shared resources | Coherence (5) → DMA/links (6) → synchronization/networks (7) |



## 1.2 From a program to hardware

A compiler translates a high-level language into assembly. An assembler encodes instructions. A static linker combines object files and resolves symbols and relocations. A loader establishes the process address space. The processor decodes and executes instructions according to the ISA.

Each stage leaves inspectable evidence. Compilation can change instruction count and register allocation. Assembly determines encodings. Linking determines final symbol references. Loading establishes initial mappings.

Source-code line count does not measure executed work. Optimization settings, data types, and compiler output can all change the dynamic instruction count.

Distinguish source, assembly, and machine code. The ISA connects symbolic instructions to their architectural effects and binary encodings.

## Interfaces and portability

An executable can run on implementations of its ISA only if its required architectural features are supported. Full binary compatibility also requires the operating-system interface and application binary interface, or ABI.

The ABI specifies register preservation, calling conventions, data alignment, and executable formats. These rules connect independently compiled code to the toolchain and runtime.

A processor can maintain an older ISA while translating its instructions into internal micro-operations. This is an implementation choice. Compatibility, portability, and equal performance are separate claims.

Ask which contract fails when code has the right instruction set but the wrong system-call interface.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 1.3 Under the covers

A system contains processors, volatile memory, nonvolatile storage, I/O devices, and interconnects. Input devices turn external state into data. Output devices turn data into observable effects.

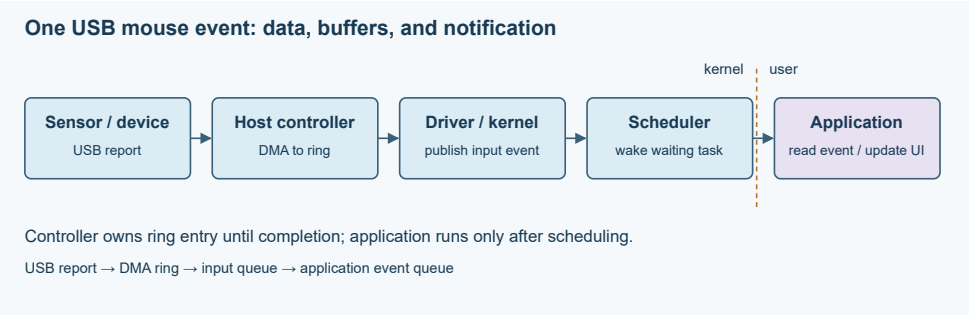
The processor contains a datapath and control. DRAM and persistent storage lie outside the core. Shared buses or point-to-point links carry addresses, data, and control messages.

Peripherals affect system performance through interrupts, direct memory access, and memory-mapped I/O. Their behavior belongs in an end-to-end model.

Fix a USB mouse, a host-controller DMA ring, and an application waiting for input. Buffers below identify separate ownership boundaries; an arrow does not imply that the entire report is copied.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## The path of a USB input event




The controller writes a report to a driver-managed ring. Completion notification lets the driver consume the finished entry and publish an input event. The scheduler then wakes the application, which reads its event queue. A ring slot cannot be reclaimed before DMA completion; application wake-up is later than packet arrival.



## A mouse as an interface example

A sensor produces motion information. A device controller packages that information and sends it through an interface such as USB or Bluetooth to a host controller. The driver passes events into the operating system's input subsystem.

Events may be batched instead of waking the application for every sample. Batching trades CPU wake-up energy against latency.

Slow visible response can originate in sampling, transport, scheduling, the application's frame rate, or display refresh. A single component rate cannot identify the cause.

Decompose end-to-end latency before assigning blame to the processor.

## A tablet as a system example

A system on chip, or SoC, can integrate CPU cores, a GPU, media processors, a memory controller, and I/O controllers. Integration reduces packaging and communication costs but concentrates heat and power density.

Touch response includes sampling, computation, GUI composition, and display scan-out. Completing a CPU burst earlier may still leave a display-stage wait.

Short bursts and long idle intervals make averages incomplete. Tail latency and energy spent changing power states can determine the user experience.

Identify five components that can compete for the same memory bandwidth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Motherboards and datapaths

The motherboard provides physical connections among chips and peripherals. Point-to-point links avoid some of the contention and electrical loading of a single shared bus.

On-chip interconnects connect cores, last-level caches, and memory controllers. Cache-coherence protocols regulate the visibility of copies of shared data.

Topology affects hop counts and contention. A link's peak bandwidth is an upper bound on traffic through that link, not a promise of sustained application bandwidth.

Describe an interconnect using nodes, links, routers, and a protocol.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Where data survives

Registers, caches, and DRAM are volatile. Flash, magnetic disks, and optical media retain data without continuous power.

Reliability, backup, and crash consistency address different failures. RAID can tolerate selected device failures. It does not replace an independent backup. A mean time between failures does not prove that a file system recovers a consistent state after a crash.

A cache is a temporary copy. A dirty line may hold the newest value before memory is updated. The logical data model must remain valid despite these changes in physical location.

A backup can exist yet take too long to restore. Recovery time is a separate requirement.



## Networks in system organization

Network latency, bandwidth, and protocol overhead affect both performance and availability. Propagation distance, switch queues, and congestion control contribute different delays.

An object may move from storage into memory and then reach the network interface through DMA. The CPU need not copy each byte, but it must arrange descriptors and handle completion.

Compute, storage, and communication must be modeled together. Fast arithmetic does not remove an I/O wait.

In a serial teaching model with an established connection, take encoding as 0.2 ms, queueing as 0.3 ms, the outbound transmission/propagation as 2.0 ms, server computation as 1.5 ms, and the return path as 2.0 ms. Latency is 0.2+0.3+2.0+1.5+2.0=6.0 ms. Accelerating only server computation by three gives 5.0 ms and speedup 6/5=1.2. These are declared model inputs. Overlapping communication and computation requires a dependency critical-path calculation instead of this serial sum.

## Technology and cost

Integration density and device dimensions constrain chip design. Smaller devices can reduce area and switching energy per function, but leakage, variability, and fabrication costs prevent automatic cost reductions.

The cost of a usable die is not simply proportional to die area. Larger dies fit fewer times on a wafer and expose a larger area to defects. Yield therefore matters as well as raw area.

Specify the cost boundary. Die fabrication, packaging, power delivery, cooling, racks, and operating energy have different denominators.

Additional transistors improve a program only when an architectural or parallel mechanism makes useful use of them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Die area, yield, and cost

Fix wafer diameter $D = 200$ mm, wafer cost 1000 cost units, and die area $A$ in mm². Use the geometric edge-loss approximation

$$N_{\text{die}} = \left\lfloor \frac{\pi (D/2)^2}{A} - \frac{\pi D}{\sqrt{2A}} \right\rfloor .$$

Assume negative-binomial yield $Y = (1 + dA_c/\alpha)^{-\alpha}$, $d = 0.5$ defects/cm², $\alpha = 2$, and $A_c = A/100$ cm² (100 mm²=1 cm²). Expected good-die cost is $1000/(N_{\text{die}} Y)$, excluding packaging, test, and redundancy.

| Die area | Area term minus edge term | Gross dies | Yield | Expected good dies | Cost/good die |
|---|---|---|---|---|---|
| 100 mm² | floor(314.159−44.429) | 269 | $(1+0.5/2)^{-2}$=0.64 | 172.16 | 5.8086 |
| 200 mm² | floor(157.080−31.416) | 125 | $(1+1/2)^{-2}$=4/9 | 55.5556 | 18.0000 |

Doubling area lowers count and yield: cost rises 18/5.8086≈3.10 times. Good-die count is an expectation; actual output is integral. All inputs are teaching assumptions, not manufacturing prices.

## 1.4 Defining performance

Response time is the elapsed time between the start and completion of a task. Throughput is completed work per unit time.

Shortening a critical path may reduce response time. Pipelining, batching, or adding cores may primarily increase throughput.

For a common workload and baseline,

$$S = \frac{T_{\text{old}}}{T_{\text{new}}}, \qquad T_{\text{new}} = \frac{T_{\text{old}}}{S}.$$

Halving time gives speedup 2 and a 50% time reduction. These are different ways of reporting the same comparison.

A claim of twice as many requests per second describes throughput. It does not specify each request's latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Latency and throughput

In a five-station pipeline, one task's latency includes all five stages. After filling, a pipeline may complete one task per stage interval. Reducing the completion interval need not reduce individual latency proportionally.

Batching amortizes setup work but may delay the first result. Cores can process separate tasks concurrently but still compete for shared resources and pay synchronization costs.

Fix the workload, input size, warm-up, software version, and observation window before comparing systems.

Decide whether twice the throughput justifies 10% worse latency for a specific application.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Benchmarks, ratios, and the geometric mean

A benchmark must specify its workload, input size, system configuration, compiler flags, warm-up, and measurement window. Scores obtained under different conditions describe different experiments.

For program $i$, divide the reference time by the measured time:

$$r_{X,i} = \frac{T_{\text{ref},i}}{T_{X,i}}, \qquad GM_X = \left(\prod_{i=1}^{n} r_{X,i}\right)^{1/n}.$$

Larger ratios mean shorter measured times. The reason for combining ratios geometrically follows directly:

$$\frac{GM_X}{GM_Y} = \left(\prod_{i=1}^{n} \frac{r_{X,i}}{r_{Y,i}}\right)^{1/n} = \left(\prod_{i=1}^{n} \frac{T_{Y,i}}{T_{X,i}}\right)^{1/n}.$$

Each program's reference time cancels in the paired X/Y ratio. Changing the reference machine leaves this ratio of geometric means unchanged, even when the new reference times change by different factors. The arithmetic mean has no such invariance. A suite summary still does not represent every workload. If one kernel occupies 90% of a user's time, reweight that workload or compare the kernel directly.

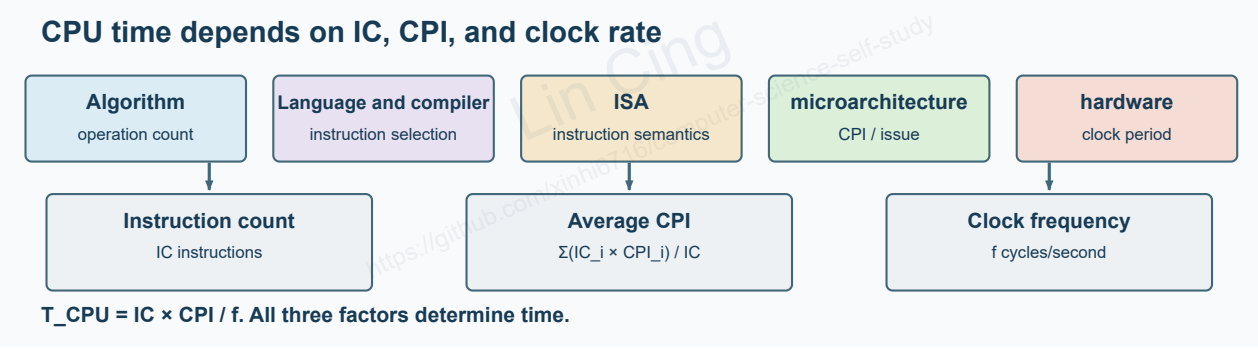


## Reconstructing CPU time

CPU time counts time spent executing for the program. For one thread over the same observation window, with disjoint user/system accounting,

$$T_{\text{elapsed}} \geq T_{\text{CPU}} = T_{\text{user}} + T_{\text{system}}.$$

I/O waits, descheduling, and OS work for other programs consume elapsed time. Summing thread CPU times can exceed elapsed time. Four cores each working for one second contribute 4 CPU-s during 1 elapsed s. State the thread/process accounting and time denominator.




Begin with the cycle count and then multiply by seconds per cycle:

$$C = IC \times CPI, \qquad T_{\text{CPU}} = CT_c = IC \times CPI \times T_c = \frac{IC \times CPI}{f}.$$

## Reconstructing CPU time (continued)

The dimensions cancel:

$$\text{instructions} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}} = \text{seconds}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Complete comparison

The same program executes $I = 10^9$ ISA instructions on A and B. A has CPI 2.0 and a 250 ps clock period. B has CPI 1.2 and a 500 ps clock period.

$$C_A = 10^9(2.0) = 2.0 \times 10^9, \qquad T_A = 2.0 \times 10^9(250\,\text{ps}) = 0.500\,\text{s}.$$
$$C_B = 10^9(1.2) = 1.2 \times 10^9, \qquad T_B = 1.2 \times 10^9(500\,\text{ps}) = 0.600\,\text{s}.$$
$$S_{A/B} = T_B/T_A = 0.600/0.500 = 1.2.$$

A is 1.2 times as fast. B's lower CPI loses to its longer cycle. Comparing any single factor omits the other two. This example directly assumes equal dynamic $IC$. The same executable and input yield equal counts if they produce the same instruction trace; the same ISA and high-level program alone do not ensure equal counts. Compiler choices, settings, and execution environment may change the count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Weighted CPI and the instruction-count trap

If class $i$ executes $IC_i$ instructions and $CPI_i$ is an additive, non-overlapping allocation of total cycles per instruction,

$$C = \sum_i IC_i CPI_i, \qquad CPI_{\text{avg}} = \frac{\sum_i IC_i CPI_i}{\sum_i IC_i}.$$

The denominator counts all executed instructions, not classes or source lines. Pipelined or superscalar instructions can overlap, so individual execution latencies cannot simply be added. The following example specifies additive cycle costs. If frequency varies, sum time over intervals or use the effective average cycle rate $C/T$ over the same active interval.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Complete counterexample

| Instruction class | A | B | C |
|---|---|---|---|
| CPI | 1 | 2 | 3 |
| Sequence 1 count | 2 | 1 | 2 |
| Sequence 2 count | 4 | 1 | 1 |

$$C_1 = 2(1) + 1(2) + 2(3) = 10, \qquad CPI_1 = 10/5 = 2.0.$$
$$C_2 = 4(1) + 1(2) + 1(3) = 9, \qquad CPI_2 = 9/6 = 1.5.$$

At the same frequency,

$$T_1 = 10T_c, \quad T_2 = 9T_c, \quad S_{2/1} = 10/9 \approx 1.111.$$

Sequence 2 executes one more instruction but takes one fewer cycle. Total cycles determine CPU time. A changed instruction mix invalidates an assumed constant CPI. Profilers, ISA simulators, and hardware counters can supply class counts and cycle observations; record each method's coverage and overhead.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Limits of the classic equation

The CPU-time equation is an accounting identity, not a guarantee that a local change improves total time. Its factors must describe the same execution.

CPU time differs from wall-clock response time. I/O waits, descheduling, and network delays may lie outside the measured CPU time.

A parallel model also needs the core count $p$, parallel fraction, imbalance, and synchronization overhead. Chapter 7 adds Amdahl's law and the Roofline model.

At fixed frequency, a lower CPI can still lose if the compiler increases instruction count enough.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Complete performance budget

Program A executes $2.0 \times 10^8$ instructions at CPI 1.25 and 2.0 GHz.

1. Cycles are $(2.0 \times 10^8)(1.25) = 2.5 \times 10^8$.
2. Cycle time is $1/(2.0 \times 10^9) = 0.5$ ns.
3. CPU time is $2.5 \times 10^8 (0.5\ \text{ns}) = 0.125$ s.

$$T_A = \frac{(2.0 \times 10^8)(1.25)}{2.0 \times 10^9} = 0.125\ \text{s}.$$

A new compiler produces $1.6 \times 10^8$ instructions, but CPI rises to 1.40 at the same frequency:

$$C_B = 1.6 \times 10^8 (1.40) = 2.24 \times 10^8,$$

$$T_B = \frac{2.24 \times 10^8}{2.0 \times 10^9} = 0.112\ \text{s}, \qquad S = 0.125/0.112 = 1.1161.$$

Instruction count falls 20%, while CPI rises 12%. The time saving is only

$$(0.125 - 0.112)/0.125 = 10.4\%.$$

Incorrectly retaining A's CPI gives 0.100 s and speedup 1.25. That calculation ignores the changed instruction mix.

## 1.5 The power wall

An approximate switching-power model is

$$P_{\text{dyn}} = \alpha C V^2 f,$$

where $C$ is switched capacitance, $V$ is supply voltage, $f$ is switching frequency, and $\alpha$ is the activity factor. Raising frequency often requires higher voltage, so power can rise faster than frequency.

Leakage also contributes. Lowering supply voltage alone generally reduces leakage but slows circuits. Lowering threshold voltage to recover speed can increase subthreshold leakage and sensitivity to process variation.

Cooling constrains sustained power density. A processor may throttle or disable resources when thermal limits are reached. A thermal design rating should not be interpreted as an exact universal throttling threshold.

Identify the controlled quantities before reasoning about device scaling or core count.



## From one core to many and the derivation of Amdahl's law

Power density, clock distribution, limited instruction-level parallelism, and memory latency prevent indefinitely raising single-core frequency. Multiple cores increase aggregate throughput and can reduce elapsed time when work splits into independent parts.

A fixed workload takes baseline time $T_0$. A fraction $fT_0$ can be shared among $N$ workers; $(1-f)T_0$ remains serial. Assume equal work and no added overhead:

$$T_N = (1-f)T_0 + \frac{fT_0}{N} = T_0\left(1 - f + \frac{f}{N}\right), \qquad S_N = \frac{T_0}{T_N} = \frac{1}{1 - f + f/N}.$$

As $N \to \infty$, the parallel term tends to zero but the serial term remains:

$$S_\infty = \frac{1}{1-f}.$$

## Required parallel fraction

For a target finite-$N$ speedup $S$,

$$\frac{1}{S} = 1 - f + \frac{f}{N} = 1 - f\left(1 - \frac{1}{N}\right),$$

so

$$f \geq \frac{N(S-1)}{S(N-1)}, \qquad 1 < S < N.$$

With $N = 100, S = 90$,

$$f \geq \frac{100(89)}{90(99)} \approx 0.998878.$$

The serial fraction must be at most about 0.1122%. This ideal model cannot attain $S > N$. It attains $S = N$ when $f = 1$ and added overhead is zero. Strict $S_N < N$ for finite $N > 1$ requires $f < 1$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## An unattainable finite target

A program takes 100 s, with multiplication using 80 s and other operations 20 s. A 5-fold overall speedup would require

$$20 = \frac{80}{s_{\text{mul}}} + 20.$$

Thus $80/s_{\text{mul}} = 0$, attainable only as a limit as $s_{\text{mul}} \to \infty$. No finite local speedup achieves it. For a 2-fold speedup,

$$50 = \frac{80}{s_{\text{mul}}} + 20 \quad \Rightarrow \quad s_{\text{mul}} = 80/30 = 2.667.$$

Measure the improved part's fraction of baseline elapsed time first. Operation-count and source-line fractions do not supply that time fraction.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Manufacturing and the AMD Opteron case

Fabrication, test, packaging, and system integration all contribute to delivery cost. Yield learning and design for test affect the count of usable chips. A configured machine's benchmark result alone does not establish manufacturing cost.

The book's AMD Opteron X4 example belongs to the late 2000s. Interpret it using its contemporary process, ISA, workload, and server configuration.

A reproducible benchmark needs at least processor configuration, memory, software versions, compiler settings, and input data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## SPEC CPU measurements

Speed-oriented results describe program execution time. Rate-oriented results describe completed work under a specified concurrent configuration.

A normalized reference score is a ratio, not a number of seconds. Adding ratios does not produce total performance.

Keep each benchmark's ratio, hardware configuration, compiler flags, and inputs. A geometric mean summarizes the suite but does not replace those observations.

If two machines win different programs, the user's workload determines which result matters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## SPEC power measurements

Power benchmarks relate completed work to power across load levels, including idle behavior. Peak performance alone can miss the cost of a server that spends much of its time at low utilization.

Power is measured in watts. Energy is measured in joules.

$$E = \int P(t)\,dt.$$

Compare energy for the same completed work. Account for idle power and state-transition costs when these occupy a material fraction of the observation window.

Determine whether a reported quantity is average power or accumulated energy before combining it with time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 1.8 Fallacies and pitfalls

A local speedup is not the overall speedup. If a component occupies only 10% of baseline time, eliminating its time saves at most 10% overall.

A benchmark win is workload-specific. Database, compiler, and numerical workloads may stress different resources.

More resources can increase energy and management cost without increasing completed work when utilization or shared bottlenecks prevent useful scaling.

For each claim, construct a counterexample and identify the assumption it violates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Abstractions and interfaces

Execution time is a joint result of the workload, ISA, compiler, microarchitecture, memory, and I/O. Use four questions throughout the course.

1. Is the measured quantity latency or throughput?
2. Which of instruction count, CPI, and frequency changed?
3. Which resources are shared, and are they saturated?
4. Can the benchmark and configuration be reproduced?

Apply these questions whenever a new optimization mechanism appears.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Historical perspective

Integration expanded the functions available on a chip and supported hardware description and synthesis. Pipelines, caches, and out-of-order execution exploited more single-core parallelism, with power and complexity costs.

Multicore processors, hardware multithreading, SIMD, and GPUs made parallelism increasingly important to mainstream software. Compilers, runtimes, and operating systems must expose or schedule useful work.

Historical comparisons confound technology, workload, and implementation unless their boundaries are stated.

A technology can enable a faster program without itself proving an application speedup.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Chapter review

- The ISA defines an interface; different microarchitectures can preserve results while taking different times.
- CPU time depends jointly on dynamic instruction count, mean CPI, and effective frequency for the same interval.
- State the baseline, units and workload; measure latency and throughput separately.
- Amdahl’ s law uses baseline time fractions, and the fixed part limits total speedup.

### References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 1. CSO course slides, 2026, Chapter 1.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

## The software/hardware contract

A compiler generates instructions that satisfy the ISA. The microarchitecture need not understand a source-language `if` or function. It must implement data movement, address calculation, exceptions, and memory effects correctly.

MIPS32 provides a traceable model of the interface. ARM and x86 supply comparisons. Reconstruct dataflow and addresses rather than memorize a catalog.

Place C, assembly, and machine code on separate levels. Identify where the program counter and stack pointer become architecturally visible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Notation and assumptions

$R[x]$ is a 32-bit register. $M[a]$ denotes data beginning at byte address $a$. $I[k:j]$ denotes bits $k$ through $j$. In target calculations, $PC$ is the current instruction address and $PC_{\text{next}} = PC + 4$.

Examples use traditional pre-Release-6 MIPS32, 32-bit instructions, and one branch or jump delay slot. An explicit `nop` occupies every control-transfer delay slot. `jal` saves $PC + 8$. The model does not include the earlier R2000 load-delay requirement.

If the assembler reorders instructions, use its supported ordering control (for example `.set noreorder`) and inspect pseudo-instruction expansion before relying on these exact delay-slot traces.

Instruction and word load/store addresses are four-byte aligned. Declare endianness when interpreting bytes. These assumptions must not be mixed with Release 6 control-transfer semantics.

ISA version, alignment, and byte order answer different questions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 2.1 The ISA contract

Programmer-visible state includes general-purpose registers, the program counter, floating-point registers, traditional HI and LO, and privilege and exception state.

Software may depend on instruction semantics, but not on pipeline depth or cache placement. Timing may change while architectural results remain the same. The ISA also defines exceptional conditions, including overflow and address faults. Privileged specifications determine exception-state and recovery rules.

"This instruction adds" is architectural. "This adder finishes in one cycle" is an implementation requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.2 Arithmetic operations

```
add $t0, $t1, $t2
```

The result is $R[t0] \leftarrow R[t1] + R[t2]$ when representable. `add`, `addi`, and `sub` trap on signed overflow. `addu`, `addiu`, and `subu` retain the modular result without that trap.

The `u` in `addiu` does not make its immediate unsigned. The 16-bit immediate is sign-extended. Choosing the trapping variant changes an observable exception behavior.

Distinguish operand extension from overflow handling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Expressiveness and cost

A small set of operations can express complex language behavior through composition. Multiplication can use iteration, shift-and-add, or a multiplier. The presence of `mult` changes available instruction sequences.

Fixed-width encoding gives predictable field positions but limits immediates. A complex instruction can reduce instruction count while adding decode or execution cost. Compare count, work per instruction, encoding, and compiler use together.

A RISC or CISC label alone is not a performance model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.3 Registers and operands

MIPS32 has 32 general-purpose registers of 32 bits. Scalar values are commonly allocated to registers. Loads and stores move data between registers and memory.

Ordinary arithmetic in this load/store architecture does not take two arbitrary memory locations as operands. Register-file ports and bypass paths are microarchitectural. The ISA specifies the value subsequent execution must observe.

Translate `c = a + b` using resident register values. A source-variable reference does not necessarily cause a load.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Memory operands

$$EA = R[base] + \text{signextend}(imm16) \pmod{2^{32}}.$$

`lw $t0, 12($s0)` loads the word beginning at that effective address. `sw` stores register data there. The signed displacement ranges from −32768 through 32767 bytes.

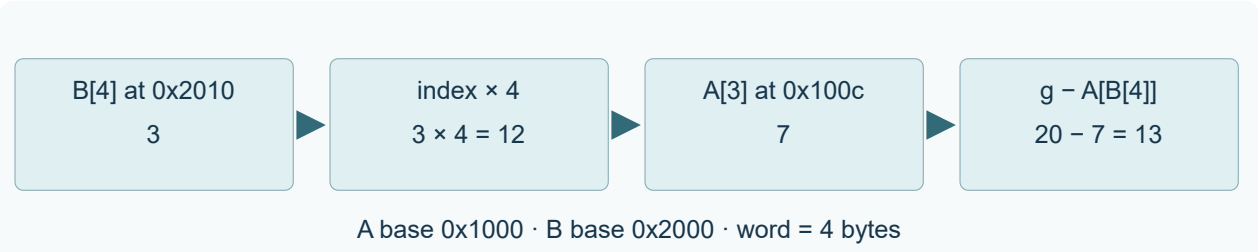
Large offsets need an address-construction sequence. Address formation does not remove alignment, translation, and permission checks.

Separate the address calculation from the interpretation of loaded data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Indirect indexing: trace both reads




Each array word is 4 bytes; base $A = 0x1000$, $B = 0x2000$, $B[4] = 3$, $A[3] = 7$, and $g = 20$. Assume valid aligned addresses and indexes, and no arithmetic exception.

```
lw   $t0, 16($s7)   # B[4] = 3
sll  $t0, $t0, 2    # 3 × 4 = 12 bytes
addu $t0, $t0, $s6  # address A[3] = 0x100c
lw   $s0, 0($t0)    # A[3] = 7
sub  $s0, $s1, $s0  # f = 20 − 7 = 13
```

Here `$s6=0x1000`, `$s7=0x2000`, and `$s1=20`; 16 is the byte offset for $B[4]$, not the index itself.

## Immediate operands

`addi $t0, $t0, -7` sign-extends the immediate. `lui` places its immediate in the upper half and clears the lower half.

```
lui $t0, 0x1234
ori $t0, $t0, 0x5678
```

This constructs `0x12345678`. `ori` zero-extends its immediate. A fixed 32-bit instruction cannot encode an arbitrary 32-bit constant plus all necessary opcode and register fields.

A memory literal may produce the same value with different code-size and cache costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.4 Signed and unsigned integers

Unsigned $n$-bit values span 0 through $2^n - 1$. Two's complement gives the high bit weight $-2^{n-1}$, so

$$-2^{n-1} \le x \le 2^{n-1} - 1.$$

The 32-bit range is −2147483648 through 2147483647. Complementing and adding one produces the negative bit pattern. Signed and unsigned addition can use the same adder, with different overflow interpretation. Comparisons and multiplication or division also require the chosen interpretation.

`0xfffffff6` is unsigned 4294967286 or signed −10.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Overflow and comparisons

Same-sign operands producing an opposite-sign result indicate signed addition overflow. For operand signs $s_a$, $s_b$ and result sign $s_r$,

$$overflow = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r).$$

Unsigned overflow is carry out. `slt` and `sltu` implement different orderings of the same bits. `0xffffffff + 1` carries out in unsigned arithmetic, but signed $-1 + 1$ does not overflow.

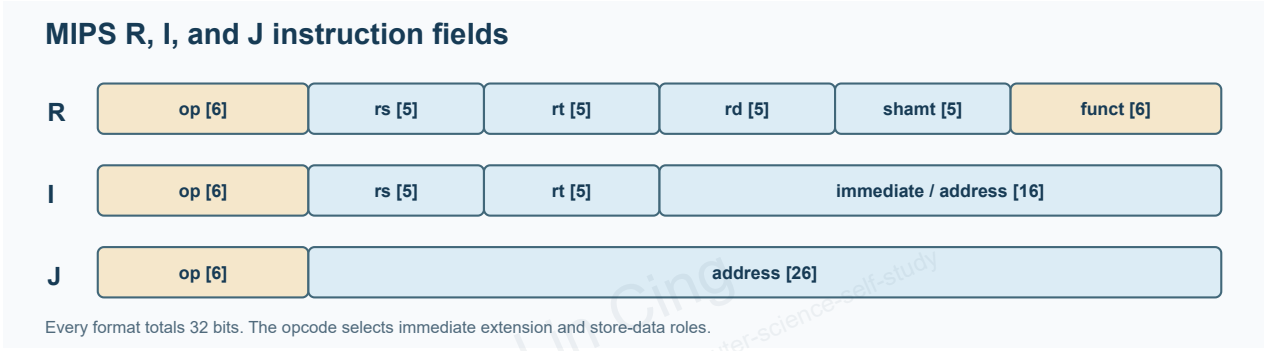
Interpretation precedes an overflow judgment.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.5 Instruction formats

R-format uses an opcode and function field. I-format contains a 16-bit immediate. J-format contains a 26-bit target index. Each occupies 32 bits.




Every format totals 32 bits. The opcode selects immediate extension and store-data roles.

Predictable register-field positions permit early register reads. Limited immediate width shifts work to multistep compiler-generated sequences.

Bit positions run from MSB 31 to LSB 0. This diagram is not a physical layout.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## R, I, and J fields

R-format is `op[31:26]`, `rs[25:21]`, `rt[20:16]`, `rd[15:11]`, `shamt[10:6]`, `funct[5:0]`.

I-format is `op`, `rs`, `rt`, `imm16`. Arithmetic and address displacements are sign-extended. Logical immediates are zero-extended. Loads and stores use the immediate for the address. Store data comes from `rt`.

J-format is `op` and `target26`. It cannot specify every absolute 32-bit target independently of location.

`rt` is a destination for `addi` and a source for `sw`. Its role follows the opcode.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.6 Logic and shifts

Bitwise operations include `and`, `or`, `nor`, and `xor`. Logical immediate forms zero-extend their masks. Logical shifts insert zeros. Arithmetic right shift replicates the sign bit. `sll`, `srl`, and `sra` can encode the shift amount in `shamt`.

```
srl  $t1, $t0, 8
andi $t1, $t1, 0x000f
```

This extracts bits 11 through 8. AND clears unselected bits, OR sets bits, and XOR toggles selected bits.

Match mask and shift before interpreting the extracted field.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 2.7 Decisions and branches

`beq rs, rt, label` chooses the target on equality. In this traditional model, its next instruction executes regardless of branch direction. Execution then reaches the target or continues after the delay slot.

Examples use `nop` in that slot. Release 6 control transfers require their own semantics. Comparison pseudo-instructions may expand into `slt` and a branch.

Branch delay is part of this architectural model. It does not remove data hazards.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Translating a loop

```
int sum = 0;
for (int i = 0; i < n; ++i)
    sum += a[i];
```

Let `$a0` hold the array base, `$a1` hold $n$, and `$v0` hold the sum.

```
move $v0, $zero
move $t0, $zero
beq  $t0, $a1, done
nop
loop:
sll  $t1, $t0, 2
addu $t1, $t1, $a0
lw   $t2, 0($t1)
addu $v0, $v0, $t2
addiu $t0, $t0, 1
bne  $t0, $a1, loop
nop
done:
```

## Translating a loop (continued)

Assume $n \geq 0$, valid aligned storage, and no address wrap. The sum is modular unless inputs guarantee representable signed C arithmetic. Scaling by four converts word index to bytes.

At iteration $i$, the accumulator equals the sum of the first $i$ elements modulo $2^{32}$. Initialization, preservation, and exit prove the result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Switch strategies

Dense case values can use a jump table. Check the index range, scale the index, load the target, and use `jr` with its delay slot. Sparse cases may use a branch chain or a binary decision tree.

Tables reduce decision depth but consume space and add indirect-branch and cache costs. The choice depends on case distribution, safety, and implementation behavior.

Indirect prediction and table locality can outweigh source-code brevity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Complete decoding and control-flow trace

Decoding must determine format and field roles before interpreting a mnemonic.

| Word | Fields | Architectural effect |
|---|---|---|
| `0x8fa8000c` | `op=0x23, rs=29, rt=8, imm16=12` | `lw $t0,12($sp)`, with `EA=R[sp]+12` |
| `0x012a4020` | `op=0, rs=9, rt=10, rd=8, shamt=0, funct=0x20` | `add $t0,$t1,$t2` |
| `0x0c100140` | `op=3, target26=0x100140` | `jal` to `0x00400500` in the same 256-MiB region |

I-format immediates are not uniformly signed. Loads and `addi/addiu` sign-extend; `andi/ori/xori` zero-extend. A store reads its data from `rt`. The opcode determines whether `rt` is a source or destination.

MIPS `addu/addiu` compute modulo $2^{32}$. C unsigned arithmetic is also modulo its type width, but C signed overflow is undefined behavior. Signed source programs require a proof that values remain representable, or an unsigned/wider type, before a modulo machine result can be claimed to implement their semantics.

For `0x1109fffc` at `0x00400020`, $imm16 = 0xfffc = -4$:

$$PC_{\text{next}} = 0x00400020 + 4 = 0x00400024,$$

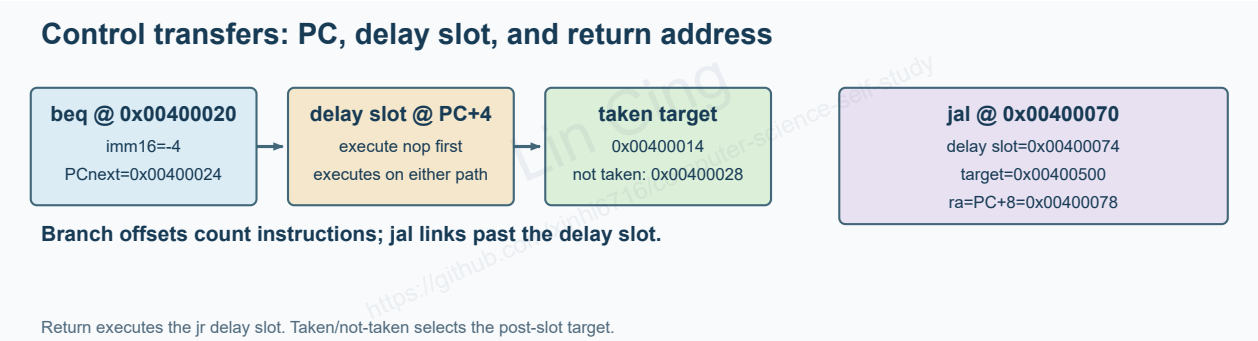


## Complete decoding and control-flow trace (continued)

$$target = PC_{\text{next}} + (\text{sign_extend}(0xfffc) \ll 2) = 0x00400024 - 16 = 0x00400014.$$

The taken PC sequence is `0x00400020 → 0x00400024 → 0x00400014`. The not-taken sequence is `0x00400020 → 0x00400024 → 0x00400028`. The delay-slot instruction at `0x00400024` executes in both cases.

For a `jal` at `0x00400070`, its slot is `0x00400074`, target `0x00400500`, and `$ra=PC+8=0x00400078`. `jr $ra; nop` executes its own delay slot before reaching that return address.




Instruction addresses must be word aligned. Branch offsets count instructions. J-format targets inherit the high four bits of PC+4. Mixing bytes and words, or using the current PC's high bits at a region boundary, changes the target.

## 2.8 Procedure calls

`jal target` saves $PC + 8$ in `$ra` and transfers after its delay slot. `jr $ra` returns after its own delay slot. A nested `jal` overwrites the incoming return address, so a nonleaf procedure must preserve that address when it needs a normal return.

The ABI defines calling conventions. Hardware does not automatically allocate the full frame or preserve all registers.

Trace `main` to `f` to `g`. Locate the point where an unsaved continuation is lost.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Register responsibilities

| Register | Convention |
|---|---|
| `$zero` | Always zero |
| `$a0–$a3` | Arguments |
| `$v0`, `$v1` | Results |
| `$t0–$t9` | Caller-saved temporaries |
| `$s0–$s7` | Callee-saved values |
| `$sp` | Stack pointer |
| `$ra` | Return address |

A caller preserves temporary values it still needs after a call. A callee restores saved registers it modifies. Hardware does not infer these obligations from register names.

Preservation is conditional on modification and liveness, not on a register merely being mentioned.

## Nested calls and the return address

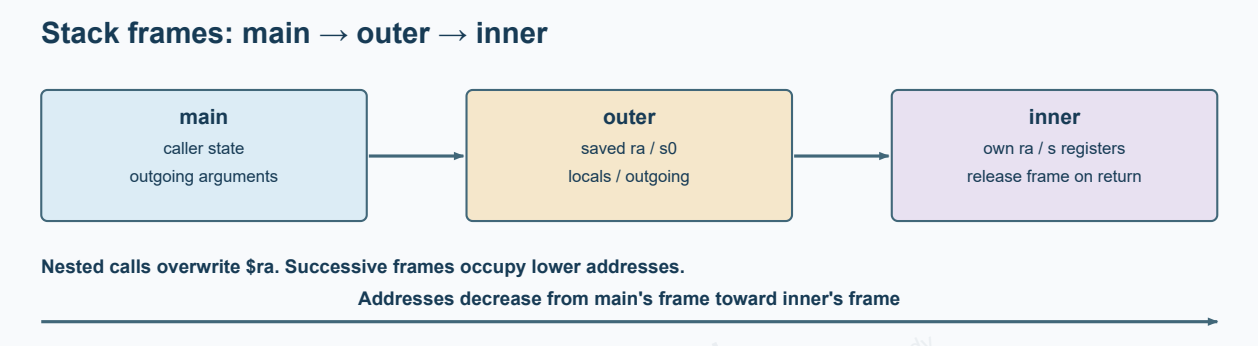
```
addiu $sp, $sp, -8
sw    $ra, 4($sp)
jal   g
nop
lw    $ra, 4($sp)
addiu $sp, $sp, 8
jr    $ra
nop
```

This conceptual minimal frame preserves the incoming return address. It is not a full ABI template. Real frames may need outgoing arguments, locals, spills, and saved registers with required alignment.

The return-address restore belongs after the call's delay slot, not inside it.



## Stack frames




A frame can hold the saved return address, saved registers, local arrays, spills, and outgoing arguments. Simultaneously live slots cannot overlap.

Entry subtracts frame size from `$sp`. Exit restores state and the incoming `$sp`. ABI rules determine slot alignment and argument layout. A pointer into a returned frame does not extend its lifetime. Another call can overwrite that storage.

Mark high and low addresses. These examples use a downward-growing stack.

## Heap storage

Stack lifetime follows procedures. Heap objects allocated through `malloc` or `new` persist until deallocation or collection. Allocators manage free blocks, metadata, and fragmentation.

Allocation may fail or trigger page faults and virtual-address-space expansion. Successful virtual allocation does not imply physical residency of every page.

Returning a pointer to a local integer does not preserve that object after return; dereferencing it violates the object lifetime even if the stack bytes have not yet changed. A heap leak instead loses the last usable pointer to a still-allocated object, preventing its release. Repeated allocation followed by overwriting the sole pointer can accumulate leaks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.9 Characters and bytes

ASCII defines seven-bit code points commonly stored in eight-bit bytes. Unicode uses encodings such as UTF-8 and UTF-16. C strings use a zero terminator; other formats may carry length metadata.

`lb` sign-extends a byte. `lbu` zero-extends it. `sb` stores a byte. Copying a string requires adequate destination capacity and correct termination.

Endianness orders multibyte values and must be specified at exchange boundaries.

Bytes `41 42 43 00` form the NUL-terminated ASCII string "ABC". As one 32-bit integer they mean 0x41424300 in big-endian order or 0x00434241 in little-endian order. Bits alone do not determine the type or byte order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 2.10 Large constants

`lui` sets bits 31 through 16 and clears the lower half. `ori` fills that half without sign extension. Construct `0x8000abcd` using `lui` with `0x8000` and `ori` with `0xabcd`.

`lui` followed by `addiu` uses a signed low half. When low bit 15 is set, the high half must be increased by one to compensate. Appendix B derives the relocation rule.

The high half for a logical OR sequence is not always the correct high half for a signed-low-half addition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Branch and jump targets

$$target = (PC + 4) + (\text{signextend}(imm16) \ll 2).$$

The displacement spans −131072 through 131068 bytes relative to $PC + 4$. Its encoded unit is an instruction word.

$$target = (PC + 4)[31 : 28] \parallel target26 \parallel 00_2.$$

J-format selects the 256-MiB region of $PC + 4$, which can differ from the current instruction's region at a boundary.

Saving $PC + 8$ for a delayed call does not change the branch-target base from $PC + 4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Addressing modes

The examples use register, immediate, base-plus-displacement, PC-relative, and pseudo-direct addressing. `lw $t0, 0($sp)` uses the architectural base value for that instruction. Forwarding or stalls handle dependencies. A branch delay slot does not preserve an old base value.

Extra addressing modes consume encoding space and complicate decode and exceptions. Pseudo-direct targets save bits but restrict placement. A linker may need a trampoline or indirect transfer for an unreachable target.

Equal addresses reached by two modes can depend on different base values.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Decoding machine code

```
0x012a4020
op    = 000000
rs    = 01001   $t1
rt    = 01010   $t2
rd    = 01000   $t0
shamt = 00000
funct = 100000  add
```

This is `add $t0, $t1, $t2`. Decode the opcode first. Confirm ISA, register names, and instruction length before field extraction.

Interpreting variable-length x86 bytes as MIPS fields cannot establish a meaningful instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 2.11 Atomic synchronization

Two cores can load $x$, independently compute $x + 1$, and both store it. One increment is lost.

`ll` loads and establishes a reservation. `sc rt, offset(base)` uses the old `rt` as store data. Success atomically stores that data and sets `rt` to 1. Failure writes no shared data and sets `rt` to 0.

```
retry:
ll    $t0, 0($s0)
addiu $t0, $t0, 1
sc    $t0, 0($s0)
beq   $t0, $zero, retry
nop
```

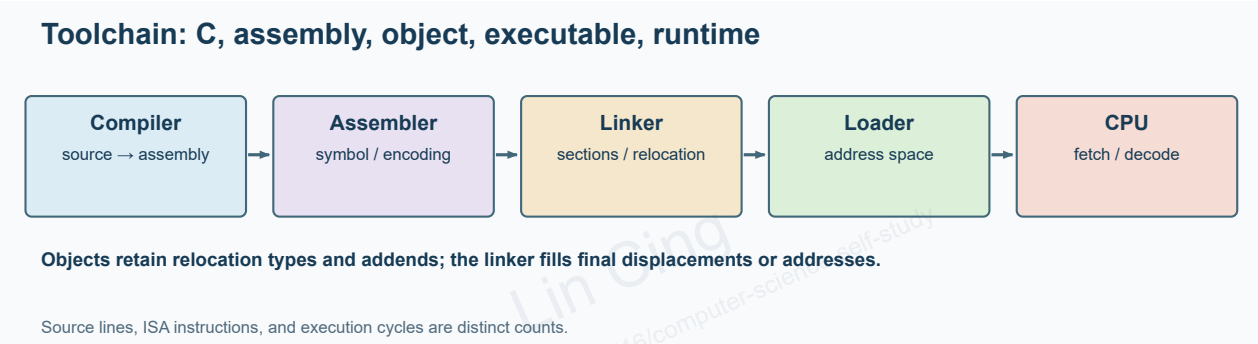
Retry rereads the counter. Reservation granularity and invalidation events depend on architectural and implementation rules. Neither atomicity nor this loop alone guarantees fairness or ordering of unrelated memory.

An uninitialized different `sc` register would store incorrect data even if its reservation succeeded.



## 2.12 From C to an executable

The compiler selects instructions and optimizes. The assembler expands pseudo-instructions and emits encodings, symbols, and relocations. The linker combines sections and resolves references. The loader creates mappings and initial state.




Language-semantics errors, invalid instruction operands, undefined symbols, and mapping failures belong to different stages.

Trace an external function through assembly relocation and runtime resolution.

## Address meaning in the toolchain

Labels initially identify section-relative locations. External symbols may remain unresolved. Relocations identify the operation needed when final symbol values become known.

Static linking resolves references into an executable. Position-independent shared libraries can share physical code pages while allowing runtime resolution.

Virtual addresses, file offsets, segment-relative offsets, and physical addresses are distinct coordinates. Loader mappings connect them but do not make them identical.

A runtime relocation can change an address without moving the corresponding bytes within the file.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A calling-convention example

```
int twice_sum(int x, int y) {
    int s = x + y;
    return s * 2;
}
```

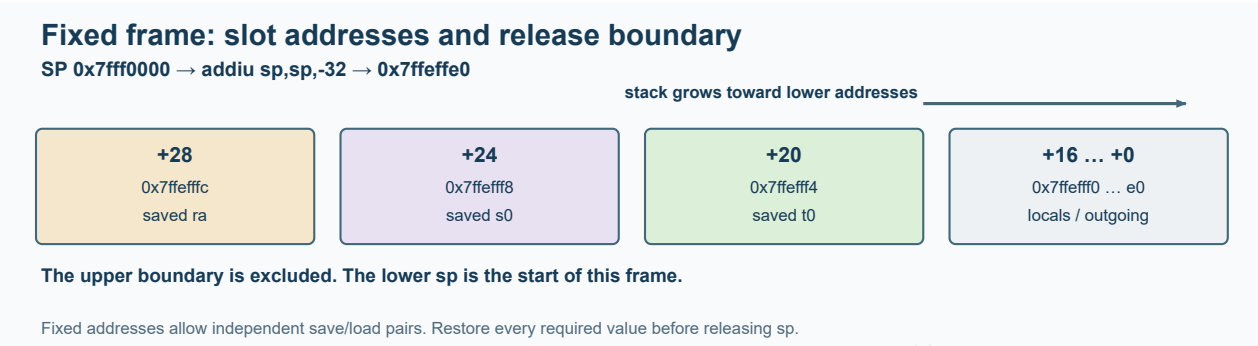
For inputs whose signed operations are representable,

```
twice_sum:
addu $v0, $a0, $a1
sll  $v0, $v0, 1
jr   $ra
nop
```

The leaf makes no nested call and modifies no saved register, so it need not preserve `$ra`. Outside the valid signed-input range, modular machine behavior does not establish defined signed C behavior.

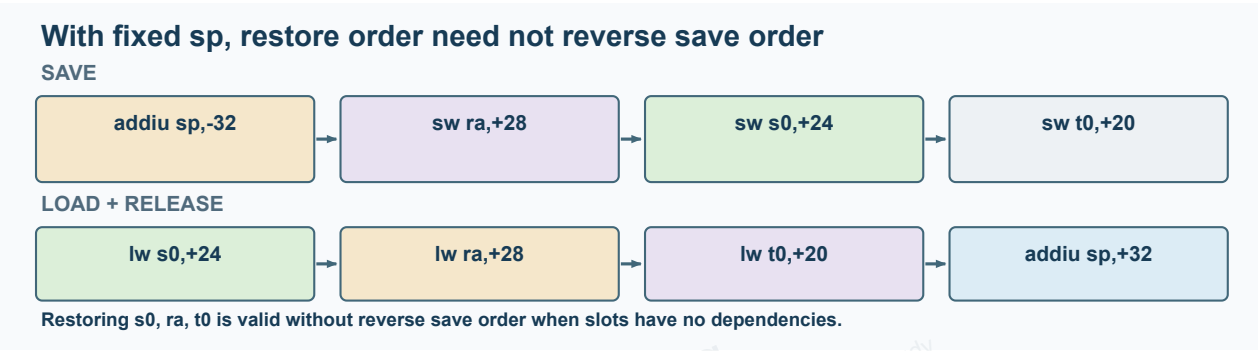
Use of `$s0` would add a restoration obligation.

# Fixed-frame slot addresses






# Fixed-frame save, load, and release order




Entry `sp=0x7fff0000` and a 32-byte allocation give `sp=0x7ffeffe0`.

| Slot | Address | Saved value | Restored value |
|---|---|---|---|
| `28(sp)` | `0x7ffefffc` | `ra` | `ra` |
| `24(sp)` | `0x7ffefff8` | `s0` | `s0` |
| `20(sp)` | `0x7ffefff4` | `t0` | `t0` |

Saving `ra,s0,t0`, then restoring `s0,ra,t0`, then adding 32 to `sp` is valid. Restore order need not reverse save order. The base stays fixed, the slots are distinct, and load destinations have no mutual dependencies.

## Fixed-frame save, load, and release order (continued)

Reverse order is necessary for a push/pop sequence that changes `sp`, or when slot/register dependencies require it. Release timing remains mandatory. Load all required values before increasing `sp`; an old offset applied to the new base addresses a different location.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Recursive activation trace

`main` calls `f` at `0x00400010`, producing return address `0x00400018`. With entry `sp=0x7fff0000`, `f` allocates 32 bytes and reaches `sp=0x7ffeffe0`.

| Store | Address | Value |
|---|---|---|
| `sw $ra,28($sp)` | `0x7ffefffc` | `0x00400018` |
| `sw $s0,24($sp)` | `0x7ffefff8` | caller's old `$s0` |

A recursive `jal f` at `0x00400120` sets `$ra=0x00400128`. The inner activation allocates its own frame at `sp=0x7ffeffc0` and saves that return address at `0x7ffeffdc`. Both frames are live. `sp+28` always identifies the current activation's slot; the inner base cannot read the outer slot with the same offset.

The base case restores its own `$ra` and `$s0`, releases its frame to `0x7ffeffe0`, and returns to `0x00400128`. The outer activation eventually reloads `0x00400018` from `0x7ffefffc`, releases 32 bytes, and returns to `main`. Every `jal` and `jr` retains its `nop` delay slot. Removing it changes the executed sequence.

This trace establishes the need to save `$ra` before a nested call, compute slots from the current base, and complete restores before release.

## Arrays and pointers

$$address = base + i \times element_size.$$

Pointer traversal increments by element size. It can avoid repeated scaling, but the compiler can apply the same strength reduction to an indexed loop.

Pointer width is platform-dependent. This machine model uses 32-bit addresses. C and C++ object rules constrain pointer arithmetic. A hardware-generated bit pattern need not be a valid language-level dereference.

Compare generated instructions rather than source spelling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Complete sorting dataflow and cost

Insertion by adjacent swaps connects C control flow, array addresses, and procedure calls:

```
for (int i = 0; i < n; i += 1)
  for (int j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1)
    swap(v, j);
```

For `v=[3,1,2]`, `i=0` has no inner iteration. At `i=1,j=0`, `3>1` causes a swap to `[1,3,2]`. At `i=2,j=1`, `3>2` produces `[1,2,3]`. Then `j=0` tests `1>2`, fails, and stops. The prefix is sorted after each outer iteration.

The complete classic MIPS32 implementation follows. Its 24-byte teaching frame omits the 16-byte o32 outgoing argument home area. It is valid with the supplied leaf `swap` under this declared convention; general o32 interoperability requires an outgoing area at offsets 0..15 and relocated saved slots. Preconditions are `n>=0`, `n` aligned accessible words, and nonwrapping effective addresses. Every control transfer retains its delay slot.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Complete sorting dataflow and cost (continued)

```
sort:
addiu $sp, $sp, -24
sw    $ra, 20($sp)
sw    $s3, 16($sp)
sw    $s2, 12($sp)
sw    $s1, 8($sp)
sw    $s0, 4($sp)
addu  $s2, $a0, $zero
addu  $s3, $a1, $zero
addu  $s0, $zero, $zero
for1:
slt   $t0, $s0, $s3
```

https://github.com/xinhi6716/com



## Complete sorting dataflow and cost (continued)

```
beq   $t0, $zero, exit1
nop
addiu $s1, $s0, -1
for2:
slti  $t0, $s1, 0
bne   $t0, $zero, exit2
nop
sll   $t1, $s1, 2
addu  $t2, $s2, $t1
lw    $t3, 0($t2)
lw    $t4, 4($t2)
slt   $t0, $t4, $t3
```

https://github.com/xinhi6716/com

## Complete sorting dataflow and cost (continued)

```
beq   $t0, $zero, exit2
nop
addu  $a0, $s2, $zero
addu  $a1, $s1, $zero
jal   swap
nop
addiu $s1, $s1, -1
j     for2
nop
exit2:
addiu $s0, $s0, 1
j     for1
```



## Complete sorting dataflow and cost (continued)

```
nop
exit1:
lw    $s0, 4($sp)
lw    $s1, 8($sp)
lw    $s2, 12($sp)
lw    $s3, 16($sp)
lw    $ra, 20($sp)
addiu $sp, $sp, 24
jr    $ra
nop
```

With `v=0x10010000`, `k=1`, scaling gives 4, so the element addresses are `0x10010004` and `0x10010008`. The leaf loads both words before storing them in exchanged order. It modifies only caller-saved temporaries and needs no frame.

## Complete sorting dataflow and cost (continued)

```
swap:
sll  $t1, $a1, 2
addu $t1, $t1, $a0
lw   $t0, 0($t1)
lw   $t2, 4($t1)
sw   $t2, 0($t1)
sw   $t0, 4($t1)
jr   $ra
nop
```

For one steady-state clear-loop iteration, array indexing executes `sll+addu+sw+addiu+slt+bne+nop`, seven instruction positions. Pointer traversal executes `sw+addiu+slt+bne+nop`, five positions. The two saved positions compute scaling and base addition. Prologues, initial guards, and exits are outside this count.

The comparison holds word size, traffic, delay-slot policy, and absence of faults/traps fixed. At `n=100`, the specified bodies execute 700 and 500 instruction positions; initialization and final exit checks are additional. Strength reduction and induction-variable elimination can transform the index version into pointer-like code, so source spelling alone does not establish performance.



## Compilation and interpretation

C compilation commonly produces target-specific native code. Java bytecode belongs to a virtual machine. A JVM can interpret, compile just in time, or combine the two.

Interpretation pays dispatch costs. A JIT can specialize hot code from profiles, but warm-up and compilation add startup cost. Integer overflow, alignment, atomic operations, and memory models also differ across language runtimes.

Separate language semantics, runtime implementation, and native ISA.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## ARM comparison

The book's ARM discussion concerns the 32-bit ARM era. Conditional execution, shifted operands, and addressing choices reduce some sequences. ARM and MIPS both illustrate load/store organization and procedure conventions.

Encoding, flags, and extensions differ. Do not transfer a 32-bit ARM example into AArch64 or assume every version has the same operations.

Identify the architecture version before interpreting a conditional instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## x86 comparison

x86 instructions have variable lengths and can contain prefixes, ModR/M, SIB, displacements, and immediates. Boundaries are not fixed at four-byte intervals.

Dense encoding and compatibility add fetch and decode complexity. Implementations can translate instructions into internal micro-operations for superscalar execution. ISA count therefore differs from micro-operation count.

A micro-operation cache can reduce repeated decode work without changing the ISA into MIPS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## x86-64 effective addresses and structure layout

Under the default System V AMD64 alignment without packing, with an allocated row and $0 \le j < 4$, for `struct row { int64_t id; int32_t vals[4]; int64_t sum; };`, the fields occupy offsets 0..7, 8..23, and 24..31. Eight-byte alignment gives a 32-byte structure. With base `0x3000`, `i=5`, and `j=3`,

$$address = 0x3000 + 5(32) + 8 + 3(4) = 0x30b4.$$

Let `%rdx` hold the base, `%rdi` hold `i`, and `%esi` hold `j`:

```
movslq %esi, %rsi
salq   $5, %rdi
leaq   8(%rdi,%rsi,4), %rax
leaq   (%rdx,%rax), %rax
```

The first `lea` produces offset `0xb4`; the second adds the base. `lea` evaluates an address expression without reading memory. The scale field admits 1,2,4,8, so a 32-byte structure stride requires the separate shift. Variable instruction length changes layout and decoding; it does not change the address arithmetic. Source-level signed bounds and object validity remain necessary even when the machine can form a bit pattern.



## ISA fallacies

A powerful instruction can have long latency or restrict parallel issue. Fewer instructions need not mean fewer cycles. More registers do not eliminate register pressure, calling costs, or spills.

Pseudo-instructions are assembler facilities. Their expansions can depend on operands and immediate ranges. Source lines, architectural instructions, and cycles count different things.

Trace one expression through all three counts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## ISA design choices

Encoding, compiler support, exceptions, translation, synchronization, and compatibility constrain ISA design. Fixed fields simplify extraction but require multistep constants. Variable formats can improve density while complicating boundaries.

Correctness requires the specified architectural effect. Performance depends on instruction mix, CPI, frequency, and memory paths.

State width, alignment, and extension for every address and condition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Historical perspective

ISA history contains both regular fixed-field designs and complex encodings. RISC emphasized regular operations suited to compilers and pipelines. CISC often emphasized density and compatibility.

Either family can use internal micro-operations. Historical constraints explain decisions more reliably than timeless claims that a label always wins.

Compare code size, decode, traffic, issue, and compatibility separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Chapter review

- ISA version, delay slots, and assembler ordering determine control flow.
- Addresses use a byte offset; check element size, bounds, and alignment.
- Signed-overflow traps depend on the instruction; the ABI defines preservation and layout.
- LL/SC success, memory ordering, and fairness are distinct guarantees.

### References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 2. CSO course slides, Chapter 2.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

## Representation and semantics

Arithmetic combines finite-width representations with operation rules. An adder processes bit patterns. Signed interpretation, instruction semantics, and status controls determine what happens when a mathematical result is outside the representable range.

The chapter develops integer ALUs, iterative multiply/divide hardware, and IEEE 754 floating point. Every model needs operand width, result width, rounding, exceptions, and timing assumptions.

Interpret `0xffffffff + 1` both ways before discussing its circuit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The arithmetic problem

A fixed 32-bit word cannot represent every integer, fraction, or real number. Overflow, underflow, rounding, and special values are part of a representation contract.

Hardware trades latency, throughput, area, and precise exceptions. A multiplier can generate more partial products in parallel. A floating-point adder retains extra bits for rounding, which requires additional datapath and pipeline state.

Define the required result before choosing a gate diagram.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.1 The basic model

Modular integer instructions compute in the ring modulo $2^n$. Trapping and saturating instructions impose different policies. Floating point maps an exact arithmetic result into a finite representable set $F$.

$$y = \operatorname{round}(x \circ z).$$

The rounding function, exceptional conditions, and special values are part of the operation. Correct rounding does not imply an infinitely precise mathematical result.

Integer division by zero depends on the ISA or language. IEEE floating-point `0/0` is invalid and ordinarily produces a quiet NaN when no trap is enabled.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.2 Two's-complement addition and subtraction

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

The low $n$ result bits of an adder are the same for signed and unsigned addition. Whether they represent the mathematical signed sum depends on overflow.

$$a - b = a + \overline{b} + 1 \pmod{2^n}.$$

Complementing the second input and setting carry-in to one lets subtraction share an adder.

A correct modular bit pattern can still be an unrepresentable signed mathematical result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Deriving signed overflow

Let the operand signs be $s_a$, $s_b$ and the result sign be $s_r$. Equal operand signs with a changed result sign indicate overflow.

$$ov = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r) = c_{\text{into MSB}} \oplus c_{\text{out of MSB}}.$$

Unsigned overflow depends only on carry out. `0x7fffffff+1` overflows signed addition. `0xffffffff+1` overflows unsigned addition while signed −1 plus 1 is valid.

Never replace both tests with an unqualified "overflow equals carry."

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Check carries separately from signed overflow

For an eight-bit adder, let $c_7$ enter the sign position and $c_8$ leave it. Signed overflow is $ov = c_7 \oplus c_8$.

| Operation | Adder inputs | Result | $c_7$ | $c_8$ | Unsigned carry | Signed overflow |
|---|---|---|---|---|---|---|
| `0x7f+0x01` | `01111111+00000001` | `10000000` | 1 | 0 | 0 | 1 |
| `0x80+0xff` | `10000000+11111111` | `01111111` | 0 | 1 | 1 | 1 |
| `0x80-0x01` | `10000000+11111110+1` | `01111111` | 0 | 1 | 1 | 1 |
| `0xff+0x01` | `11111111+00000001` | `00000000` | 1 | 1 | 1 | 0 |

The last row has a carry without signed overflow. `slt` and `sltu` also interpret identical bits differently. Subtraction uses $a + \bar{b} + 1$, with overflow $(s_a \oplus s_b) \wedge (s_a \oplus s_r)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Adder timing and exceptions

A ripple-carry adder propagates carry along the width. Lookahead and parallel-prefix designs shorten this dependency using more gates and wiring.

An overflowing `add` must not commit an ordinary destination result in place of its exception. `addu` retains the modular result without the signed-overflow trap.

Clock design uses the worst valid path and setup/hold constraints, not average operand behavior. Result generation and exception detection can have distinct paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Saturating multimedia arithmetic

Saturation clamps an out-of-range result to an endpoint. Signed eight-bit $100 + 100$ saturates to 127 instead of wrapping to a negative value.

This avoids wraparound but loses reversibility. It cannot replace modular arithmetic arbitrarily in checksums or cryptographic algorithms.

Packed SIMD arithmetic operates on several short lanes. Carries do not cross lane boundaries. Lane width and saturation policy belong to the instruction semantics.

Audio clipping and raw byte addition require different numerical contracts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.3 Multiplication hardware

An exact unsigned $n \times n$ product can require $2n$ bits. Two 32-bit inputs therefore need up to a 64-bit result.

$$AB = \sum_{k=0}^{n-1} b_k (A \ll k).$$

Shift-and-add accumulates a shifted multiplicand for each set multiplier bit. Shifting handles a power of two. Generating arbitrary partial products also requires bitwise gating.

Truncating an input or partial product before the exact computation changes the problem.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Sequential multiplication and correctness

For unsigned n-bit M and Q, inspect Q0, conditionally compute $C : A \leftarrow A + M$, and shift the full $(C, A, Q)$ right. C enters A's high bit. For M=13 and Q=11:

| Round | Q0 | Add result C:A | A after shift | Q after shift |
|---|---|---|---|---|
| 0 | | | `0000` | `1011` |
| 1 | 1 | `0:1101` | `0110` | `1101` |
| 2 | 1 | `1:0011` | `1001` | `1110` |
| 3 | 0 | `0:1001` | `0100` | `1111` |
| 4 | 1 | `1:0001` | `1000` | `1111` |

The final product is `10001111` =143. Combined states R0..R4 are 11,109,158,79,143.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Sequential multiplication: prove the register invariant

Fix the initial multiplier $q < 2^n$, and define $R_k = 2^n A_k + Q_k$ after round $k$. The exact invariant is

$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)2^{n-k}, \qquad 0 \le k \le n.$$

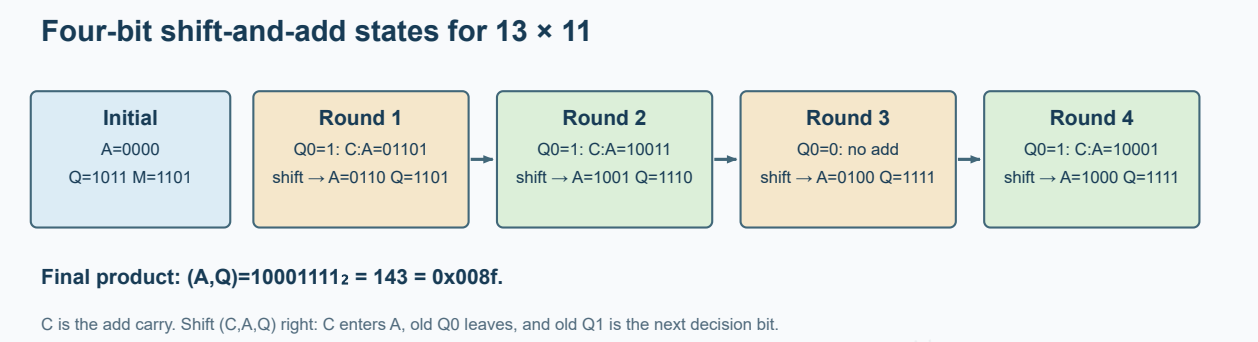
At $k = 0$, $A_0 = 0, Q_0 = q$, so it holds. When $k < n$, the second term is even. The current low bit is therefore original multiplier bit $q_k$. The conditional addition produces full combined value $R_k + q_k M 2^n$, including the carry. Shifting gives

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1})2^{n-k-1}.$$

The equality uses $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$, completing induction. At $k = n$, $q < 2^n$ makes the first term zero, so $R_n = Mq$. A contains the high half and Q the low half. Carry C enters A during the shift and is not lost. The processed old Q0 leaves; old Q1 becomes the next decision bit.



# Sequential multiplication: match the register movements




$$T_{\text{serial}} = t_{\text{setup}} + n t_{\text{iter}} + t_{\text{finalize}}.$$

Thirty-two iterations are 32 cycles only if each add and shift fits one cycle. Separate-cycle operations require a different cost. A parallel tree reduces dependent additions while adding wires, compressors, and power. Pipelining reduces initiation interval without necessarily reducing one operation's latency.

## Signed multiplication starts with a width contract

The exact product of two $n$-bit two's-complement inputs fits in $2n$ signed bits. One method multiplies magnitudes and applies the sign at the end. Booth recoding instead generates signed partial products directly.

**Do not take the absolute value of the most negative input in its original signed width.** Four signed bits represent $[-8, 7]$: the positive magnitude of $-8$ needs five signed bits. An unsigned four-bit container can represent that magnitude as `1000`.

Here radix-2 Booth takes signed $n$-bit multiplicand $M$ and multiplier $Q$. Internally, sign-extend $M$ to $n + 1$ bits, use an $n + 1$-bit accumulator $A$, and retain one history bit $q_{-1}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## What do the two Booth bits decide?

Write the original multiplier as $q_{n-1} \cdots q_0$, append $q_{-1} = 0$, and define $d_i = q_{i-1} - q_i$. Read the pair with **the current low bit first, the history bit second**:

| $Q_0\, q_{-1}$ | d | Action before shifting |
|---|---|---|
| `00` | 0 | Keep A |
| `01` | +1 | A ← A + M |
| `10` | −1 | A ← A − M |
| `11` | 0 | Keep A |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why does the recoding preserve the signed value?

Collect the contributions of each original bit. A positive contribution comes from the next digit and a negative one from the current digit. The top bit has no following positive term, leaving the negative weight required by two's complement.

The recoding preserves the signed value:

$$\begin{aligned}\sum_{i=0}^{n-1}(q_{i-1}-q_i)2^i &= \sum_{i=0}^{n-2} q_i 2^i - q_{n-1}2^{n-1} \\ &= Q_{\text{signed}}.\end{aligned}$$

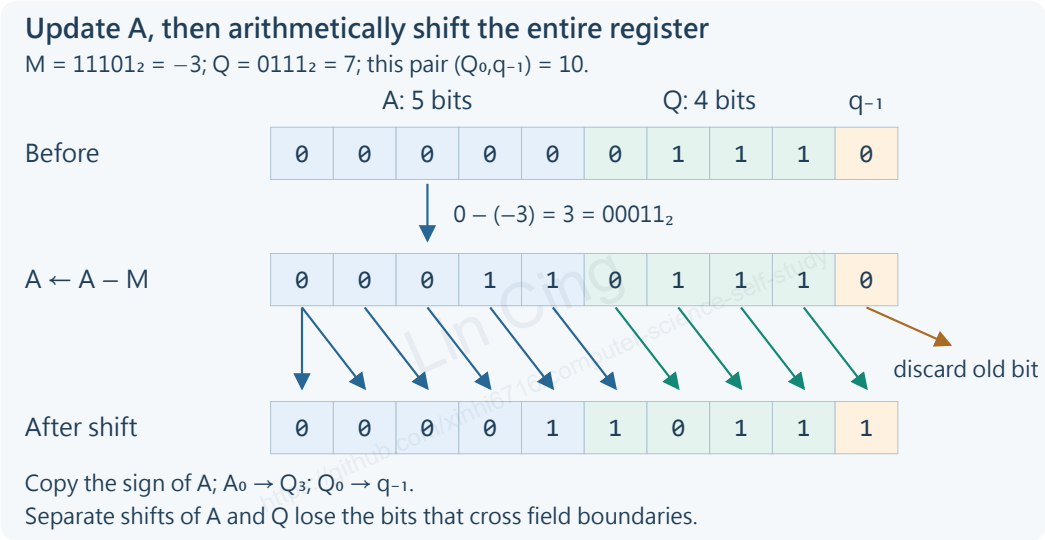
Each intermediate bit contributes $(2^{i+1}-2^i)q_i = 2^i q_i$. The highest bit retains a negative weight, explaining why the rule also handles negative multipliers.

For `0111`, low-to-high digits are −1, 0, 0, +1, giving −1+8=7. For `1101`, they are −1, +1, −1, 0, giving −1+2−4=−3. Individual digits may be negative while their weighted sum equals the original multiplier.



## Add or subtract, then shift the concatenated register

Initialize $A = 0$, load $Q$, and set $q_{-1} = 0$. Repeat $n$ times: update $A$, then **arithmetically shift the entire $(A, Q, q_{-1})$ right by one bit**.




Copy A's sign bit; old $A_0$ enters Q's top bit and old $Q_0$ enters the history bit. After n rounds, drop the history and redundant top sign bit to obtain the 2n-bit product.

## −3 times 7: trace all four rounds

Let $n = 4$, sign-extend $M = -3$ as `11101`, and start with `00000 0111 0`. Each row shifts only after its addition or subtraction; do not use the post-shift pair to select that same row's action.

| Round | Old $Q_0q_{-1}$ | A action | After add/sub: A Q $q_{-1}$ | After shift: A Q $q_{-1}$ |
|---|---|---|---|---|
| 1 | `10` | −M | `00011 0111 0` | `00001 1011 1` |
| 2 | `11` | 0 | `00001 1011 1` | `00000 1101 1` |
| 3 | `11` | 0 | `00000 1101 1` | `00000 0110 1` |
| 4 | `01` | +M | `11101 0110 1` | `11110 1011 0` |

Final $(A, Q) =$`11110 1011` is nine-bit $-21$. Removing the redundant sign bit gives `11101011`, still $-21$ in eight bits.

Original multiplier `0111` has low-to-high digits $(-1, 0, 0, +1)$: $7 = -1 + 8$ and $(-3) \cdot 7 = 3 - 24 = -21$. Only two rounds add or subtract, but all four rounds still shift.



## Booth example: positive multiplicand, negative multiplier

M = 00010 (+2) Q = 1101 (−3) A = 00000 $q_{-1} = 0$

| round | old $Q_0q_{-1}$ | operation | after arithmetic shift: A Q $q_{-1}$ |
|---|---|---|---|
| 1 | 10 | A − M | 11111 0110 1 |
| 2 | 01 | A + M | 00000 1011 0 |
| 3 | 10 | A − M | 11111 0101 1 |
| 4 | 11 | no change | 11111 1010 1 |

Let $n = 4$, $M = 2$, and $Q = -3$. Both $A, M$ use five bits while $Q$ uses four. Each iteration chooses the operation from the old $Q_0q_{-1}$, then arithmetically shifts the combined $(A, Q, q_{-1})$.

After four rounds, $(A, Q) =$ `11111 1010` is −6 in nine bits. Removing the redundant sign bit gives 8-bit `11111010`, agreeing with $2 \times (-3) = -6$.

## The extra sign bit covers the most negative input

For $M = -8, Q = -3$, the first pair is `10`, requiring $A - M = +8$. With only four bits for $A, M$, pattern `1000` instead means $-8$: the first round is already wrong.

### $-8 \times (-3)$: widen before negating the multiplicand

| 4-bit A and M: fails | 5-bit A and M: correct |
|---|---|
| M = $1000_2$ = $-8$ | M = $11000_2$ = $-8$ |
| $-$M = $1000_2$ still means $-8$ | $-$M = $01000_2$ = +8 |
| First A$-$M: 1000, not +8 | First A$-$M: 01000 = +8 |
| After shift: 1100 0110 1 | After shift: 00100 0110 1 |
| After four rounds: $11101000_2$ = $-24$ | After four rounds: $00011000_2$ = +24 |

Both start with A=0, Q=$1101_2$, $q_{-1}$=0; only the A/adder width differs.

Five bits preserve +8 before shifting to +4; the four-bit path shifts −8 to −4. The next page proves the extra bit is sufficient for every input.



## Prove the accumulator width is sufficient

The extra bit has a range proof. After each shift, $A \in [-2^{n-1}, 2^{n-1} - 1]$. Adding or subtracting the $n$-bit $M$ stays within $[-2^n, 2^n - 1]$, which $n + 1$ bits represent exactly. Shifting restores the earlier range. This model does not rely on accidental cancellation of intermediate overflow.

Initially A=0 satisfies the bound. If A starts a round in that range, A+M ranges from $-2^n$ to $2^n - 2$; A−M ranges from $-2^n + 1$ to $2^n - 1$. Keeping A unchanged also fits in the same n+1-bit range.

The next arithmetic shift replaces A by the floor of half the updated value. This restores $[-2^{n-1}, 2^{n-1} - 1]$. Induction proves that every addition or subtraction is exact, including the minimum-negative multiplicand.

## Express the register state as an invariant

Let $u$ be the **unsigned value** of the original multiplier bits. Digits $d_i = q_{i-1} - q_i$ still refer to those original bits. After $k$ rounds define

$$R_k = 2^n \operatorname{signed}(A_k) + \operatorname{unsigned}(Q_k), \quad D_k = \sum_{i=0}^{k-1} d_i 2^i.$$

The invariant is

$$R_k = \left\lfloor \frac{u}{2^k} \right\rfloor + M 2^{n-k} D_k, \qquad 0 \le k \le n.$$

Initially $R_0 = u, D_0 = 0$. For $k < n$, the low $n - k$ bits of $Q_k$ have not been overwritten by incoming accumulator bits. Its low bit is original $q_k$, and the history bit is $q_{k-1}$. Thus the hardware pair really selects $d_k$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## One round preserves the invariant and yields the product

Updating $A$ adds $d_k M 2^n$ to $R_k$. The concatenated arithmetic shift gives

$$R_{k+1} = \left\lfloor \frac{R_k + d_k M 2^n}{2} \right\rfloor.$$

Substitute the invariant. For $k < n$, the term containing $M$ is divisible by two, even when negative. Therefore

$$R_{k+1} = \left\lfloor \frac{u}{2^{k+1}} \right\rfloor + M 2^{n-k-1} (D_k + d_k 2^k).$$

This is the next round's invariant. After $n$ rounds, $0 \le u < 2^n$ makes the first term zero. The recoding identity gives

$$R_n = M D_n = M Q_{\text{signed}}.$$

An arithmetic right shift rounds a negative integer toward negative infinity, not toward zero. The floor in the recurrence accounts for this distinction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Fewer additions need not mean fewer cycles

`0111` needs arithmetic only at the ends of its run of ones. For `0101`, every pair is `10` or `01`, requiring four additions/subtractions; unsigned shift-add needs only two additions. Radix-2 Booth is not cheaper for every input.

In this model each round still controls and shifts the register, so both inputs take $n$ rounds. Converting arithmetic counts into latency or throughput needs a timing contract: zero-run skipping, combined add/shift cycles, and pipelining are separate choices. Two fewer additions do not establish a twofold speedup.

Radix-4 processes two new bits plus one overlapping history bit. It changes partial-product count and selection circuitry; changing only the diagram's shift distance does not specify a radix-4 implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Radix-4 signed-digit table

For even n, read $(q_{2j+1}, q_{2j}, q_{2j-1})$, with $q_{-1} = 0$. Adjacent groups overlap by one bit; sign-extend odd widths first. Equal digits share a row below; all eight patterns are included.

| Three-bit patterns | Digit $c_j$ | Partial-product multiple |
|---|---|---|
| `000`, `111` | 0 | 0 |
| `001`, `010` | +1 | +M |
| `011` | +2 | +2M |
| `100` | −2 | −2M |
| `101`, `110` | −1 | −M |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Where do the radix-4 digits come from?

Combine two adjacent radix-2 terms while retaining the lower position's weight:

$$\begin{aligned} c_j &= d_{2j} + 2d_{2j+1} \\ &= (q_{2j-1} - q_{2j}) + 2(q_{2j} - q_{2j+1}) \\ &= -2q_{2j+1} + q_{2j} + q_{2j-1}. \end{aligned}$$

Thus $Q = \sum_j c_j 4^j$. Binary input bits restrict each digit to −2, −1, 0, 1, or 2. For example, `011` gives 0+1+1=2; `100` gives −2+0+0=−2; `111` gives −2+1+1=0.

Each higher group multiplies the base weight by four. Append a zero history bit to the lowest group; later groups overlap at the preceding group's top bit. For `001111`, read `110`, `111`, `001` from low to high, giving digits −1, 0, 1 and value −1+0×4+1×16=15.



## Radix-4 example: −7 times 15

Use six-bit inputs $M = -7, Q =$`001111`. Form full twelve-bit signed partial products: extend the sign before shifting by the appropriate $4^j$.

| j | Bit positions | Three bits | $c_j$ | $c_j M 4^j$ | Twelve-bit partial product |
|---|---|---|---|---|---|
| 0 | $q_1\ q_0\ q_{-1}$ | `110` | −1 | +7 | `000000000111` |
| 1 | $q_3\ q_2\ q_1$ | `111` | 0 | 0 | `000000000000` |
| 2 | $q_5\ q_4\ q_3$ | `001` | +1 | −112 | `111110010000` |

Since $15 = -1 + 0 \cdot 4 + 1 \cdot 16$, the product is $7 - 112 = -105$, or `111110010111`. Addition uses a twelve-bit container; a carry beyond that width is outside the result, whose signed value is still $-105$.

**Recheck internal width for radix-4.** With the most negative multiplicand, $-2M = 2^n$ needs $n + 2$ signed bits. This section forms partial products in $2n$ bits; it does not claim that the radix-2 $n + 1$-bit accumulator directly supports every radix-4 design.

## Faster multiplication starts with aligned partial products

Use unsigned four-bit inputs M=13 and Q=15. Generate each partial product in its actual bit position, using eight-bit rows throughout.

| Row | Source | Eight bits | Unsigned value |
|---|---|---|---|
| $P_0$ | $M \cdot q_0$ | 00001101 | 13 |
| $P_1$ | $M \cdot q_1 \cdot 2$ | 00011010 | 26 |
| $P_2$ | $M \cdot q_2 \cdot 4$ | 00110100 | 52 |
| $P_3$ | $M \cdot q_3 \cdot 8$ | 01101000 | 104 |

The target is their sum, 195. Repeatedly adding two rows into a conventional binary number can require a full carry propagation each time. Carry-save instead keeps two rows during reduction and converts them into a single result only at the end.



## Three equal-weight bits become two output bits

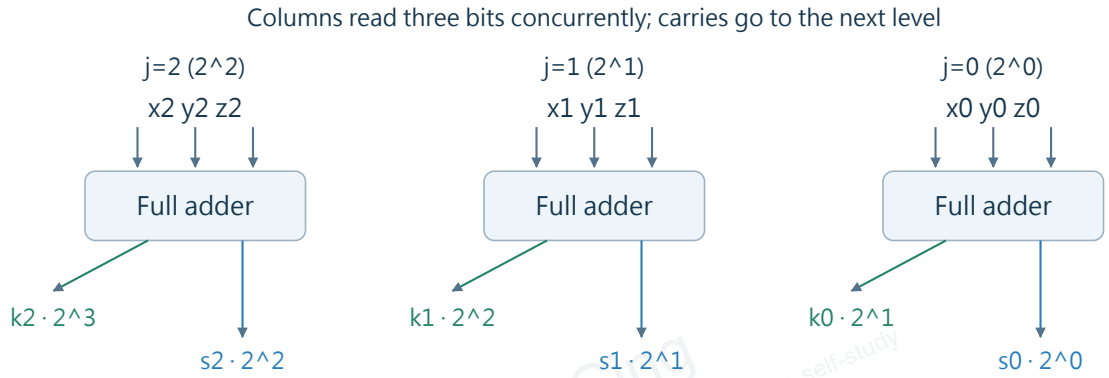
At position j, $x_j$,$y_j$,$z_j$ each have weight $2^j$. A full adder reads these three bits and produces a same-position bit $s_j$ and a higher-weight bit $k_j$:

$$s_j = x_j \oplus y_j \oplus z_j, \qquad k_j = (x_j y_j) \vee (y_j z_j) \vee (z_j x_j).$$

| $x_j+y_j+z_j$ | Input patterns | $s_j$ | $k_j$ | Identity |
|---|---|---|---|---|
| 0 | 000 | 0 | 0 | 0=0+2 · 0 |
| 1 | 001,010,100 | 1 | 0 | 1=1+2 · 0 |
| 2 | 011,101,110 | 0 | 1 | 2=0+2 · 1 |
| 3 | 111 | 1 | 1 | 3=1+2 · 1 |

Multiply each column identity by $2^j$ and sum: X+Y+Z=S+C, with $S = \sum_j s_j 2^j$ and $C = \sum_j k_j 2^{j+1}$. C already includes the one-bit left shift; do not shift it again. Products of bits denote AND; $\vee$ denotes OR.

## A CSA level does not wait for a carry from its neighbor



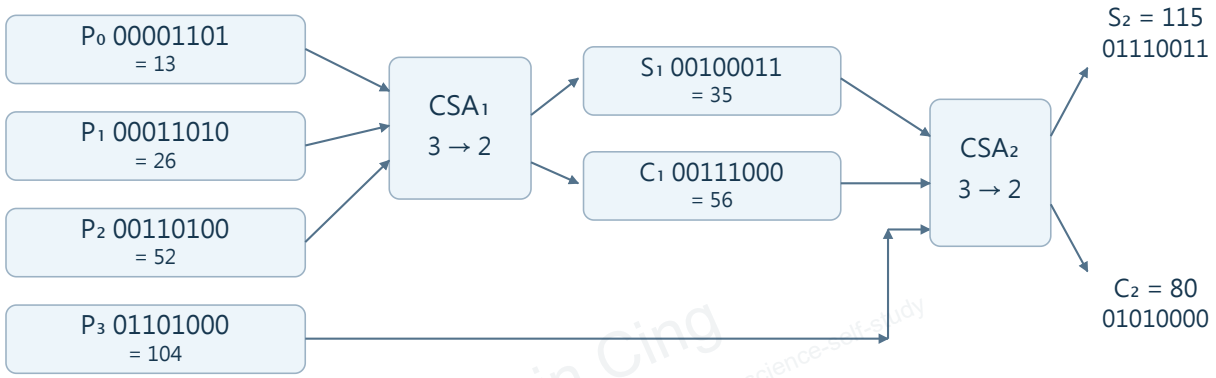

Blue: S row. Green: shifted C row. No carry link between same-level adders.

Each full adder reads the three current-level rows. Its carry goes into the next-level C row, not into its current-level neighbor. The columns evaluate concurrently once their inputs arrive.

The carry remains as data: only S+C represents the total. Adding these rows immediately at each level would reintroduce full carry propagation.



## 13×15: two reduction levels, then one ordinary addition




Total: 13+26+52+104 = 35+56+104 = 115+80 = 195

$CSA_1$ replaces $P_0$,$P_1$,$P_2$ with 35+56; $P_3$=104 bypasses it. $CSA_2$ replaces these three rows with 115+80. Each level preserves 195. $C_1$ and $C_2$ already include their left shifts; reduction neither divides the total nor discards a carry. The final ordinary addition still remains.

## The final addition still has a carry dependency

Now add $01110011_2 + 01010000_2$. Let $c_j$ be the carry entering position j, starting with $c_0$=0:

| Position j, low to high | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $S_2[j]$ | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 |
| $C_2[j]$ | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 |
| Carry-in $c_j$ | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| Result bit | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |
| Carry-out $c_{j+1}$ | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 |

Read result bits from high to low: $11000011_2 = 195$. The carry from position 4 affects positions through 7. A carry-propagate adder (CPA) resolves this dependency, using a ripple or faster prefix structure. Its delay is not zero.



## When may a high carry be discarded?

For three arbitrary unsigned W-bit inputs, S needs W bits, C may need W+1, and the exact total may need W+2. With W=4 and X=Y=Z=15, S=15 and C=30, so the exact total is 45.

If the specified result is only the low W bits, then

$$X + Y + Z \equiv S + (C \bmod 2^W) \pmod{2^W}.$$

Here truncating C to four bits gives 14. Then 15+14=29, whose low four bits are 13, equal to 45 mod 16. A correct modular result does not imply that the exact sum fits.

Unsigned n×n multiplication has an additional bound: all partial products are nonnegative and their total is below $2^{2n}$. Every nonnegative row at every exact reduction level is at most this total, so 2n bits suffice. Booth rows require the declared two's-complement width and sign extension instead. For the earlier twelve-bit example, 7+0+3984=3991 represents −105 in twelve bits. Do not reuse the nonnegative unsigned-partial-product argument for the original signed values.

## Reduction depth is not a count of full-width additions

For a model that groups each three rows into two and passes leftover rows directly to the next level,

$$R_{\ell+1} = 2\lfloor R_\ell/3 \rfloor + (R_\ell \bmod 3).$$

Starting with 32 rows gives 22,15,10,7,5,4,3,2: eight reduction levels. This exact count belongs to the declared grouping model. Actual Wallace/Dadda bit heaps schedule full and half adders column by column.

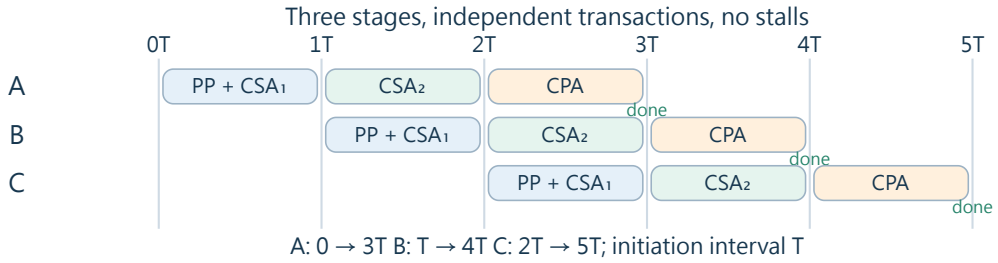
If t_CSA bounds each reduction level, an upper-bound model is

$$T \le t_{\mathrm{PP}} + 8t_{\mathrm{CSA}} + t_{\mathrm{CPA}} + t_{\mathrm{wire}}.$$

t_PP covers partial-product generation; t_wire covers additional wiring delay. Eight CSA levels are not eight W-bit CPAs. A balanced tree of ordinary two-input adders has only five levels for 32 rows, but each level may propagate carries. Comparing 8 with 5 alone says nothing about which circuit is faster.



## Pipeline initiation interval and result latency differ




Stage 1 generates partial products and performs $\mathrm{CSA}_1$; stage 2 performs $\mathrm{CSA}_2$; stage 3 performs the CPA. Each stage meets period T. Assume independent transactions and no stalls.

A enters at edge 0 and completes at edge 3; B enters at 1 and completes at 4. Latency is 3T, initiation interval T; N≥1 transactions finish at $(N+2)T$.

If B consumes A's product, it cannot enter at edge 1. T includes the slowest stage, register and clock costs; these timings alone do not establish speedup over an unpipelined circuit.

## Traditional MIPS multiply instructions

`mult` and `multu` write a 64-bit product into HI and LO. `mfhi` and `mflo` move the halves into general-purpose registers.

Signed and unsigned hardware can share components, but upper product bits have different mathematical interpretations. Traditional `mult` and `multu` do not trap merely because the result exceeds 32 bits.

Taking only LO gives a low modular product. Exactness requires checking the full result against the intended result range.

`multu` interprets input bits as unsigned values. It is not an assertion about source-language types.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.4 Division starts with a quotient and remainder convention

For an unsigned dividend D and nonzero divisor M, require

$$D = QM + R, \qquad 0 \le R < M.$$

The identity and remainder bound together determine a unique answer. For example, 13=2・5+3 and 13=1・5+8 both satisfy the identity, but only the first satisfies the bound.

First use restoring division for unsigned inputs. Signed division additionally needs a rounding convention. Division by zero and the most negative signed input divided by −1 need separate handling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Bring in one input bit, then try one subtraction

If the current remainder is A and the next dividend bit is d, form $T = 2A + d$ and try $T - M$.

| Trial | New quotient bit b | New remainder A′ |
|---|---|---|
| $T - M \geq 0$ | 1 | $T - M$ |
| $T - M < 0$ | 0 | T; add M back if subtraction has already occurred |

Because $0 \leq A < M$ and $d \in \{0, 1\}$, we have $0 \leq T < 2M$. A single subtraction suffices: either branch restores $0 \leq A' < M$. This bound explains why each round emits just one quotient bit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 13 divided by 5: prefix, quotient, and remainder

Read D=1101 from left to right. The quotient column below contains only generated bits, not the entire combined hardware register Q.

| k | New d | Prefix $P_k$ | Trial T | T−5 | Generated quotient | Remainder |
|---|---|---|---|---|---|---|
| 0 | — | 0 | — | — | empty | 0 |
| 1 | 1 | 1 | 1 | −4 | `0` | 1 |
| 2 | 1 | 3 | 3 | −2 | `00` | 3 |
| 3 | 0 | 6 | 6 | 1 | `001` | 1 |
| 4 | 1 | 13 | 3 | −2 | `0010` | 3 |

Every row satisfies $P_k = q_k M + A_k$. Row 3 gives 6=1 • 5+1; the final row gives 13=2 • 5+3 with 3<5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Q mixes unread input and quotient bits

Initialize A=`00000`, Q=`1101`, M=`00101`. Each left shift makes room for one quotient bit.

| Round k | Remainder A (5 bits) | | Q: unread input → quotient bits | $q_k$ | $P_k=q_k\cdot5+A$ |
|---|---|---|---|---|---|
| 0 | 00000 | = 0 | 1 1 0 1 | 0 | 0 |
| 1 | 00001 | = 1 | 1 0 1 \| 0 | 0 | 1 |
| 2 | 00011 | = 3 | 0 1 \| 0 0 | 0 | 3 |
| 3 | 00001 | = 1 | 1 \| 0 0 1 | 1 | 6 |
| 4 | 00011 | = 3 | 0 0 1 0 | 2 | 13 |

Teal: unread dividend bits Orange: quotient bits | Only at k=4 does the whole Q equal quotient 2.

Only the orange portion is the generated quotient $q_k$; the teal portion is unread input. The whole Q equals the quotient only after round n.



# Round 3: shift across the register boundary before writing the bit

After round 2, A=3 and Q=`0100`. The next dividend bit is 0, so shifting forms T=6; subtract M=5.

**1 Before the shift**

A **00011**

Q **0 1 0 0**

A=3; the next input bit is $Q_3$=0

**2 Shift A and Q together**

A **00110**

Q **1 0 0 0**

T=2·3+0=6

Temporarily fill $Q_0$ with 0

**3 Keep T−M; write quotient bit 1**

A **00001**

Q **1 0 0 1**

6−5=1 ≥ 0

A←1, $Q_0$←1

Old $Q_3$ enters $A_0$; other Q bits shift left. The vacated low bit stores the new quotient bit.

The trial is +1, so retain A=`00001` and set $Q_0$=1, giving Q=`1001`. This Q is not a quotient of 9: its high 1 is still the last unread dividend bit.

## Prove the prefix invariant and the remainder bound

Initially $P_0 = q_0 = A_0 = 0$. Suppose $P_k = q_k M + A_k$ and $0 \le A_k < M$. For the next bit d, let $T = 2A_k + d$, with b=1 exactly when T≥M. Then

$$P_{k+1} = 2P_k + d = (2q_k + b)M + (T - bM).$$

Thus $q_{k+1} = 2q_k + b$ and $A_{k+1} = T - bM$; T<2M preserves the remainder bound. After n rounds the prefix is D, proving the required identity and bound.

This implementation uses n+1 signed bits for T and the trial difference. The consumed prefix has at most n bits, and T is no larger than that prefix, so T≤2ⁿ−1. Since M≤2ⁿ−1, the difference is also within [−(2ⁿ−1),2ⁿ−1]. The weaker bound T<2M alone would not justify the same signed width for arbitrary T.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Signed division truncates the quotient toward zero here

Divide magnitudes in unsigned containers, then restore signs. A nonzero remainder takes the dividend's sign.

| D | M | Quotient Q | Remainder R | Check |
|---|---|---|---|---|
| 7 | 2 | 3 | 1 | 7=3 • 2+1 |
| −7 | 2 | −3 | −1 | −7=(−3) • 2−1 |
| 7 | −2 | −3 | 1 | 7=(−3) • (−2)+1 |
| −7 | −2 | 3 | −1 | −7=3 • (−2)−1 |

The bound is $|R| < |M|$, not necessarily R≥0. With floor division, −7/2 instead gives quotient −4 and remainder +1. The identity still holds; the rounding convention differs. A zero remainder needs no sign assignment.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Boundary cases and timing require separate checks

The magnitude $2^{n-1}$ of the most negative signed input fits in n unsigned bits but not in n positive signed bits. Dividing it by −1 produces a positive quotient beyond the signed upper bound.

Traditional MIPS32 `div` and `divu` place quotient in LO and remainder in HI. In this chapter's version, zero division and signed quotient overflow do not automatically cause an arithmetic trap; software must check these inputs rather than depend on HI/LO contents.

The combined-register algorithm here consumes n bits in n rounds. The book's initial wide-divisor algorithm uses n+1 iterations. Do not mix their loop counts. SRT designs may choose multiple quotient bits per round, but selection, normalization, and correction also cost time; two bits per round alone does not prove twice the speed.



## Arithmetic right shift and signed division can disagree

An arithmetic right shift rounds a two's-complement input toward negative infinity; the signed division convention here rounds toward zero.

| Operation on −7 | Result | Rule |
|---|---|---|
| Arithmetic right shift by one | −4 | $\lfloor -7/2 \rfloor$ |
| Signed division by two | −3 | Truncate toward zero |
| Add one, then arithmetic shift | −3 | $\lfloor (-7+1)/2 \rfloor$ |

For division by $2^k$, negative inputs may first receive bias $2^k - 1$, then shift arithmetically. Specify $1 \leq k < n$ and an overflow-safe typed implementation. Reciprocal-multiply replacements must likewise preserve the original quotient rule.

## 3.5 IEEE 754 representation

$$value = (-1)^s \, significand \, 2^e.$$

Binary32 has 1 sign bit, 8 exponent bits, and 23 fraction bits. Normal values use exponent fields 1 through 254, bias 127, and an implicit leading 1.

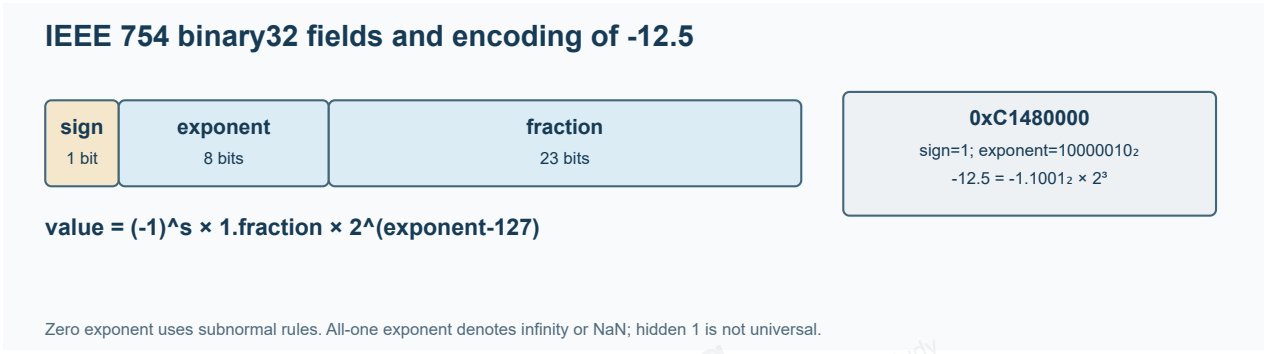
Binary64 has fields of 1, 11, and 52 bits, with bias 1023. Exponent range and significand precision solve different representation problems.

"Floating point" alone does not specify a format.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Floating-point fields: precision includes an implicit bit




For normal values, $(-1)^s (1.F)_2 \, 2^{E-\text{bias}}$. Binary32 has p=24, binary64 p=53. Subnormals use a leading 0; binary32 uses exponent −126. The hidden 1 does not apply when E=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Special values and flags

An all-ones exponent with zero fraction denotes signed infinity. A nonzero fraction denotes NaN. Quiet and signaling NaNs have distinct invalid-operation behavior and payload rules.

IEEE exception classes include overflow, underflow, inexact, invalid, and division by zero. A flag and a language exception are not the same mechanism. Trap controls determine whether ordinary nontrapping continuation occurs. Exception flags are normally accrued until explicitly cleared; clear old flags first when identifying conditions raised by one operation. Overflow is assessed after rounding to the target precision with an unbounded exponent; directed rounding may return the maximum finite value while still raising overflow.

$+\infty + (-\infty)$ is invalid and ordinarily returns quiet NaN. $1/(+\infty) = +0$ under the usual IEEE rules.

Separate the numerical result, status flag, and enabled trap.



## Floating-point addition: alignment precedes rounding

For finite normal inputs, compare exponents and shift the smaller significand to a common scale. Opposite signs require magnitude subtraction; when magnitudes differ, the larger magnitude determines the sign. Exact cancellation has a zero sign determined by rounding direction: +0 in RNE and −0 when rounding toward negative infinity. Information shifted beyond retained precision must survive until rounding.

Normalize the sum or difference, round to the target precision, and renormalize if rounding carries out. Special values, zeros, overflow, and subnormal results need their own paths.

The following reduced model has **p=4 significant bits including the leading 1, RNE, and an unbounded exponent range**. It exposes every intermediate bit; it is not binary32.

## An alignment trace where sticky changes the answer

Let a=1 and b=9/128=0.0703125. Both are exact at p=4. Aligning $b=1.001_2\times2^{-4}$ gives $0.0001001_2\times2^0$.

**p=4 including the leading 1; RNE; unbounded exponent range**

| ×2⁰ | Retain 4 bits | G | R | Tail bits |
|---|---|---|---|---|
| a = 1 | 1.000 | 0 | 0 | 00 |
| b = 9/128 | 0.000 | 1 | 0 | 01 |
| **a + b** | **1.000** | **1** | **0** | **01** |

$b = 1.001_2 \times 2^{-4} = 0.0001001_2 \times 2^0$

**S = OR(0,1) = 1 → GRS = 101 → $1.001_2$ = 1.125**

Replacing the tail by 00 would create a false tie and return $1.000_2$.

The exact sum is 137/128. Its adjacent candidates are 1 and 9/8, with midpoint 17/16=136/128. RNE selects 9/8 with error 7/128. Returning 1 after discarding the tail would give error 9/128, exceeding half an ULP=1/16.



## Cancellation: find the leading 1 again

At p=4, compute exact 0.75−0.6875. After subtraction, shift left three positions.

**p=4, RNE: 0.75 − 0.6875; both inputs are exact**

| | | |
|---|---|---|
| a | **1.100 000** | $\times 2^{-1}$ |
| − b | **1.011 000** | $\times 2^{-1}$ |
| Subtract | **0.001 000** | $\times 2^{-1}$ |
| Shift left 3 | **1.000 000** | $\times 2^{-4}$ |

Each left shift decreases the exponent by one; value is unchanged.

GRS=000; the result 1/16=0.0625 is exact, so inexact=0.

The difference is exact. Cancellation can amplify existing input error without creating rounding error. At the minimum exponent, use subnormals.

## Derive nearest-even from G, R, and S

Work on the result's magnitude. L is the least retained bit, G the next bit, and R the following bit. S is the OR of every remaining bit, summarizing whether the tail is nonzero.

In retained-ULP units, G has weight 1/2, R has weight 1/4, and the tail is below 1/4. G=0 is below midpoint. G=1,R=S=0 is exactly halfway. G=1 with R or S nonzero is above halfway. At a tie, choose the candidate with even retained encoding. Thus

$$\text{inc} = G \wedge (R \vee S \vee L).$$

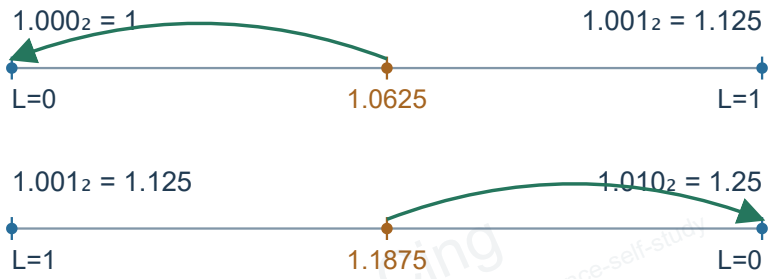
If inc=1, increment the retained **magnitude** by one ULP, then restore the sign. For a negative result this can make the number more negative; it is not always rounding toward $+\infty$. Any nonzero discarded part implies inexact.



## Two midpoints, two parity decisions

Even refers to the significand encoding: the intervals below select opposite endpoints.

**p=4, RNE; each number line shows one adjacent interval**



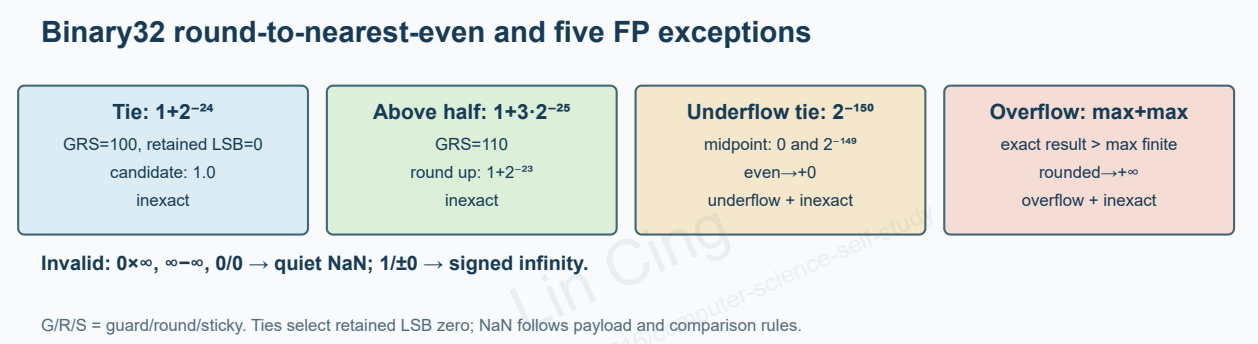

**inc = G ∧ (R ∨ S ∨ L)**

Round the magnitude, then restore the sign: −1.1875 → −1.25.

At p=4, $1.1111_2$ has L=1,GRS=100. Increment: $1.111_2+0.001_2=10.000_2=1.000_2\times2^1$. Renormalization increases the exponent.

## Two binary32 rounding checkpoints

Binary32 has p=24 and spacing $2^{-23}$ on [1,2). For $1+2^{-24}$, L=0 and GRS=100 select 1 (0x3f800000). For $1+3\cdot 2^{-25}$, GRS=110 selects $1+2^{-23}$ (0x3f800001). Both operations set inexact.

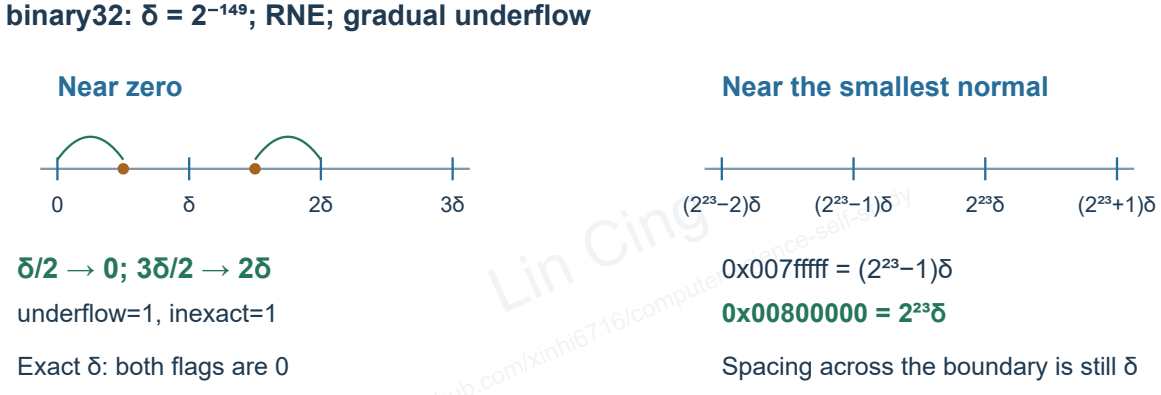



Here $2^{-24}$ is half the adjacent-value spacing. FMA rounds exact ab+c once, whereas separate multiply and add have two rounding points. A later worked example compares their output bits.



## Subnormal representation and underflow flags are different questions

Put $\delta=2^{-149}$. Positive binary32 subnormals are $F\delta$ for fraction integers $1\le F\le 2^{23}-1$. The smallest normal is $2^{23}\delta=2^{-126}$. Spacing remains $\delta$ across this boundary.




**Subnormal does not imply an underflow flag.** Exact $2^{-126}/2$ gives 0x00400000 without error; the two inexact midpoint cases set both flags. The next page specifies the environment.

## Check exactness at the subnormal boundary

Fix RNE, gradual underflow, disabled traps, initially clear flags, and tininess after rounding. Every input is exact in binary32. UF and NX denote underflow and inexact.

| Operation | Exact value | RNE output bits | UF | NX |
|---|---|---|---|---|
| $2^{-126} / 2$ | $2^{-127}$ | 0x00400000 | 0 | 0 |
| $\delta / 2$ | $\delta/2$ | 0x00000000 | 1 | 1 |
| $(3\delta) / 2$ | $3\delta/2$ | 0x00000002 | 1 | 1 |
| Smallest normal − largest subnormal | $\delta$ | 0x00000001 | 0 | 0 |

At the midpoints, even fraction integers win: choose 0 between 0 and 1, and 2 between 1 and 2. Flags record conditions, not error magnitudes. Flush-to-zero or treating subnormal inputs as zero changes this model.

See Oracle Numerical Computation Guide § 2.3 for representation and gradual underflow.



## Special values: report the result and the exception

Maximum finite plus itself returns $+\infty$ under RNE, setting overflow and inexact. The invalid operations $0 \times \infty$, $\infty - \infty$, and 0/0 return quiet NaN when traps are disabled.

Finite nonzero divided by signed zero returns infinity with the XOR of the input signs, setting divide-by-zero. Infinity divided by +0 remains infinity without that flag. See the Oracle Numerical Computation Guide exception table.

NaN comparison depends on the predicate: ordinary equality is false even for a NaN compared with itself; inequality is true. Distinguish ordered and unordered predicates. +0 and −0 compare equal, but reciprocal and signbit distinguish them.

## Floating-point multiplication

For normal inputs, multiply $(1.F_a)(1.F_b)$, add unbiased exponents, and XOR signs. The exact significand product lies in $[1, 4)$. Normalize and round it to target precision.

Subnormal inputs may require normalization before multiplication. A tiny result can enter the gradual-underflow path. Special combinations are handled first. In particular, $0 \times \infty$ is invalid.

Multiplication avoids exponent alignment but needs a wide significand product and final rounding.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## MIPS floating-point instructions

The book's model uses coprocessor-1 registers. `lwc1` and `swc1` move words. `add.s` and `add.d` select single and double formats.

Comparisons set condition information examined by instructions such as `bc1t` and `bc1f`, with the traditional control-transfer delay slot. NaNs require ordered and unordered comparison semantics.

Floating-point and general-purpose register spaces differ. The compiler must respect register format and saved-area requirements.

These encodings and register rules are not interchangeable with ARM or RISC-V.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Accuracy and rounding modes

Nearest-even is a common default. Directed modes round toward positive infinity, negative infinity, or zero. Directed rounding can support interval bounds.

An ULP is the spacing between adjacent representable values in the relevant binade. Small absolute error does not imply small relative error. Spacing grows with exponent.

Significand width controls precision. Exponent width controls range. Adding exponent bits does not add significand accuracy.

Compare binary32 and binary64 spacing at the same value rather than comparing their largest values alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## NaNs, signed zero, and comparisons

NaN is unequal to every value under ordinary equality, including itself. Use `isnan` or an unordered comparison to detect it.

IEEE equality treats positive and negative zero as equal, but reciprocals, sign-copy operations, and serialization can expose their different sign bits.

$\infty - \infty$, $0 \times \infty$, and $\infty / \infty$ are invalid. Signed integer ordering of raw bits cannot implement ordinary floating-point ordering across negative values and NaNs.

A comparison algorithm must specify its treatment of NaNs and signed zeros.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.6 Parallelism and nonassociativity

Integer addition remains associative modulo a fixed power of two. Floating-point rounding can instead make

$$(a + b) + c \neq a + (b + c).$$

A changed reduction tree changes rounding points. Different answers can therefore be a numerical reproducibility issue without any data race. Bitwise reproducibility requires a fixed order and matching formats, rounding, intermediate precision, and FMA contraction, or an accumulator with an explicit order-independent guarantee. Compensation and higher precision improve error but do not alone guarantee order independence.

For standard Kahan summation with each operation rounded to binary32 RNE, `[2^25,1,-2^25]` gives 0, while `[2^25,-2^25,1]` gives 1. In the first order, adding 1 leaves the sum at $2^{25}$ and compensation at -1. Then $-2^{25} + 1$ is a midpoint and rounds back to $-2^{25}$, producing zero. The second order cancels exactly before adding 1. Compensation has not removed dependence on order or environment.



## Changing grouping changes rounding points

At $x = 2^{24}$, the binary32 ULP is 2. `x+1` is the midpoint between x and x+2, and the even candidate is x:

$$(x + 1) - x = 0, \qquad (x - x) + 1 = 1.$$

These expressions agree over exact reals but differ under per-operation rounding.

## FMA rounds the exact multiply-add only once

Let $a = b = 1 + 2^{-23}$, bits `0x3f800001`, and $c = -(1 + 2^{-22})$, bits `0xbf800002`.

$$ab = (1 + 2^{-23})^2 = 1 + 2^{-22} + 2^{-46}.$$

Separate multiply and add first round the product to $1 + 2^{-22}$, then cancel with c to +0. A fused operation computes exact $ab + c = 2^{-46}$ before its sole rounding, yielding `0x28800000`. It changes the intermediate rounding, not real algebra. The old MIPS core does not guarantee FMA; an extension must specify format and rounding mode.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.7 The x86 floating-point case

The book compares x87 extended-precision stack registers with SSE2 XMM scalar and packed operations. Keeping an intermediate wider than its memory format can differ from rounding every operation to that format.

ABI rules, compiler flags, spills, and precision controls affect the rounding points. SIMD operations also require correct lane and alignment semantics.

Do not silently substitute modern AVX behavior for a historical x87 example. State the actual arithmetic environment.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.8 Arithmetic fallacies

Approximation has exact rules. Applications can depend on rounding and exception flags.

Fast-math transformations can permit reassociation or assumptions about NaNs and signed zero. They can change reproducibility and numerical behavior.

There is no universal fixed ratio between divide and multiply costs. Operand dependence, pipelines, specialization, and instruction mix all matter.

Pair each performance claim with a specified operation and timing model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.9 Arithmetic synthesis

Integer width, overflow, and modular behavior must be specified together. Multiply/divide iteration count does not by itself determine latency or throughput.

A floating-point unit implements representation, arithmetic, and rounding with exceptions. Addition emphasizes alignment and lost-bit information. Multiplication emphasizes the significand product and normalization.

Verification needs normal values, zeros, subnormals, infinities, NaNs, overflow, underflow, and halfway cases.

Ordinary exact sums alone do not exercise the rounding contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Numerical history

Earlier machines used incompatible exponent widths, biases, and representations. IEEE 754 standardized formats, rounding, and special values to make numerical behavior more predictable.

Hardware evolved through iterative arithmetic, recoding, compression trees, pipelined floating point, and SIMD. This did not eliminate numerical analysis. Ill-conditioning, large dynamic range, and reproducible parallelism remain algorithmic concerns.

Separate format, arithmetic hardware, rounding, and numerical algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## End-of-chapter check

Determine signed and unsigned overflow, trace small multiply and divide states, decode binary32 special cases, and justify guard/round/sticky decisions. Give a counterexample to floating-point associativity.

Use exercises and solutions separately. Show representation and state transitions, not only the final decimal number.

A bit-level trace makes both the arithmetic policy and its boundary cases inspectable.

### References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 3. CSO course slides, arithmetic.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## From an ISA to state transitions

An ISA specifies the state an instruction may observe and change. A microarchitecture chooses the cycles, functional units, and energy used to implement that contract. This chapter builds a single-cycle subset, a five-stage pipeline, hazard controls, and exception recovery before introducing multiple issue and speculation.

For each operation, identify the source register, destination state, and the cost of an incorrect transition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 4.1 Design scope

The datapath contains storage, movement, and arithmetic resources. Control derives selectors and write enables from instructions, flags, and requests. PC, registers, and memory hold state.

The subset supports `lw`, `sw`, `add`, `sub`, `and`, `or`, `slt`, `beq`, and `j`. Immediate arithmetic, byte accesses, multiply, floating point, and complete exception support require additional logic.

Pipeline timing questions use a simplified subset without architectural branch delay slots. A wrong-path instruction is squashed. This deliberately differs from the delayed pre-R6 assembly in Chapter 2 and Appendix B. Their flush counts and return rules cannot be substituted here.

Mark every bus width. A MIPS32 word is 32 bits; sequential PC advance is four bytes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## An implementable instruction contract

Specify source operands, destination state, memory accesses, target calculation, exception behavior, and adjacent read/write dependencies.

`lw rt, imm(rs)` reads the base, sign-extends the displacement, adds an address, obtains data, and writes `rt`. A following consumer must wait for that data or receive it through an appropriate bypass. An address is not the loaded value.

Correctness concerns final architectural effects rather than a plausible-looking waveform.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 4.2 Clocking and edge-triggered state

Combinational outputs respond after propagation delay. An edge-triggered register samples only at its active edge. A combinational feedback path cannot stand in for several clocked updates.

$$T_{clk} \geq T_{cq} + T_{logic,max} + T_{setup} + T_{skew}.$$

Here clock-to-Q, maximum logic delay, setup requirement, and a conservative adverse clock-skew allowance all have time units. Hold correctness instead constrains the fastest path. A maximum-delay check does not prove hold timing.

Physical timing depends on process, voltage, temperature, wires, and clock distribution. A gate is not intrinsically one cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 4.3 Datapath building blocks

PC, adders, instruction memory, data memory, a register file, an ALU, and multiplexers form the teaching datapath. The register file provides two reads and one write. Reads are combinational under the chosen timing model; writes occur at an edge.

Separate instruction and data memories permit IF and MEM concurrently. A unified single-port memory would introduce a conflict. Additional ports or different staging are alternative implementations.

The 32 architectural registers include `$zero`, whose reads return zero and whose writes are discarded.

Count accesses and specify read/write timing before deciding whether a structural hazard exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## ALU control and main control

Main control generates `RegDst`, `ALUSrc`, `MemtoReg`, `RegWrite`, `MemRead`, `MemWrite`, `Branch`, and `Jump`. ALU control combines the instruction class with the R-format function field.

| ALUOp | Class | Operation |
|---|---|---|
| 00 | Load/store | Address addition |
| 01 | Branch equal | Subtraction or equality comparison |
| 10 | R-format | Function decode |

A truth table or ROM can implement control. Its propagation delay also belongs in the timing analysis.

Selecting addition alone does not complete an instruction. The destination selector and write enable must also be correct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Single-cycle dataflow

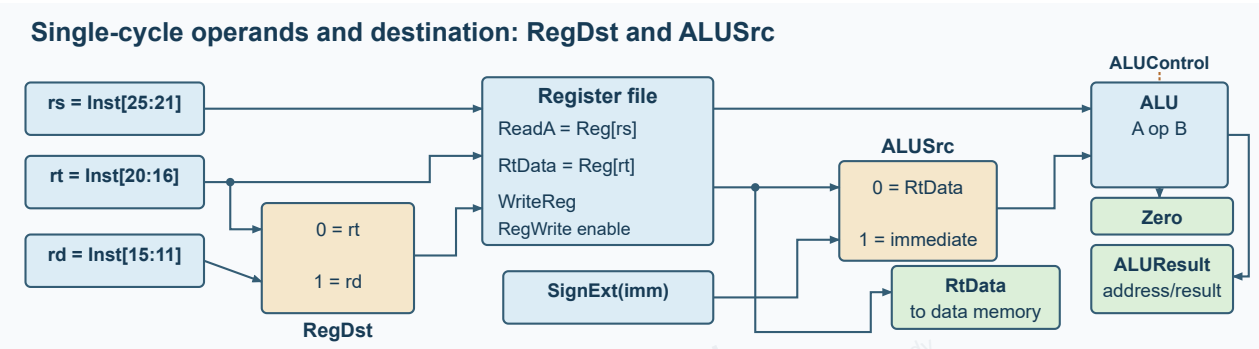
This teaching subset has no architectural delay slot. It supports R-type ALU operations, `lw`,`sw`,`beq`,`j`, separate instruction/data memories, combinational reads, and edge-committed PC/register/store writes. It has no multicycle IR/A/B/MDR state or shared-memory IorD selector.

PC.Q addresses instruction memory. Its fields feed decoding and register reads in parallel. Only enabled state updates commit. Instructions do not all traverse a single PC→IM→Reg→ALU→DM→WB chain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



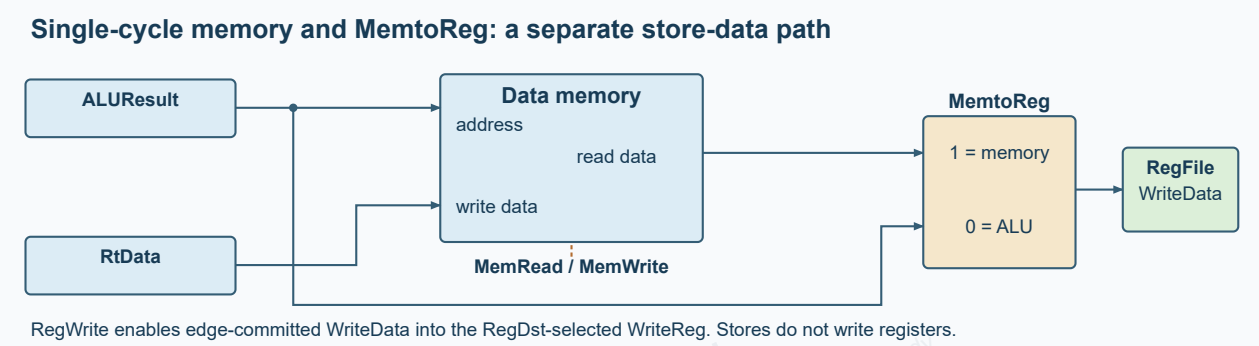
# Single-cycle dataflow (continued)




RegDst=0 selects rt at bits 20:16; RegDst=1 selects rd at 15:11. ReadA=Reg[rs] and RtData=Reg[rt]. ALUSrc=0 selects RtData for ALU input B; ALUSrc=1 selects sign-extended immediate. Input A is ReadA. RtData independently supplies store data. Selecting an immediate for address arithmetic must not replace the store value.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Memory and write-back selection



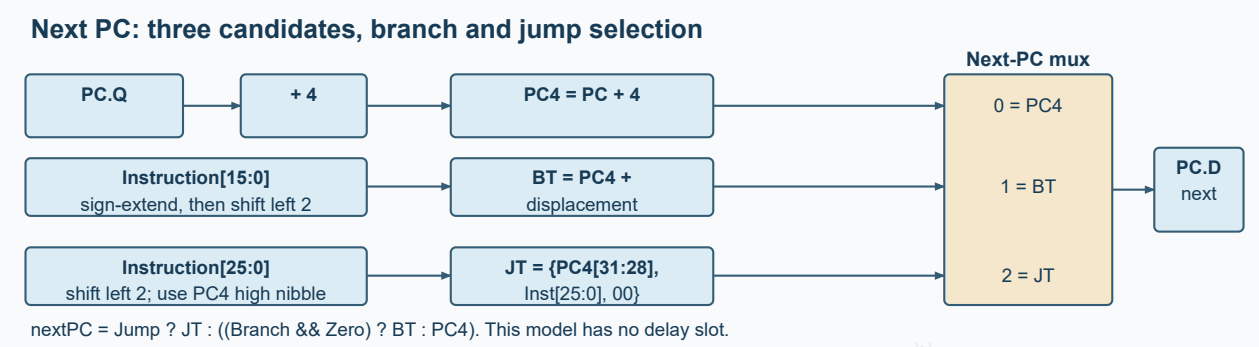

For lw/sw, ALUResult is the effective address; for R-type it is the arithmetic result. MemWrite gates the edge-committed memory write from RtData. MemRead supplies the addressed value to MemtoReg.

MemtoReg=0 selects ALUResult; =1 selects memory data. RegWrite gates WriteData into the RegDst-selected WriteReg. A destination of zero leaves `$zero` unchanged. Identically named endpoints across figures are wires of the same cycle, not extra pipeline registers.

For `sw $t2,-4($s0)`, with s0=0x1000, t2=0x12345678, ALUSrc=1 yields address 0x0ffc, while store data stays 0x12345678. MemWrite=1 and RegWrite=0. Connecting the ALU's immediate input to memory data would incorrectly store -4.



# Three next-PC candidates




$$PC4 = (PC + 4) \bmod 2^{32}, \qquad BT = (PC4 + (\text{signext}(imm) \ll 2)) \bmod 2^{32}.$$

A beq subtracts ReadA−RtData; Zero tests equality. Jump target JT concatenates PC4[31:28], instruction[25:0], and 00. It does not use the old PC's high bits.

```
nextPC = Jump ? JT : ((Branch && Zero) ? BT : PC4)
```

Two two-input muxes can select branch/sequential first and jump second. The next edge commits nextPC; this subset executes no delay slot. Every PC4 reference belongs to the same cycle.

## Three next-PC candidates (continued)

At PC=0x00400020, immediate=-2, PC4=0x00400024 and BT=0x0040001c. Equal operands select BT; unequal operands select PC4. At PC=0x0ffffffc and jump field zero, PC4=0x10000000 and JT=0x10000000. Using the old PC's high nibble would wrongly produce zero.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Control table and mux selections

| Signal | lw | sw | R-type | beq | j |
|---|---|---|---|---|---|
| RegDst | 0 | X | 1 | X | X |
| ALUSrc | 1 | 1 | 0 | 0 | X |
| MemtoReg | 1 | X | 0 | X | X |
| RegWrite | 1 | 0 | 1 | 0 | 0 |
| MemRead | 1 | 0 | 0 | 0 | 0 |
| MemWrite | 0 | 1 | 0 | 0 | 0 |
| Branch | 0 | 0 | 0 | 1 | 0 |
| Jump | 0 | 0 | 0 | 0 | 1 |
| ALUOp | 00 | 00 | 10 | 01 | X |

ALUOp 00 selects address addition, 01 subtraction for equality, and 10 funct decoding for add/sub/and/or/slt. X is a don't-care only when its output cannot commit. RegWrite and MemWrite must remain correct. Illegal opcodes and exceptions need additional control outside this subset.

## Complete state effects for five instruction classes

Each row independently starts at PC=0x00400020, s0=0x1000, s1=7, s2=5, t2=0x12345678, and Mem[0x100c]=0xabcdef01. Word accesses are aligned and no unsupported overflow occurs.

| Instruction | Selected data | State committed at edge |
|---|---|---|
| `addu t0,s1,s2` | ReadA=7, RtData=5; ALUSrc 0; ALU=12; MemtoReg 0; RegDst 1 | t0=12, PC=0x00400024 |
| `lw t1,12(s0)` | ReadA=0x1000, imm=12; ALU=0x100c; MemtoReg 1; RegDst 0 | t1=0xabcdef01, PC=0x00400024 |
| `sw t2,-4(s0)` | ALU=0x0ffc; RtData=0x12345678 goes directly to memory | Mem[0x0ffc]=0x12345678, PC=0x00400024 |
| `beq s1,s2,-2` | ALU=7−5=2, Zero 0, Branch 1, Jump 0 | no register/data writes, PC=0x00400024 |
| `j 0x00401000` | field=0x00100400, Jump 1 | no register/data writes, PC=0x00401000 |

Changing only both branch inputs to 5 sets Zero and next PC=0x0040001c. The rt field is a destination for lw and a source for sw/beq.



## The slowest path sets the common clock

The load path includes PC clock-to-Q, instruction read, register read, address ALU, memory read, MemtoReg, and setup:

$$T_{clk} \geq T_{PCQ} + T_{IM} + T_{regread} + T_{ALU} + T_{DM} + T_{mux} + T_{setup}.$$

The stated combined delays sum to $30 + 200 + 120 + 180 + 250 + 20 + 40 = 840$ ps. The simplified R-type path is $30 + 200 + 120 + 180 + 40 = 570$ ps but still uses the common 840 ps period. Unlisted wires/control/mux costs must already fit the combined terms; this is not timing closure for a physical chip.

## Single-cycle performance and limits

In a separate timing model from the previous 840 ps budget, with period 800 ps and CPI 1, a mix of 60% arithmetic, 25% loads/stores, and 15% control transfers takes

$$T = IC \times 800 \text{ ps}.$$

Shortening a noncritical arithmetic path does not change this time if the load path still determines the clock. Extending the critical load path slows every instruction. Multicycle control and pipelining avoid requiring all instructions to fit one long common period.

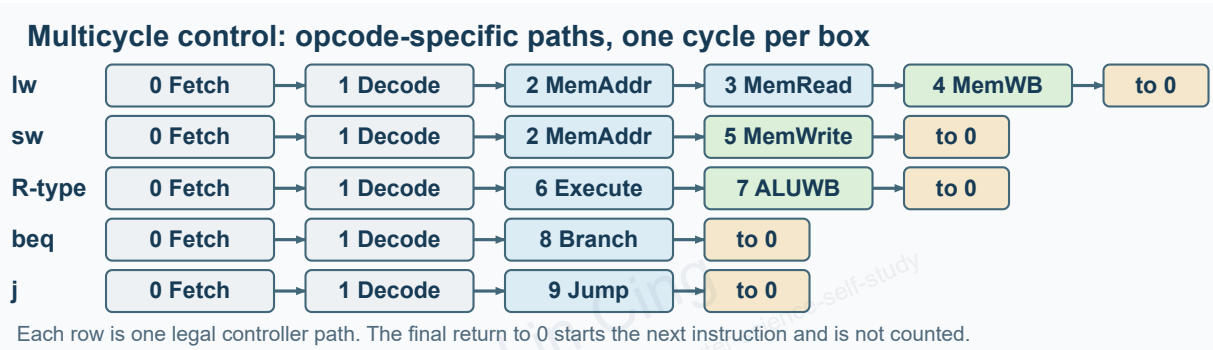
A local reduction in logic delay matters only if it changes the limiting path or resource.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Multicycle datapath and control

Internal IR, A, B, ALUOut, MDR retain the instruction, operands, ALU result, and memory data between cycles. PC tracks the architectural instruction sequence.




Each row is one legal controller path. The final return to 0 starts the next instruction and is not counted.

| Instruction | State path | Cycles |
|---|---|---|
| R-type | Fetch → Decode → Execute → ALUWB → Fetch | 4 |
| lw | Fetch → Decode → MemAddr → MemRead → MemWB → Fetch | 5 |
| sw | Fetch → Decode → MemAddr → MemWrite → Fetch | 4 |
| beq | Fetch → Decode → Branch → Fetch | 3 |
| j | Fetch → Decode → Jump → Fetch | 3 |

The last Fetch starts the next instruction. A store never passes through MemRead.

## Multicycle datapath and control (continued)

| State | Edge effect | Enables and selection |
|---|---|---|
| 0 Fetch | IR←Memory[oldPC]; PC←oldPC+4 | IorD 0, MemRead 1, IRWrite 1, PCWrite 1 |
| 1 Decode | A←Reg[rs]; B←Reg[rt]; ALUOut←PC+branch displacement | no architectural register/memory write |
| 2 MemAddr | ALUOut←A+signext(imm) | base/immediate ALU inputs |
| 3 MemRead | MDR←Memory[ALUOut] | IorD 1, MemRead 1 |
| 4 MemWB | Reg[rt]←MDR | RegDst 0, MemtoReg 1, RegWrite 1 |
| 5 MemWrite | Memory[ALUOut]←B | IorD 1, MemWrite 1 |
| 6 Execute | ALUOut←A op B | ALUOp selects funct decoding |
| 7 ALUWB | Reg[rd]←ALUOut | RegDst 1, MemtoReg 0, RegWrite 1 |
| 8 Branch | if A=B, PC←saved target | PCWriteCond 1, PCSrc=branch; ALU compares A−B |
| 9 Jump | PC←{currentPC[31:28], IR[25:0], 00} | PCWrite 1, PCSrc=jump |



## Multicycle datapath and control (continued)

Unlisted architectural enables are zero. Decode dispatches lw/sw→2, R-type→6, beq→8, j→9; other opcodes use an explicit error transition. State 3 goes only to 4, never 5.

Fetch reads Memory[oldPC] and increments PC at the same edge. Decode therefore uses the already incremented PC for its target and must not add another four. For oldPC=0x00400008, imm=-2, fetch leaves PC=0x0040000c and target=0x00400004. A repeated increment wrongly gives 0x00400008. A not-taken branch leaves sequential PC unchanged. Branch/jump selection cannot also assert unconditional sequential update.

For a mix of 45% R-type, 20% branch, 20% load, 15% store,

$$CPI = 0.45(4) + 0.20(3) + 0.20(5) + 0.15(4) = 4.0.$$

$$T = 10^9(4.0)(300\ \text{ps}) = 1.2\ \text{s}.$$

The comparison single-cycle design at 800 ps and CPI 1 takes 0.8 s. Shorter clocks do not guarantee a win when extra cycles outweigh the reduction. Hold the delay budget and mix fixed.

Two FSM invariants are that an instruction never commits RegWrite/MemWrite twice in the same state, and completion retains all necessary operands/results before returning to Fetch. Undefined opcodes must not default to an arbitrary memory write.

## 4.5 The five-stage pipeline

IF fetches and forms PC+4. ID decodes, reads registers, and forms immediates/control. EX computes ALU results, addresses, or branch equality. MEM accesses data memory. WB writes enabled register results.

| Pipeline register | Required saved state |
|---|---|
| IF/ID | valid, instruction PC, instruction, PC4 |
| ID/EX | valid, PC/PC4, rs/rt/rd, A/B, sign-extended immediate, controls |
| EX/MEM | valid, PC, ALU result, store data, destination, MemRead/MemWrite/RegWrite/MemtoReg |
| MEM/WB | valid, PC, memory data or pass-through ALU result, destination, RegWrite/MemtoReg |

These are edge-updated snapshots. A destination or RegWrite omitted from EX/MEM cannot be reconstructed at WB. A load's EX result is only an address; its value arrives at the end of MEM.



## 4.5 The five-stage pipeline (continued)

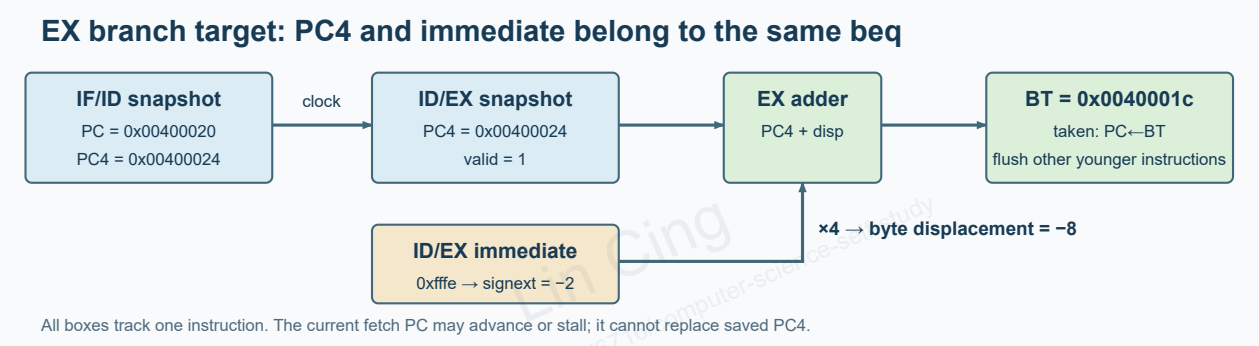
PC/valid, operands, and controls belong to the same instruction and must hold or advance together. Bubble/flush sets valid=0, and valid gates all register/memory writes. A load-use stall holds PC and IF/ID, inserts an invalid ID/EX item, and lets older entries advance. Per-instruction PC identifies the correct EPC for faults. Keeping both PC and PC4 is a teaching choice; PC can also be recovered as PC4−4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## The branch target in EX

IF's PC4 travels through IF/ID and ID/EX. EX must not use the global PC, which already fetches younger instructions.

$$BT = (\mathrm{ID/EX.PC4} + 4\,\mathrm{signext}_{16\to32}(imm_{16})) \bmod 2^{32}.$$




A branch at 0x00400020 with bits 0xfffe retains PC4=0x00400024. Signed immediate -2 gives -8 bytes and BT=0x0040001c. Taken selects BT; not-taken keeps the sequential path. A younger stall/flush does not change this saved PC4. Subtracting a fixed amount from the current fetch PC is not equivalent.



## Ideal time without hazards

For balanced stage time t and k=5,

$$C_{ideal} = k + (n - 1).$$

A steady-state result completes every cycle but each instruction still takes 5 t. A hundred instructions need 104 cycles. In the long-run model below, each penalty is an actual extra cycle per retired instruction, with no overlap or double counting:

$$CPI_{pipe} = 1 + stall_{loaduse} + mispredict_cycles + structural_bubbles.$$

## Ideal pipeline time

For $n$ instructions and $k$ balanced stages, absent hazards and register overhead, completion requires $k + n - 1$ cycles. With five stages and 100 instructions this is 104 cycles.

If the unpipelined instruction takes five equal stage-time units, the time speedup is $500/104 \approx 4.81$. It approaches five for a long stream. Individual instruction latency remains five stages; throughput improves by overlap.

Comparing cycle counts without comparing cycle durations is not a valid physical speedup.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Structural hazards

Two active instructions may need the same exclusive resource. A single memory port cannot simultaneously serve an instruction fetch and a data access.

Register files can permit WB writes and ID reads in one cycle under an explicit write-before-read convention or bypass. Multiple issue increases operand-port demand. Extra ALUs and ports reduce conflicts at area and power cost.

A resource conflict is determined by simultaneous demand, not by the resource's name.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Data dependencies

```
add $8, $9, $10
sub $11, $8, $12
```

The subtraction has a true read-after-write dependence. Scheduling and forwarding reduce waiting but cannot remove the need for the produced value.

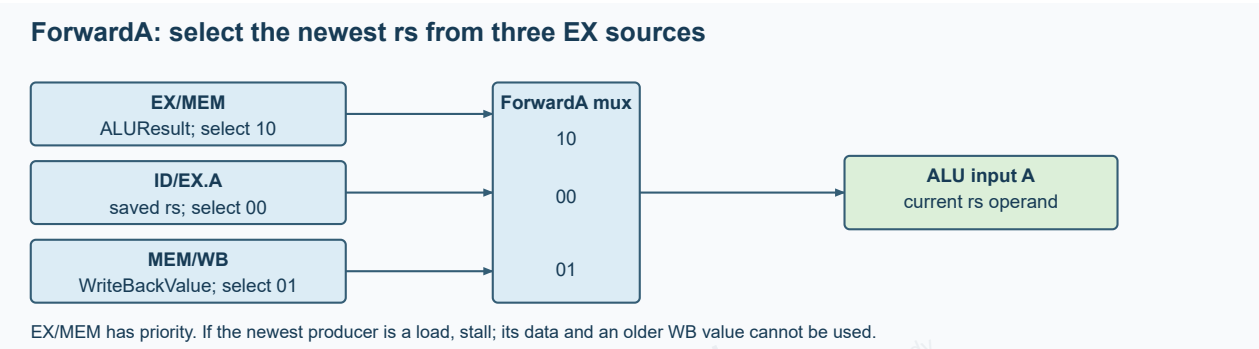
```
add $8, $9, $10
sub $8, $11, $12
```

This pair instead illustrates output-name reuse. Anti-dependencies similarly arise when a later write reuses an earlier source name. Register renaming can remove these false dependencies by assigning different physical registers.

Track the producer of a value rather than treating every repeated architectural name as a true dependence.



## Correct forwarding paths




Produced EX/MEM or MEM/WB values can bypass the register file. An ALU consumer immediately after its producer uses EX/MEM. Every comparison checks producer RegWrite, nonzero destination, source use, matching register number, and availability. A load in EX/MEM contains an address rather than loaded data.

For an EX-used source s, define E as `EX/MEM.RegWrite && EX/MEM.rd!=0 && EX/MEM.rd==s` and W as the equivalent MEM/WB match.

## Correct forwarding paths (continued)

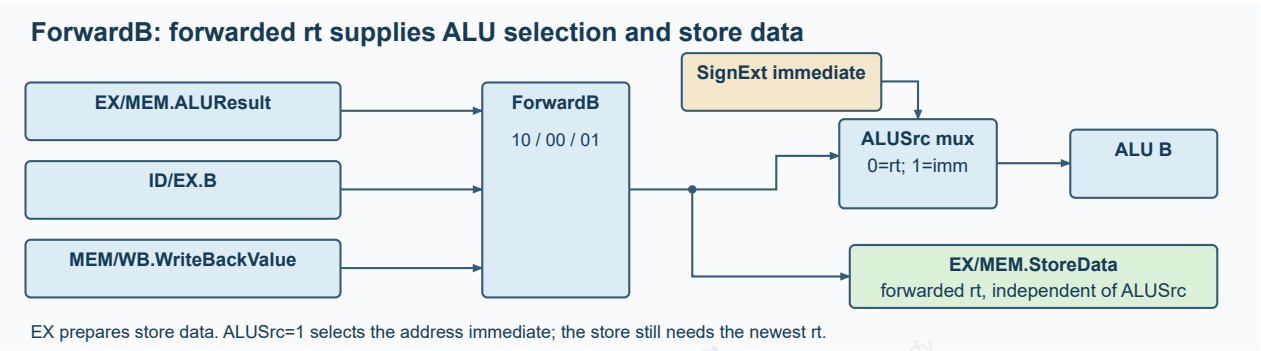
| Condition | ALU input |
|---|---|
| E and producer is not load | EX/MEM.ALUResult, code 10 |
| not E and W | MEM/WB.WriteBackValue, code 01 |
| not E and not W | ID/EX saved value, code 00 |
| E and producer is load | stall; neither its address nor an older WB value is correct |

ForwardA applies to rs. ForwardB applies to the register operand rt, then ALUSrc chooses that value or the immediate. This model prepares store data in EX; an immediately preceding load therefore conservatively requires a bubble even for a store-data dependence. A separate MEM store-forwarding path can remove it only with stated wiring and conditions.

If two producers write s, EX/MEM is newer and wins. In `add $1,...; sub $1,...; and $2,$1,$3`, the AND needs the SUB result.

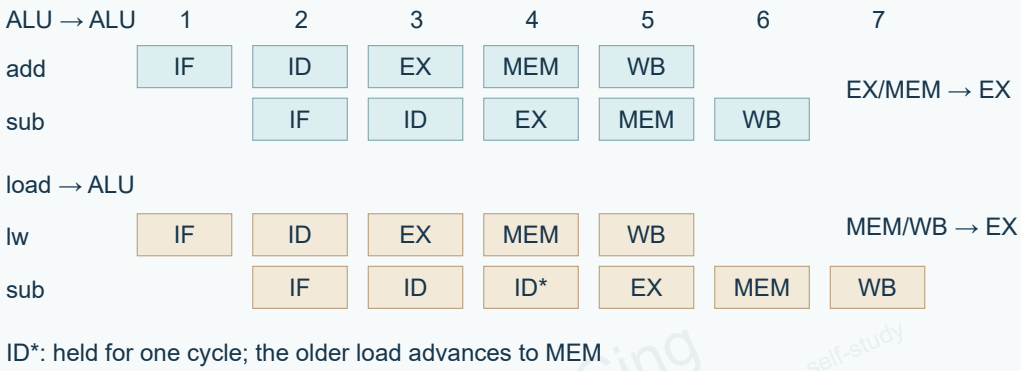


## Store forwarding and ALUSrc




ALUSrc selects only the second ALU operand. A store computes base plus immediate while independently carrying the newest rt data. ForwardB branches to the ALUSrc register input and EX/MEM.StoreData. These cannot be replaced by one common mux selection.

## Two dependencies: why does only one stall?




Assume a five-stage single-issue machine, one-cycle memory, EX/MEM and MEM/WB forwarding to EX, and no other hazards. For `add $s0,$t0,$t1; sub $t2,$s0,$t3`, the add result is ready at the end of EX in C3 and forwards from EX/MEM to sub in C4: no stall.

For `lw $t0,4($s0); sub $t2,$t0,$s1`, data is ready only at the end of MEM in C4, too late for EX at the beginning of that cycle. Hold PC and IF/ID for one cycle and insert a bubble; sub uses MEM/WB in EX at C5.



## Load-use stalls

```
lw  $2, 20($1)
and $4, $2, $5
```

The load obtains data at MEM's end. The adjacent consumer would otherwise need it in EX in that same cycle. One bubble is required in this model.

Hold PC and IF/ID, clear ID/EX side-effect controls, and let the producer advance. The following cycle forwards the data from MEM/WB. Only genuine source operands should trigger this check. A load's `rt` is a destination, whereas a store's `rt` supplies store data.

Store-data timing can admit a later MEM-stage bypass. State which bypasses exist before declaring a store stall necessary.

For this subset, use_rs/use_rt describe operands needed in EX, including store data prepared there. Both are 1 for R-type/beq/sw; lw uses only rs; j uses neither. A load's destination is ID/EX.rt.

## Load-use stalls (continued)

```
stall = ID/EX.MemRead && ID/EX.rt != 0 &&
        ((use_rs && ID/EX.rt == IF/ID.rs) ||
         (use_rt && ID/EX.rt == IF/ID.rt))
PCWrite = !stall
IFIDWrite = !stall
IDEXControls = stall ? all_zero : decoded_controls
```

`lw $2,0($1); lw $2,4($3)` has no dependence through rt because the second load overwrites it. Loading `$zero` creates no register dependency, although its memory access may still fault. Larger ISAs need per-opcode source-use stages rather than treating every encoded field as a read.



## A numeric trace: an older value cannot replace an unready one

Use the five-stage, single-issue, in-order model with no cache misses, branches, or structural conflicts. EX/MEM and MEM/WB bypasses feed EX operands. Load data becomes available at the end of MEM for use in the next cycle. WB writes the register file in the first half of a cycle; ID reads it in the second half.

Initially, `$1=0x1000`, `$6=0x30`, `$7=0x03`, `$5=0x0f`, `$9=0x80`, and `Mem[0x1014]=0x3c`. Other relevant registers are zero. Accesses are aligned and no exceptions occur.

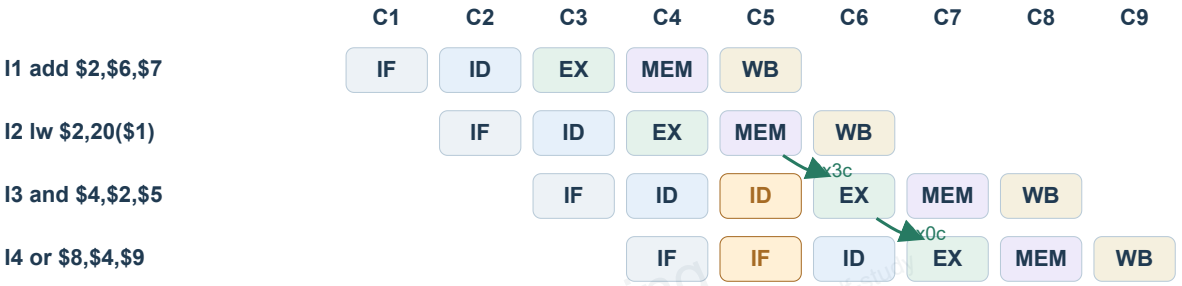
```
I1  add $2,$6,$7     # $2 <- 0x33
I2  lw  $2,20($1)    # $2 <- 0x3c, replacing I1's value
I3  and $4,$2,$5     # $4 <- 0x0c
I4  or  $8,$4,$9     # $8 <- 0x8c
```

I3 needs I2's most recent definition of `$2`, not I1's older value. The RAW chain is `I2 → I3 → I4`. I1 and I2 share a destination, but I2 does not read `$2`, so there is no RAW dependence between them.

## Nine cycles: hold the front end, advance older work

**IF → ID → EX → MEM → WB**

Amber: held instruction. Green arrows: actual forwarded values.




C4: detect dependence → C5: invalid EX bubble; hold I3 in ID and I4 in IF.
C6: MEM/WB → I3.EX. C7: EX/MEM → I4.EX. I4 completes in C9.

Each cell shows the stage occupied during that cycle. Pipeline registers change at the ending edge. C4 detects the dependence between I2.EX and I3.ID. At C4's ending edge, hold PC and IF/ID and inject an invalid ID/EX entry. C5 therefore has no valid EX instruction; the load itself keeps advancing.



## Check the values and controls against the timing diagram

| Cycle | Instruction positions and available values | Control or result |
|---|---|---|
| C4 | I2.EX computes address `0x1014`; I3.ID needs `$2` | `PCWrite=0`, `IFIDWrite=0`; inject an invalid ID/EX entry |
| C5 | I1.WB writes `0x33`; I2.MEM obtains `0x3c` at its end; I3 remains in ID | I3 reads and latches old `0x33`; the hazard is now cleared, allowing front-end advance at the edge |
| C6 | I2.WB carries `0x3c`; I3 is in EX | `ForwardA=01` selects MEM/WB, overriding the stale operand; `0x3c AND 0x0f=0x0c` |
| C7 | I3.MEM carries ALU result `0x0c`; I4 is in EX | `ForwardA=10` selects EX/MEM; `0x0c OR 0x80=0x8c` |

Without C5's bubble, I3 would reach EX too early. EX/MEM would contain the newest producer's load **address**, `0x1014`, while MEM/WB would hold the older value `0x33`. Neither is the required load result. Using the older value would incorrectly produce `$4=0x03`.

## Compare the same program with and without bypasses

| Model | I3.EX | I4.EX | Final WB | Extra stall cycles |
|---|---|---|---|---|
| EX/MEM and MEM/WB forwarding | C6 | C7 | C9 | 1 |
| No forwarding; register-file reads only | C7 | C10 | C12 | 4 |

Without forwarding, I3 stalls in ID during C4 and C5, then reads the new `$2` after C6's WB. I4 stalls in ID during C7 and C8, then reads `$4` after C9's WB. Same-cycle write-before-read avoids an additional wait.

Four useful instructions ideally take `4+4=8` cycles. These models take `8+1=9` and `8+4=12` cycles. Their finite-sequence CPIs, `9/4` and `12/4`, include pipeline filling and draining; they are not steady-state CPIs for a long program.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Control hazards, delay slots, and flush

Before a branch reaches EX, younger instructions have entered ID/IF. A taken branch predicted not taken requires recovery. Resolve stage and predictor determine penalty; architectural delay slots are a separate ISA property.

$$mispredict_cycles = f_{branch} \sum_{r} P(wrong_r \mid r) C_r.$$

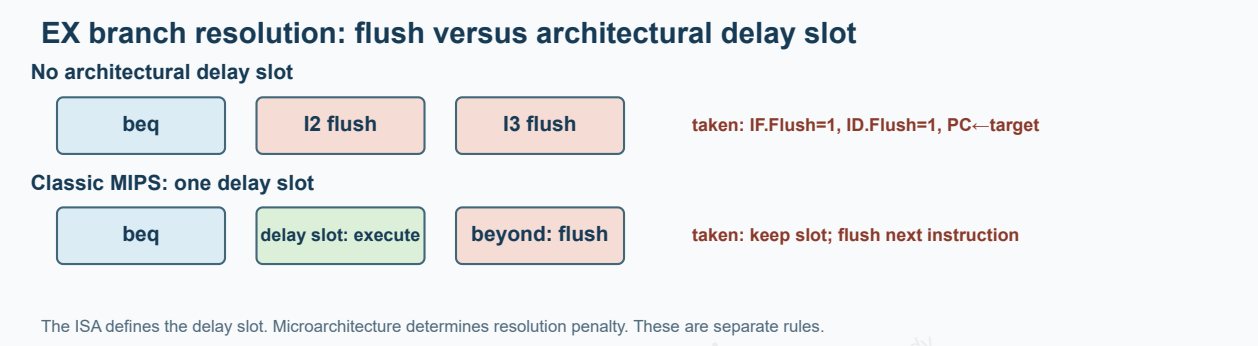
With 20% branches, 60% taken, always-not-taken prediction, and two flushed younger slots,

$$0.20(0.60)(2) = 0.24 \text{ cycles/instruction}.$$

Classic MIPS requires the immediately following slot to execute on either outcome. When the branch is in EX, that slot is in ID and the later instruction in IF.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Control hazards, delay slots, and flush (continued)




The no-slot model flushes ID and IF. The classic model preserves its ID slot and flushes only the later IF instruction. Predict, fetch, resolve, and recover are distinct events. A BTB hit provides a target without guaranteeing correct direction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Complete CPI boundary

If 10% of instructions additionally require one load-use bubble,

$$CPI = 1 + 0.20(0.60)(2) + 0.10 = 1.34.$$

This includes only the stated penalties. Structural conflicts, cache/target misses, long-latency units, and front-end limits require their own terms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Dynamic branch prediction

A two-bit saturating counter has strongly/weakly taken and strongly/weakly not-taken states. Taken outcomes increment toward strongly taken; not-taken outcomes decrement toward strongly not taken. Correctness of the prediction does not alone determine update direction.

Correlating predictors use branch history. Tournament predictors select among mechanisms. A branch-target buffer supplies target information separately from direction prediction. A BTB miss is not necessarily a direction error.

Examine initialization, alternating outcomes, correlation, and table aliasing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 4.9 Exceptions and interrupts

An exception commonly means a synchronous event caused by an instruction, such as overflow or an illegal opcode. An interrupt commonly means an asynchronous device or timer event. Terminology varies by ISA.

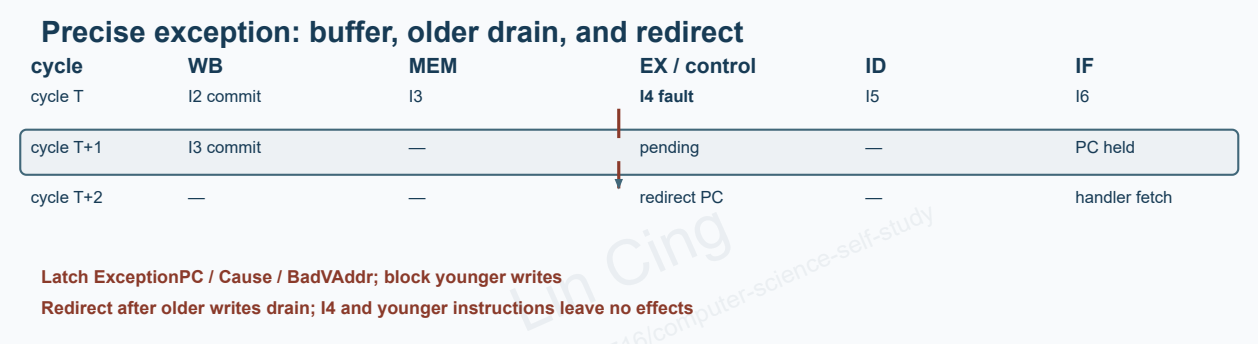
Recovery saves the relevant PC, cause, and address information, transfers to a handler, and either resumes or terminates. The handler needs a state consistent with the architecture rather than an arbitrary mixture of pipeline stages.

A pipeline flush alone does not specify which earlier effects must remain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Precise exception timing and state

Suppose I4 faults in EX. The legal single-issue snapshot has I2 in WB, I3 in MEM, I4 in EX, I5 in ID, I6 in IF; I1 already committed. I2 and I3 have no fault; I3 is a non-store instruction that commits a register result in WB. Prevent architectural writes by I4 and younger entries.




| Cycle | WB | MEM | EX/control | ID | IF |
|---|---|---|---|---|---|
| T | I2 commits | I3 | I4 arithmetic fault | I5 | I6 |
| T+1 | I3 commits | empty | exception pending | empty | PC held |
| T+2 | empty | empty | PC←handler, saved cause | empty | handler fetch |



# Precise exception timing and state (continued)

Rows show stage activity during each cycle; commit completes at its end. After I3 commits at the end of T+1, control selects the handler PC at the start of T+2. The first handler instruction is fetched in IF during T+2 and has not yet reached ID.

At T, capture I4.PC, cause, and an address only if required for the exception type; arithmetic overflow need not have a bad address; suppress faulting/younger writes and flush I5/I6. Buffer the exception while fetch stops and older instructions drain. The handler must observe exactly the state after I1, I2, I3 in program order, with no effects from I4 or later. It cannot enter before I3's commit at T+1; this controller redirects at T+2.

Simultaneous exceptions need a specified priority. The oldest synchronous fault commonly wins, while asynchronous interrupts and machine checks have separate policies. Handler entry also saves/switches privilege, status, EPC, cause, and address context.

An out-of-order ROB enforces the same contract. Younger instructions may execute, but retirement remains ordered. Once I4 reaches the head after all older commits, discard I4 and younger entries, restore the committed rename state, and redirect.

## 4.10 Instruction-level parallelism and multiple issue

ILP is overlap among independent operations in one instruction stream. Pipelining overlaps stages; multiple issue starts several operations in a cycle.

Static issue packets and VLIW expose scheduling decisions to the compiler and binary contract. Dynamic issue selects ready instructions at runtime, tolerating variable latencies at the cost of dependency tracking, wakeup/select, and recovery state.

A four-wide maximum does not imply sustained IPC four.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Superscalar execution and dynamic scheduling

A superscalar front end attempts to supply several instructions each cycle. Issue logic checks operand readiness and functional-unit availability. Later ready work may execute before older stalled work.

Speculation follows an unresolved predicted branch. Recovery discards younger results and restores a checkpoint or rename map. Renaming provides more physical names without increasing the ISA's visible register count.

Out-of-order execution and in-order commit are compatible. Their separation supports precise exceptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A cycle-by-cycle rename, execution, and ROB example

Fix one in-order dispatch and one in-order commit per cycle. Dispatch allocates a ROB entry and a fresh physical destination. Execution starts no earlier than the following cycle. A separate load unit has four-cycle latency; a pipelined integer ALU has one-cycle latency. Results become available at a cycle's end and may feed consumers or commit only in the next cycle. Assume no cache miss, branch, fault, other resource conflict, or ALU overflow.

```
I1: lw  r1,0(r2)
I2: add r3,r1,r4
I3: add r5,r6,r7
I4: add r1,r8,r9
```

Initially architectural r1/r2/... map to p1/p2/.... Rename allocates I1→p11, I2→p12, I3→p13, I4→p14. I2 captures p11 as its r1 source at dispatch. I4 later remaps speculative r1 to p14 but cannot change I2's saved source tag. True RAW dependence I1→I2 remains; distinct physical destinations separate the I1/I4 output name conflict and I2/I4 anti-dependence.



# A cycle-by-cycle rename, execution, and ROB example (continued)

**Out-of-order results, in-order commit (one commit/cycle)**

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| **I1: lw → p11** | D | LD1 | LD2 | LD3 | LD4 | C | | | |
| **I2: add → p12** | | D | wait | wait | wait | EX | C | | |
| **I3: add → p13** | | | D | EX | ready | ready | ready | C | |
| **I4: add → p14** | | | | D | EX | ready | ready | ready | C |

D: dispatch; EX/LD: execute; wait: operand unavailable; ready: wait for ROB head; C: commit

I2 source = p11 (I1); I4 destination = p14. Result end of cycle → use next cycle.

## A cycle-by-cycle rename, execution, and ROB example (continued)

| Instruction | Dispatch | Execution | Ready at cycle end | Commit | Reason |
|---|---|---|---|---|---|
| I1 | 1 | Load cycles2–5 | 5 | 6 | Ready ROB head commits next cycle |
| I2 | 2 | EX cycle6 | 6 | 7 | Captured p11 waits for I1 |
| I3 | 3 | EX cycle4 | 4 | 8 | Ready result waits behind I1/I2 |
| I4 | 4 | EX cycle5 | 5 | 9 | Fresh p14 does not overwrite p11 |

Take memory[0(r2)]=10, r4=1, r6=2, r7=3, r8=4, and r9=5. I3 produces p13=5 first; I4 produces p14=9; I1 produces p11=10; I2 finally consumes p11 to produce p12=11. Final architectural r1=9, r3=11, r5=5 agrees with program order. Reading p14 for I2 would incorrectly produce r3=10. Executed, ready, and committed are distinct states.



## A cycle-by-cycle rename, execution, and ROB example (continued)

In a separate variant, keep I1's loaded value 10 but set r4=2147483647. I2's signed add then reports overflow during EX cycle6. The older I1 may still commit in cycle6. Record I2's fault without committing it. When I2 reaches the ROB head, discard I2/I3/I4 and restore the committed rename map: r1 maps to p11, not p14. The handler sees r1=10 and the pre-sequence architectural r3/r5. Already-computed p13/p14 never committed and cannot survive merely because they finished early.

This model excludes speculative stores, memory disambiguation, and branch checkpoints. Adding them requires a store queue and explicit exception/recovery rules.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Limits to wider issue

If a machine can issue four operations but the program usually exposes only one ready operation, width alone offers little benefit. Average IPC needs a distribution with an explicit denominator, such as instructions issued per observed cycle.

Dependencies, cache bandwidth, prediction, functional-unit mix, register ports, ROB capacity, and power can all limit utilization. An IPC histogram can distinguish these situations better than peak width.

A percentage of cycles and a percentage of instructions produce different weighted averages.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 4.11 The Opteron X4 case

The book's Barcelona-era Opteron translates x86 instructions into AMD RISC operations, or Rops, before dynamically scheduled speculative execution. The historical model describes up to three Rops issued per cycle and an approximately twelve-stage integer pipeline, with longer floating-point paths.

Complex decoding, prediction misses, dependency chains, cache misses, and translation/rename resources can limit throughput. These figures describe the book's period, not current CPUs.

Keep ISA instruction count separate from internal-operation count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 4.12 HDL and simulation

An HDL describes ports, combinational equations, and clocked state. A Verilog positive-edge process models a state update. Blocking and nonblocking assignments have different simulation semantics.

Simulation checks the testbench's cases. Synthesis, static timing, formal assertions, and physical testing address other questions. The supplied PDF contains only this section's overview; full HDL is on an unavailable CD. This edition does not claim to reconstruct that missing source.

Assertions about hazards, flushes, and commit order are more informative than a final waveform screenshot alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 4.13 Processor fallacies

Pipelining primarily improves throughput, not individual instruction latency. Forwarding cannot eliminate a dependence on data that has not yet arrived. Technology affects logic delay, clock distribution, power, and memory access, so pipeline organization is not independent of it.

Give a cycle trace that contradicts each overgeneralization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 4.14 Checking the complete model

Validate architectural results, enumerate simultaneous resource demands, and specify branch and exception recovery. Report latency, throughput, IPC, frequency, and energy against a workload.

Single-cycle execution exposes critical-path constraints. Pipelining adds overlap together with forwarding, stalls, prediction, and recovery obligations. Multiple issue and dynamic scheduling add more opportunity and more state.

Complete the separate exercises before consulting solutions.

Reconstruct a pipeline diagram from the rules rather than memorizing its appearance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 4.15 Historical perspective

Multicycle execution distributes a long path over time. Pipelining distributes its stages across different instructions. Both need state to identify unfinished work. Forwarding and stalls maintain value dependencies; prediction trades recovery cost for continuous fetching.

Dynamic scheduling and renaming let ready operations pass stalled ones while retaining true dependencies. Ordered commit preserves the software contract. Wider issue and larger windows increase selection, wiring, energy, and verification costs, so stage count and peak IPC alone cannot rank historical designs.

For each development, identify the constraint removed, the state added, and the state recovery must reconstruct.

### References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 4. CSO course slides, processor datapaths and pipelines.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

## The hierarchy problem

A DRAM access can take many processor cycles. Sending every load and store directly to DRAM would leave a fast datapath waiting. A memory hierarchy combines levels of different capacity, cost, and latency to make most accesses appear fast while providing substantial capacity.

The benefit depends on locality. It is not a guarantee for arbitrary access patterns. Every model in this chapter identifies capacity, transfer unit, miss cost, and write policy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Address units and notation

A virtual address, VA, identifies a process location. Removing its page offset gives a virtual page number, VPN. Translation produces a physical address, PA. Removing the cache-block offset gives a physical block number.

Addresses count bytes unless stated otherwise. A word index must be scaled by four in this MIPS32 model. A block number is the byte address divided by block size. Let $B$ denote block bytes and $P$ denote page bytes when those symbols are used.

A correct-looking bit calculation with the wrong address unit still gives the wrong mapping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.1 Locality and levels

Registers, L1, lower-level caches, DRAM, persistent storage, and remote storage trade increasing capacity for generally increasing latency and lower cost per byte.

Temporal locality means a recently used location is likely to be reused soon. Spatial locality means nearby locations are likely to be accessed. Consider consecutive 4-byte integers at byte addresses 0, 4, 8, 12, and 16. With an initially empty cache, 16-byte blocks, no prefetch, and no eviction, accesses are miss, hit, hit, hit, miss. The three hits exploit spatial locality. Reading 16 again while its block remains resident illustrates temporal locality.

Locality is a workload property exploited by a design, not a universal theorem about programs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Average access time

Let $H$ be hit time, $m$ miss probability per access, and $P_m$ the additional miss penalty after the hit lookup.

$$AMAT = H + mP_m.$$

At $H = 1$, $m = 0.05$, and $P_m = 80$ cycles, AMAT is five cycles. Raising miss rate to 20% gives 17 cycles. Most references can be fast while the few misses dominate average time.

Distinguish misses per instruction from misses per data reference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The memory wall and design tradeoffs

Processor execution and DRAM latency do not improve at identical rates. Capacity, associativity, and block size can reduce selected misses while increasing lookup delay, fill traffic, area, or power.

L1 prioritizes short hit paths. Lower caches can afford larger capacity and associativity. DRAM supplies capacity and bandwidth. Persistent and remote levels accept much larger access costs.

A lower miss rate alone does not prove a faster hierarchy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 5.2 The basic cache model

A cache line contains data, a tag, and a valid bit. A write-back line also has a dirty bit. An access selects a set, compares valid tags, selects a matching way, and uses the offset to choose data.

On a miss, a controller obtains a block from the next level and chooses a location or victim. Valid prevents uninitialized tags from creating false hits. Dirty conservatively tracks possible unpropagated writes, not whether the tag matches or whether bytes necessarily differ.

Record data, tag, valid, and dirty changes separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Direct mapping

$$index = block_number \bmod N_{blocks}.$$

Each memory block has one possible cache position. For a power-of-two block count, the low block-number bits select it. Offset chooses within the block; tag supplies the remaining identity.

Frequently reused blocks mapping to the same position evict each other even if other positions are unused. Addresses separated by the cache's data capacity provide a useful conflict example.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



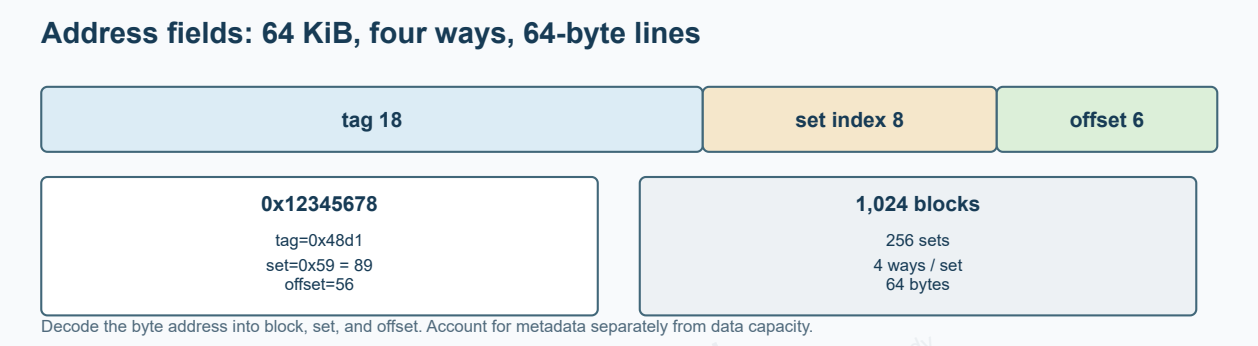
## A concrete access trace

A four-line direct-mapped cache, initially invalid, receives block numbers `0,1,2,3,0,4,0`.

| Access | Block | Index | Result |
|---|---|---|---|
| 1 | 0 | 0 | Miss, initially invalid |
| 2 | 1 | 1 | Miss, initially invalid |
| 3 | 2 | 2 | Miss, initially invalid |
| 4 | 3 | 3 | Miss, initially invalid |
| 5 | 0 | 0 | Hit |
| 6 | 4 | 0 | Miss, evicts 0 |
| 7 | 0 | 0 | Miss, evicts 4 |

There are six misses in seven accesses. First touches of 0–4 are compulsory. The final access is a conflict miss under the same-capacity fully associative LRU comparison, which would retain recently used block 0.

# Address decoding for a 64-KiB four-way cache




Decode the byte address into block, set, and offset. Account for metadata separately from data capacity.

With 64-byte lines and 32-bit addresses,

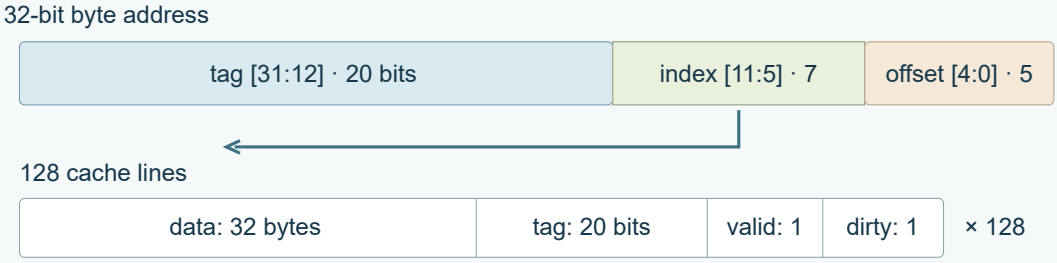
$$N_{blocks} = 2^{16}/2^{6} = 1024, \quad N_{sets} = 1024/4 = 256.$$

Offset has six bits, index eight, and tag $32 - 6 - 8 = 18$. For `0x12345678`, offset is `0x38` or 56, index is `0x59` or 89, and tag is `0x48d1`. Index bits are `01011001`.

`0x12345658` shares block base `0x12345640` and differs only in offset, so it hits if the block remains valid.



# Reconstruct capacity and metadata from address fields




For a direct-mapped write-back cache with 32-bit byte addresses, five offset bits give 32 bytes per line and seven index bits give 128 lines. Data capacity is therefore 128 × 32 = 4096 bytes.

Each line also holds a 20-bit tag, one valid bit and one dirty bit. Metadata occupies 128 × 22 = 2816 bits = 352 bytes. Counting only these fields, total storage is 4448 bytes, or 1.0859375 times the data capacity; ECC and other control circuitry are excluded.

## Handling misses

A blocking controller saves the request address, requests the lower-level data, handles a victim, installs data/tag/valid, and retries. An instruction miss needs the original request PC. A data miss needs the original address and any store data.

A dirty victim must be written back or safely buffered before its storage is overwritten. Critical-word-first can return the requested word before the entire fill completes.

A cache miss is ordinarily a hardware transaction and stall. A page fault invokes OS recovery and has different state requirements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Supporting a block fill

For four words, assume one address cycle, 15 cycles per memory activation, and one cycle per word transfer. A one-word-wide organization takes

$$1 + 4(15) + 4 = 65 \text{ bus cycles.}$$

A two-word-wide memory and bus can take $1 + 2(15) + 2 = 33$ cycles in this model. Widening only memory gives $1 + 2(15) + 4 = 35$. These are explicit transaction models, not universal DRAM timings.

Larger blocks exploit spatial locality but can increase fill cost and pollution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Interleaved memory: overlapping independent banks

Place four consecutive words in four independent banks. Assume one shared address operation starts all four in cycle 1, each bank takes 15 cycles, and the bus transfers one word per cycle. Once data is ready, four serial transfers complete the fill in 1 + 15 + 4 = 20 bus cycles.

Compare 65 cycles for the single-bank model: the gain comes from overlapping bank latency, not transferring four words at once on a narrow bus. The result changes if addresses cannot be issued together, accesses collide in a bank, or responses require additional spacing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Handling writes

Write-through propagates each write to the next level, usually through a buffer. Write-back changes the cached copy and marks it dirty, propagating later on eviction or protocol demand.

Write allocation installs a missed block before updating it. No-write-allocation forwards the store without installing the line. Allocation and propagation are independent decisions.

A partial-word or partial-line update needs byte enables or an equivalent cache-array update mechanism. It does not inherently require a main-memory read-modify-write transaction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Write-policy boundary cases

| Policy | Write-miss action | Main consequence |
|---|---|---|
| Through, no allocation | Forward store | Low allocation cost, high write traffic |
| Through, allocation | Fetch, update both levels | Possible later hits, additional traffic |
| Back, allocation | Fetch, update and dirty | Coalesces repeated writes, dirty victims |
| Back, no allocation | Forward missed store | Avoids pollution, misses later reuse |

A store buffer must also supply or order a following read of an address whose write is pending. Returning an older cache value would violate the required memory semantics.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



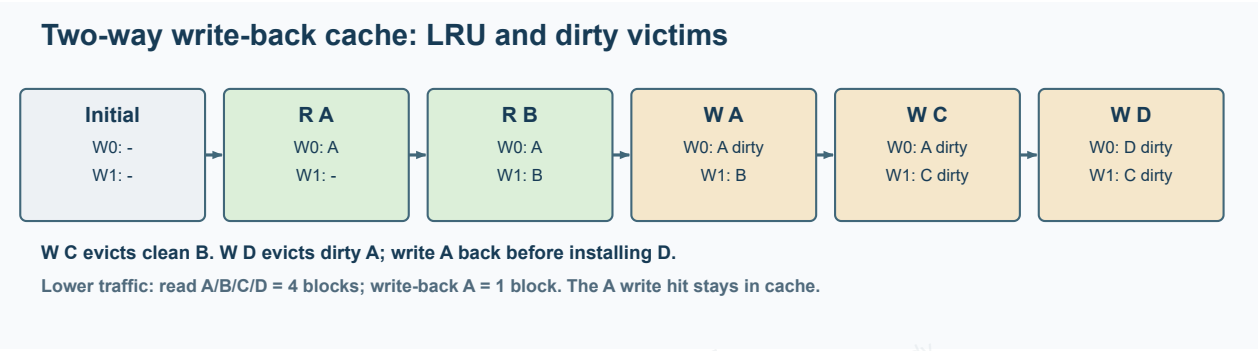
## Write-back, allocation, and dirty victims

Use a two-way set, 64-byte blocks, four-byte stores, LRU, write-back, and write-allocate. A/B/C/D map to this set. Both ways start invalid; choose the lower-numbered empty way. Ways are fixed physical slots; LRU updates do not move lines.

| Step | Access | Way 0 | Way 1 | Victim and lower-level action |
|---|---|---|---|---|
| 0 | initial | invalid | invalid | none |
| 1 | read A | A clean | invalid | read A block |
| 2 | read B | A clean | B clean | read B block |
| 3 | write A | A dirty, MRU | B clean | hit, no lower write |
| 4 | write C | A dirty | C dirty, MRU | discard clean B, read C, then modify C |
| 5 | write D | D dirty, MRU | C dirty | write back dirty LRU A, then read D |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Write-back, allocation, and dirty victims (continued)




Four blocks are read and A alone is written back. Overwriting A before preserving its newest value loses committed data. Dirty does not affect hit detection, but determines whether eviction must preserve a newer value. Correct write-back alone is not a persistence guarantee: cache and DRAM contents may both be lost on power failure.

With write-through/no-write-allocate, write A immediately updates the lower level. Writes C and D do not allocate or evict B. The final set retains A/B, with two block reads and three word writes.



## Write-back, allocation, and dirty victims (continued)

Counting payload only, no headers/acks or final flush, WB/WA moves $4(64) + 1(64) = 320$ bytes. WT/NA moves $2(64) + 3(4) = 140$ bytes. Both have five lower-level actions but different transfer sizes. Flushing final dirty C/D would add 128 bytes to WB; persistence still depends on platform flush/ordering guarantees.

In a write-invalidate coherence protocol, a cache must obtain exclusive write permission before completing a store to that block. Write-back delays the lower-level update while the processor still obeys its memory model. Other protocols may use different sharing or update rules. A dirty victim’s latest data must reach memory, a reliable buffer, or another owner with guaranteed completion before its storage is reused; a clean victim can be discarded as permitted by the protocol.

## The FastMATH case

The historical Intrinsity FastMATH example separates instruction and data caches. Each 16-KiB cache contains 4096 words in 16-word blocks, giving 256 blocks.

Under direct mapping, index is eight bits and byte offset six. Capacity alone does not establish every timing or protocol detail. Split caches remove a shared IF/MEM port conflict but cannot pool unused instruction capacity for data.

Keep historical implementation numbers separate from current measurements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 5.3 Cache performance and CPI

$$T_{CPU} = (C_{execution} + C_{stall})T_{clock}.$$

For a blocking model with instruction miss rate $m_i$, data references per instruction $f_d$, data miss rate $m_d$, and common extra penalty $P_m$,

$$CPI = CPI_{base} + m_i P_m + f_d m_d P_m.$$

Out-of-order execution and memory-level parallelism can overlap costs. Do not apply full-stall addition without checking that assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Split and unified caches: align denominators first

Consider 100 retired instructions, each with exactly one instruction fetch, plus 30 data references; exclude speculative fetches and extra accesses. The split I-cache has two misses and the D-cache three, so their miss rates are `2/100=2%` and `3/30=10%`.

If each miss causes 80 additional non-overlapping cycles and base CPI excludes them, added CPI is `(2 + 3)×80/100 = 4`. Equivalently, per instruction it is `1×0.02×80 + 0.3×0.10×80 = 4`.

If a separate unified-cache configuration happens to have five misses on this trace, its miss rate uses all 130 references: `5/130`. With `1.3` references per instruction, added CPI is `1.3×(5/130)×80 = 4`. This converts denominators; it does not guarantee equal miss counts across designs. Account separately for any single-port fetch/data conflict, without double counting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## An AMAT and CPI example

One-cycle hits, 6% misses, and 80-cycle additional penalty give $AMAT = 1 + 0.06(80) = 5.8$ cycles.

Separately, with base CPI 1.4, instruction miss rate 2%, 0.25 data references/instruction, and data miss rate 8%, stalls add

$$0.02(80) + 0.25(0.08)(80) = 3.2.$$

Total CPI is 4.6. One billion instructions at 2 GHz take 2.3 s under this full-stall model.

Data miss rate applies to data references rather than every instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Flexible placement

For associativity $E$, $N_{sets} = N_{blocks}/E$. Direct mapping has one way. Full associativity has one set.

At constant data capacity, doubling associativity halves sets, removes an index bit, and adds a tag bit. Each selected set needs more parallel comparisons and selection. Reduced conflicts can come with higher hit time and energy.

Equal data capacity does not imply equal tag-array cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Finding a block and choosing a victim

Select a set, compare valid tags, and select the matching way. Multiple valid matches for the same block indicate an invariant failure in this model.

Random, LRU, and approximations choose victims. Two-way LRU needs one recency bit; exact LRU becomes more expensive at higher associativity. In a two-way set containing A and B, accessing B makes A the next LRU victim.

Replacement normally affects performance. Coherence and dirty-data handling impose additional correctness obligations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Multilevel AMAT denominators

Let $T_\ell$ be hit service time when level $\ell$ is reached, $M_\ell^{local}$ its miss fraction among requests sent there, and $P_{mem}$ the extra memory penalty after the last miss.

$$AMAT_1 = T_1 + M_1 T_2 + M_1 M_2^{local} P_{mem}.$$

For $T_1 = 1$, $M_1 = 0.04$, $T_2 = 8$, $M_2^{local} = 0.25$, and $P_{mem} = 100$,

$$AMAT = 1 + 0.04(8) + 0.04(0.25)(100) = 1 + 0.32 + 1 = 2.32.$$

| Quantity | Denominator | Value |
|---|---|---|
| M1 | all L1 references | 0.04 |
| L2 local miss | L1 misses only | 0.25 |
| L2 global miss | all L1 references | 0.01 |
| L2 hit fraction | L1 misses only | 0.75 |

Seventy-five percent of L1 misses pay only eight L2 cycles; one percent of all references additionally pay memory. Treating 25% as global gives the wrong $1 + 0.04(8) + 0.25(100) = 26.32$.

If $P_{2,miss,total} = T_2 + P_{mem} = 108$, this is conditional on also missing L2. L2 hits still cost eight. Conditional averaging is necessary:

$$AMAT_1 = T_1 + M_1[(1 - M_2^{local})T_2 + M_2^{local} P_{2,miss,total}] = 1 + 0.04[0.75(8) + 0.25(108)] = 2.32.$$



## Multilevel AMAT denominators (continued)

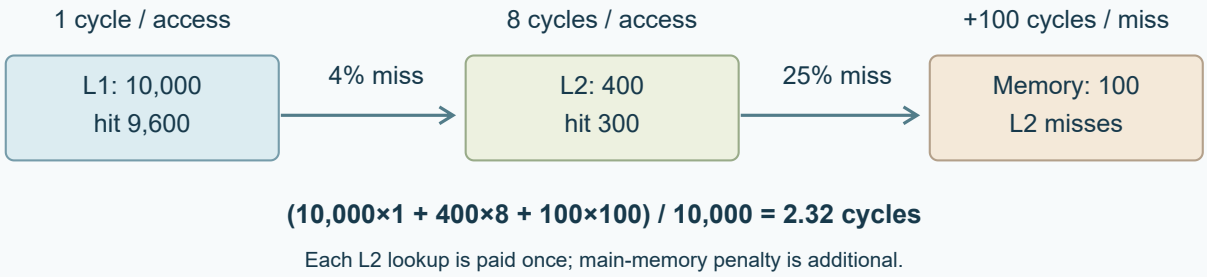
Only the expected cost per L1 miss,

$$\overline{P}_{1,miss} = T_2 + M_2^{local} P_{mem} = 33,$$

permits $AMAT_1 = 1 + 0.04(33)$. Multiplying conditional 108 directly by the L1 miss rate wrongly gives 5.32. Including L2 time in a penalty does not remove the need to distinguish its hit and miss cases.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Multilevel AMAT over 10,000 accesses




**(10,000×1 + 400×8 + 100×100) / 10,000 = 2.32 cycles**

Each L2 lookup is paid once; main-memory penalty is additional.

Assume the preceding proportions apply to 10,000 references. L1 misses `10,000×0.04=400` times; 25% of those, or 100, also miss L2, leaving 300 L2 hits.

Every reference pays one L1 cycle, 400 pay eight L2 cycles, and 100 pay an additional 100 cycles. The total is `10,000+3,200+10,000=23,200` cycles, averaging `23,200/10,000=2.32` cycles. The 100-cycle penalty starts after the L2 lookup; do not charge those eight L2 cycles twice.



## 5.4 Why virtual memory exists

Virtual memory maps a process's addresses to physical frames, supports protection and controlled sharing, and simplifies allocation and relocation. It can also keep only active pages resident while backing other content with storage.

Address-space size is not physical residency. Historical overlays made software arrange code movement explicitly; paging transfers much of this work to the OS and hardware.

Virtual memory provides an address and protection contract, not unlimited fast storage.

## Page tables and translation

With 4-KiB pages, a 32-bit VA has a 20-bit VPN and 12-bit offset. A PTE can hold a physical page number, present/valid state, permissions, dirty/reference information, and caching attributes. Exact fields are platform-specific.

$$PA = PTE.PPN \parallel VA.offset.$$

The page offset is copied unchanged because a virtual page and physical frame have the same internal size and layout.

A missing page's backing location is typically interpreted through OS-managed metadata rather than assumed identical across PTE formats.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Page faults are more than cache misses

A TLB miss can occur while the page is resident. A fault can indicate absent content or denied access. For a recoverable not-present fault, save the address and context, obtain a frame, preserve a dirty victim, load or initialize content, update mappings, invalidate stale translations, and retry.

A major fault involving storage is expensive. Minor faults such as demand-zero allocation or copy-on-write need not access disk. Permission faults do not automatically authorize loading and retrying.

A faulting store must not leave an unauthorized partial architectural update.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Virtual-memory writes

Paging ordinarily uses write-back because storage cannot absorb a transaction for every processor store. A dirty page may have updates not yet preserved in its backing copy; a same-value store can still set dirty. A clean victim can be discarded if its content can be reconstructed.

An invalid PTE may mean unallocated, swapped, or specially managed content. The OS combines fault address, access type, and mapping metadata to decide whether recovery is legal.

Dirty is a conservative record of possible updates, not a timestamp or proof that bytes differ. Clearing it must be coordinated with writeback and concurrent writes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Fast translation with a TLB

A TLB caches translations and permission information. Its tag can combine VPN and address-space identifier, ASID. A hit provides the PPN without walking the full page table.

A miss triggers hardware or software refill, not necessarily a page fault. TLBs are small and often highly associative. Permissions and dirty/reference management must remain consistent with PTE changes.

A context switch can flush translations or retain them under ASID tags.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

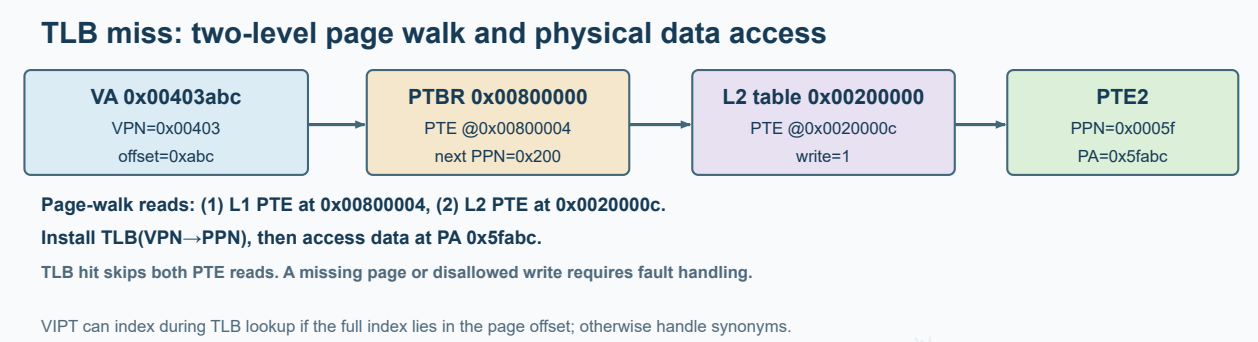
# A TLB miss, two-level walk, and cache lookup

Use 32-bit virtual and physical addresses, 4 KiB pages, four-byte PTEs, and 1024 entries per level. `VA=0x00403abc` gives `VPN=0x00403` and offset `0xabc`. The ten-bit indices are L1=1, L2=3. The following PTE contents are explicit model inputs.

| Step | Address calculation | Read or decision |
|---|---|---|
| 1 | `PTBR=0x00800000`, add `0x001*4` | read L1 PTE at `0x00800004` |
| 2 | L1 PPN `0x00200` gives table base `0x00200000` | read L2 PTE at `0x00200000+0x003*4=0x0020000c` |
| 3 | L2 `present=1`, `write=1`, `PPN=0x0005f` | install `VPN 0x00403→PPN 0x0005f` |
| 4 | `PA=(0x0005f<<12)\|0xabc` | access data at `0x5fabc` |



# A TLB miss, two-level walk, and cache lookup (continued)




In this example, a miss without a walk-cache hit requires two PTE accesses plus data-cache lookup/fill. Page-table accesses themselves can hit/miss caches; dedicated walk caches change the count. A TLB hit skips both PTE reads. Valid resident PTEs mean a translation miss. A missing present bit causes a not-present page fault. Present 1/write 0 on a store means protection fault.

A VIPT cache can index from the untranslated page offset while the TLB finds the physical tag. A simple safe configuration requires

$$\text{block-offset bits} + \text{index bits} \le \text{page-offset bits}, \qquad BS \le P,$$

or data capacity/ways≤page size. Comparing index width alone omits block offset.

## A TLB miss, two-level walk, and cache lookup (continued)

For 64 KiB, four ways, 64-byte blocks, there are 256 sets. Offset bits 5:0 and index 13:6 total 14>12. Aliases VA `0` and `0x1000` for the same physical page/offset select sets 0 and 64. Sixteen ways gives 64 sets and 6+6=12, so these aliases select the same set.

A larger virtual index requires coloring, synonym maintenance, or PIPT. Comparing only PA[31:14] would omit physical bits 13:12 and allow a false hit. Comparing the full 20-bit PPN prevents this false identification, but cannot by itself eliminate duplicate copies in different sets. The 18-bit PIPT tag is not a sufficient physical identifier for this VIPT configuration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Two VIPT hazards, traced with addresses

**64 KiB · 4-way · 64 B blocks · 4 KiB pages**

**1. Synonym: same PA, different sets**

| | | |
|---|---|---|
| VA 0x0000 → set 0 | → PA 0x0000 | duplicate copies |
| VA 0x1000 → set 64 | → PA 0x0000 | |

**2. False hit: different PA, same set and short tag**

| | | |
|---|---|---|
| VA 0x0000 → set 0 | → PA 0x0000 | PA[31:14] = 0 |
| VA 0x4000 → set 0 | → PA 0x1000 | PPN = 0 / 1 |

set = VA[13:6]; the page offset is VA[11:0]. Mappings are explicit examples.

Lin Cing

Keep the 64 KiB, four-way, 64-byte-block, 4 KiB-page configuration. If both `VA 0x0000` and `VA 0x1000` map to `PA 0x0000`, they select sets 0 and 64: the same block can appear in two sets. A full PPN identifies each block but tag comparison alone does not eliminate cross-set copies.

In a separate example, `VA 0x0000 → PA 0x0000` and `VA 0x4000 → PA 0x1000`. Both VAs select set 0, and both PAs have `[31:14]=0`. Comparing only this short tag can falsely hit the first block on the second access. Their full PPNs are 0 and 1, so a complete comparison rejects the false hit.

## Protection and context changes

Read, write, execute, and user/supervisor permissions must be enforced on a TLB hit as well as during a page-table walk.

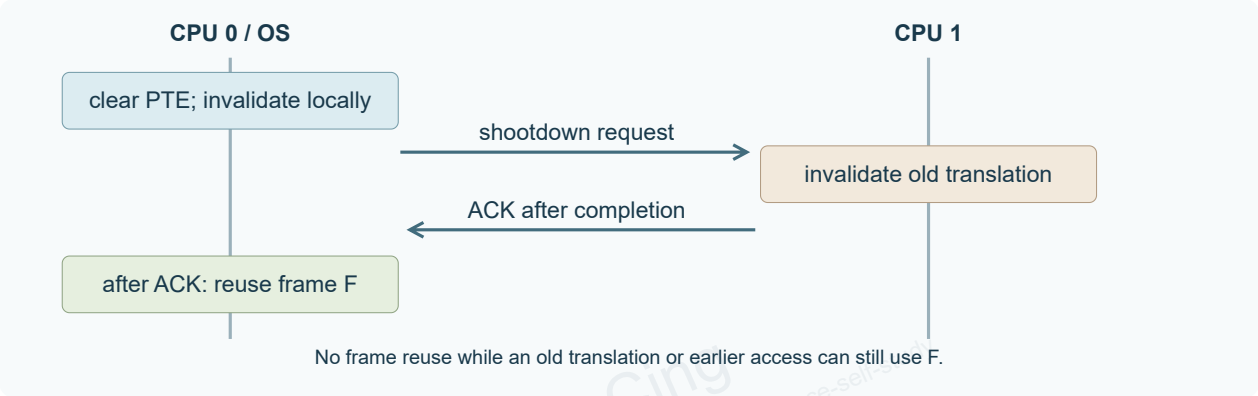
Flushing on a context switch prevents a new process from using another process's same-VPN translation. ASIDs allow coexistence, but the OS must still handle identifier reuse, mapping changes, and invalidation.

Identity tags never replace access-permission checks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Multicore TLB shootdown: when can a frame be reused?




Assume CPU 0 and CPU 1 run the same address space and may both cache $V \to F$. The OS revokes the mapping and plans to reassign F. It updates the PTE, applies architecture-required ordering, invalidates its local translation, and requests CPU 1 to invalidate the matching translation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Multicore TLB shootdown: when can a frame be reused? (continued)

CPU 1 acknowledges after the required invalidation and synchronization. The OS reuses F only after every potentially retaining core acknowledges and earlier accesses are safe under platform rules. Reusing F early can leave CPU 1’ s stale translation pointing at reallocated data. ASIDs distinguish address spaces but mapping changes within an ASID or ASID reuse still require invalidation or an equivalent mechanism; DMA and pinned pages require separate management.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



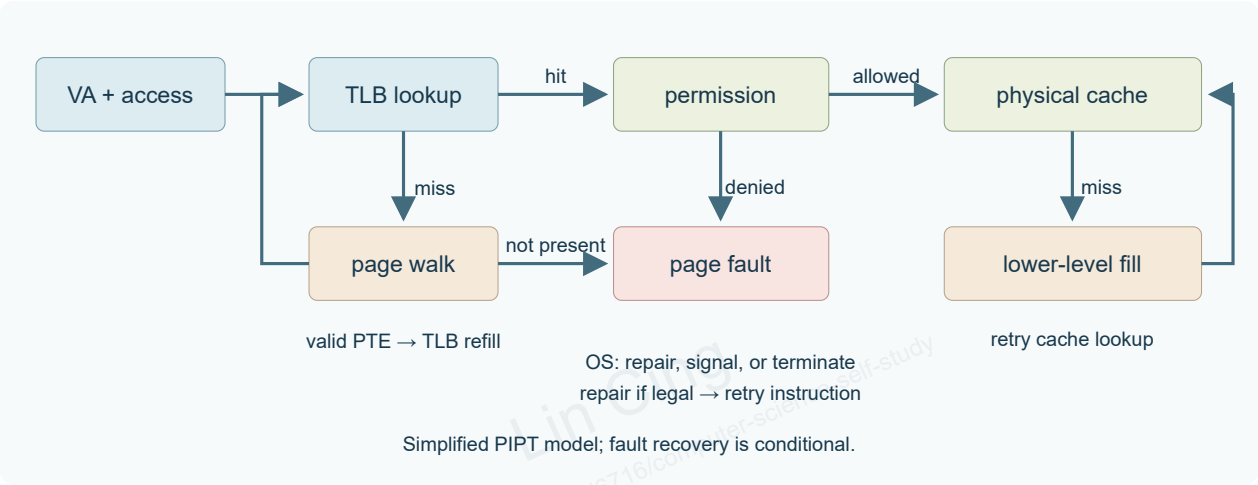
## Miss and fault recovery paths

| Event | Evidence | Response |
|---|---|---|
| Cache miss | No valid matching tag | Fill lower-level data |
| Resident TLB miss | No TLB match, valid permitted PTE | Refill translation |
| Not-present page | PTE indicates absent content | OS recovery if mapping is valid |
| Protection fault | Access permission denied | OS policy, signal, termination, or legitimate repair |
| Undefined VA | No valid mapping | OS fault handling |

Retries require precise or otherwise recoverable state. Sharing a fault-address register does not make these events equivalent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## One access: three different event paths




In this simplified physically indexed, physically tagged (PIPT) model, a TLB miss starts a page-table walk. A valid mapping can refill the TLB; a TLB miss is not itself a page fault. An absent mapping or denied permission invokes OS handling: repair and retry when legal, otherwise signal or terminate.

After translation and permission checks, a cache miss fetches a block from a lower level before completion or retry. This is a teaching model: VIPT designs may overlap some lookups, and exact fault/refill mechanisms depend on the ISA and OS.



## 5.5 Four common questions

Where may a block reside? How is it found? Which victim is selected? How do writes propagate?

Caches, TLBs, and paging answer these with different placement, lookup, replacement, and write policies. Page-table lookup costs additional accesses but supports flexible physical-frame placement. Full associative placement is not free parallel associative lookup hardware for the entire page table.

Separate the freedom to place an object from the mechanism used to locate it.

## Same questions, three different objects

| Structure | Stored/looked-up object | Miss handling | Update concern |
|---|---|---|---|
| Data cache | Physical data block, found by set and tag | Fetch the block from a lower level | Propagate data under write-through or write-back |
| TLB | Virtual-to-physical translation and permissions | Walk the page table or use software refill | Invalidate or synchronize stale translations after changes |
| Virtual memory | Mapping of a virtual page to a frame and its backing | OS checks validity and may allocate or load content | Preserve dirty pages when their contents must survive |

A TLB entry stores a translation, not the data block held by a cache. Updating a PTE may require TLB invalidation; writing back page content concerns the page and its backing. Both may be needed, but they are distinct operations.



## Three miss types require an explicit reference

Use a fully associative LRU shadow with the same block size and data capacity, updated on every reference. First establish that the actual cache misses, then classify:

| Type | Condition for this miss | Common remedy | Possible cost |
|---|---|---|---|
| Compulsory | First-ever access to this block | Larger blocks, prefetch | Traffic, pollution |
| Capacity | Previously accessed; shadow misses | More capacity, better locality | Hit time, cost |
| Conflict | Previously accessed; shadow hits | More associativity | Comparison and selection cost |

A first touch takes precedence as compulsory. Classification is per access, for a specified trace and reference policy. A working-set slogan or a replacement in the same line cannot classify every later miss.

## Classify three miss types using one shared trace

Use 32-bit byte addresses, 64 B of data capacity, 16 B blocks, an initially empty cache, and reads only: no prefetching or coherence invalidations. The actual cache is four-line direct-mapped (DM). Its reference is an equally sized, four-block fully associative **LRU shadow cache**.

```
block sequence: 0, 4, 0, 1, 2, 3, 4, 0, 1
byte addresses: 0x0000, 0x0040, 0x0000, 0x0010, 0x0020,
                0x0030, 0x0040, 0x0000, 0x0010
```

Every access uses offset zero. DM has four offset bits, two index bits, and 26 tag bits: `line=block mod 4`, `tag=floor(block/4)`. Blocks 0 and 4 have different tags but compete for line 0. The shadow instead permits any block in any of its four positions.

Check the actual cache first. A hit is not a 3C miss. On a miss, classify a first touch as compulsory; for later touches, use a shadow hit/miss to classify conflict/capacity. **Both caches process every access, including actual hits.** An LRU hit also updates recency.



## The third access conflicts despite unused capacity

**Same capacity: 4 blocks × 16 B = 64 B; initially empty**

**#1 block 0 (0x0000)**

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 0 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 0 | — | — | — |

**DM miss / FA miss**

First touch of this block

**#2 block 4 (0x0040)**

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 4 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 0 | 4 | — | — |

**DM miss / FA miss**

First touch of this block

**#3 block 0 (0x0000)**

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 0 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 4 | 0 | — | — |

**DM miss / FA hit**

Conflict: 4 had displaced 0

DM rows have fixed line numbers. The FA boxes show LRU-to-MRU order, not fixed way numbers. Access 2 forces block 4 to displace block 0 in DM line 0. FA has spare space and retains both. Access 3 revisits block 0: DM misses while the shadow hits, making this a conflict miss.

## Accesses 1–5: compulsory and conflict misses

States are **after** each access. A dash means invalid. H/M is the current hit/miss; FA entries run from LRU to MRU.

| # | block | DM [line 0,1,2,3] | DM | FA LRU → MRU | FA | DM class |
|---|---|---|---|---|---|---|
| 1 | 0 | `[0,–,–,–]` | M | `[0]` | M | compulsory |
| 2 | 4 | `[4,–,–,–]` | M | `[0,4]` | M | compulsory |
| 3 | 0 | `[0,–,–,–]` | M | `[4,0]` | H | conflict |
| 4 | 1 | `[0,1,–,–]` | M | `[4,0,1]` | M | compulsory |
| 5 | 2 | `[0,1,2,–]` | M | `[4,0,1,2]` | M | compulsory |

Lin Cing https://github.com/xinhi6716/computer-science-self-s



## Accesses 6–9: distinguish capacity misses from hits

Access 6 leaves four blocks in the shadow. Continue the same state, without restarting from empty.

| # | block | DM [line 0,1,2,3] | DM | FA LRU → MRU | FA | DM class |
|---|---|---|---|---|---|---|
| 6 | 3 | `[0,1,2,3]` | M | `[0,1,2,3]` | M | compulsory |
| 7 | 4 | `[4,1,2,3]` | M | `[1,2,3,4]` | M | capacity |
| 8 | 0 | `[0,1,2,3]` | M | `[2,3,4,0]` | M | capacity |
| 9 | 1 | `[0,1,2,3]` | H | `[3,4,0,1]` | M | hit |

DM has eight misses: five compulsory, one conflict, and two capacity. Accesses 7 and 8 revisit blocks and miss in the shadow too. Replacement in DM line 0 alone does not make them conflict misses.

Lin Cing https://github.com/xinhi6716/computer-science-self-s

## A shadow miss need not be an actual miss

**#9 Read block 1 (byte address 0x0010)**

**Direct-mapped**

Fixed lines 0 / 1 / 2 / 3

| block 0 | block 1 | block 2 | block 3 |
|---|---|---|---|

**HIT: block 1 remained in line 1**

State unchanged; this is not a miss.

Accesses #7 and #8 replace only line 0.

**Fully associative LRU shadow**

Before access: LRU → MRU

| block 2 | block 3 | block 4 | block 0 |
|---|---|---|---|

**MISS: access #8 evicted block 1**

Evict 2 → new order [3, 4, 0, 1]

Update the shadow, but count no actual miss.

Access 9 is an **actual hit and shadow miss**, contributing no actual miss. Both caches total eight misses: DM miss/FA hit at access 3 is canceled by DM hit/FA miss at access 9. A zero difference in totals does not erase the conflict. Classification must follow individual accesses.



## Two ways change replacement, even at the same capacity

With 64 B and 16 B blocks, a two-way cache has two sets, `set=block mod 2`, each with its own LRU order. Its pattern is `M M H M M M M M H`: seven misses. Before access 9, set 0 is `[4,0]` and set 1 is `[1,3]`, both LRU-to-MRU, so block 1 still hits.

| Design | Hits | Misses | Hit time H | Total time (additional miss penalty = 20 cycles) |
|---|---|---|---|---|
| DM | 1 | 8 | 1 cycle | `9×1+8×20=169 cycles` |
| Two-way LRU | 2 | 7 | 1.25 cycles | `9×1.25+7×20=151.25 cycles` |

Every reference pays H; a miss pays the additional penalty. All references are serialized, without overlap. For this trace, two-way is faster exactly when `9×1.25+7P < 9×1+8P`, or `P>2.25 cycles`.

FA offers more placement freedom, but **LRU is not an optimal minimum-miss replacement algorithm**. Two-way has one fewer miss on this trace. More legal placements do not make every replacement policy better on every trace.

## 5.6 System virtual machines

A system VM presents an environment in which a guest OS can execute. The VMM manages privileged state, translation, interrupts, and devices while isolating guests.

Sensitive operations must be safely trapped, translated, or directly managed by virtualization support. Guest virtual, guest physical, and host physical addresses are different levels.

A process VM such as a JVM does not present the same complete system-ISA/device contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## VM translation and I/O

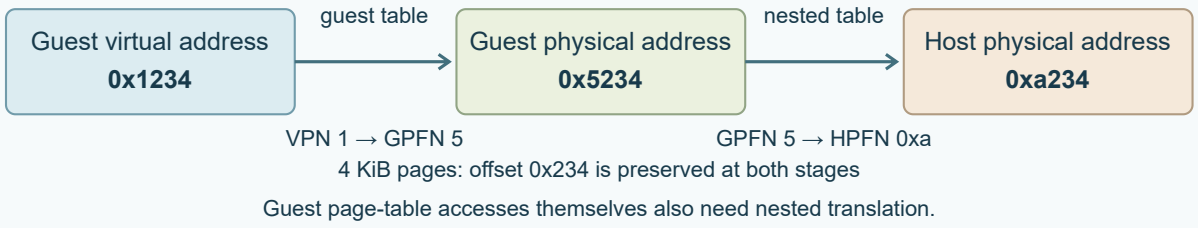
Shadow page tables can map guest VA directly to host PA while the VMM tracks guest table changes. Hardware nested paging instead performs composed walks, reducing some traps while increasing work on a translation miss.

Device emulation, passthrough, and IOMMUs make different isolation and performance tradeoffs. DMA, interrupts, queues, and ownership all need virtualization rules.

Evaluate which state is trapped or isolated before estimating the overhead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Nested paging: translating one address



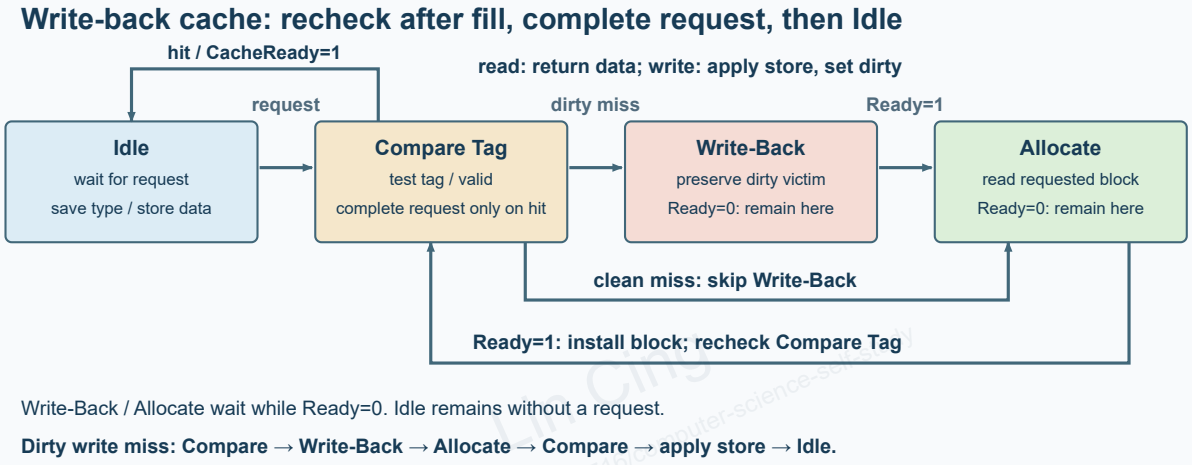

Assume 4 KiB pages at both stages, valid mappings, and permitted access. A guest instruction emits GVA `0x1234`: guest VPN `1` and offset `0x234`. The guest page table maps VPN `1` to guest physical frame `5`, producing GPA `0x5234`.



## The extra page-table translations

The nested table maps guest frame `5` to host frame `a`, producing host PA `0xa234`; both stages preserve the offset. The guest page table itself resides in guest physical memory, so reading a guest PTE also requires nested translation of that PTE’s GPA. TLB and walk-cache hits can reduce actual memory reads; no universal fixed cost follows from this example.

# 5.7 Cache-controller state machine




Idle accepts a request. Compare Tag decides hit/miss and updates a write hit. Write-Back preserves a dirty victim while waiting for memory `Ready`. Allocate fetches the new block and returns to comparison after installation.

Variable memory latency requires handshake-controlled waiting. Unasserted enables default inactive, but selectors and transaction data still need defined behavior.



# 5.7 Cache-controller state machine (continued)

Write-back must complete or become safely buffered before allocation overwrites the victim.

After Allocate receives Ready, install the block, clear its initial dirty state, and return to Compare Tag. The recheck completes the original saved read or write; only then assert CacheReady and return to Idle. Write-Back and Allocate hold their states while Ready=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## State-transition traces

```
Clean read miss:
Idle -> Compare Tag -> Allocate -> Compare Tag -> Idle
                         | !Ready: remain here

Dirty write miss:
Idle -> Compare Tag -> Write-Back -> Allocate -> Compare Tag -> Idle
                        !Ready       !Ready
```

Save the original read/write type, address, and store data while waiting. The final comparison must complete the original request rather than confuse a store with a load.

Early restart and buffering add concurrency and therefore add state invariants.



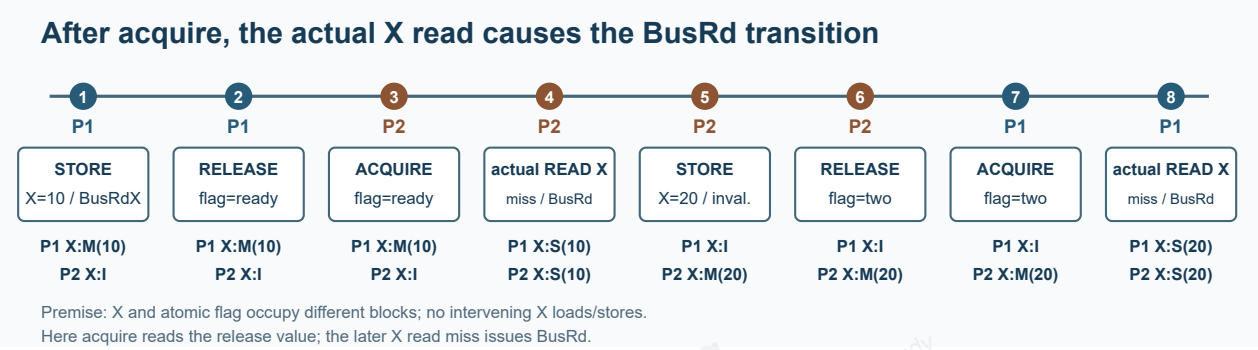
## 5.8 Cache coherence

Private caches can hold multiple copies of one physical address. Coherence specifies read values for ordered accesses and a common serialization of writes to one location. At the hardware level, a stale read among unordered accesses does not by itself prove a coherence error; the consistency model determines which ordering exists. In languages such as C++, a non-atomic data race can be undefined behavior, so hardware observations are not automatically language guarantees.

The required properties include a processor reading its own earlier write when no intervening effective write to that address occurs, all processors observing writes to one address in the same order, and a synchronized read returning the preceding ordered write when no later same-address write intervenes.

Consistency defines cross-location and synchronization ordering; coherence regulates one location's copies and values. An atomic operation removes a particular race only if its specified synchronization covers those conflicting accesses. It does not repair unrelated unordered accesses.

# Write-back MSI after intervening writes

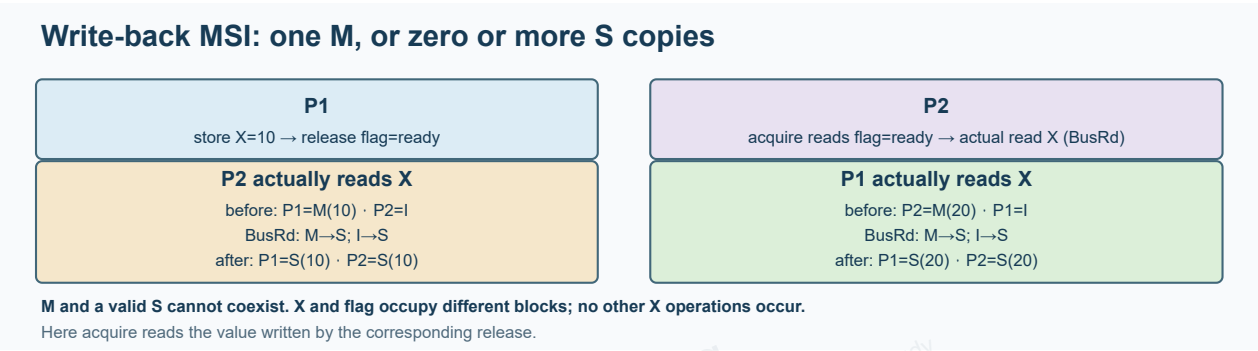



P1 writes 10 and releases `ready`. P2 must actually read that released value with acquire before its later read of X; acquire itself does not read X. The X read miss issues BusRd and the M owner supplies 10.

P2 then writes 20 and releases `two`. After P1 acquires that value, its subsequent read obtains 20. The example places X and flag in separate blocks and assumes no other X accesses between these events.



# MSI invariants and acquire versus data reads




Assume X and atomic flag occupy different cache blocks, and no additional X operations occur between the listed events. In this example, the acquire directly reads the value written by the matching release, establishing synchronizes-with.

## MSI invariants and acquire versus data reads (continued)

| Event | P1 X | P2 X | Ordering and memory consequence |
|---|---|---|---|
| initial | I | I | X not cached, no flag synchronization |
| P1 store X=10, BusRdX | M(10) | I | local cache updated, no cross-thread HB yet |
| P1 release flag=ready | M(10) | I | orders prior X store; dirty X need not flush |
| P2 acquire reads ready | M(10) | I | synchronizes with release; no X access/state change |
| P2 actually reads X, BusRd | S(10) | S(10) | M owner supplies 10, downgrades; memory becomes 10 |
| P2 store X=20, upgrade | I | M(20) | P2 obtains exclusivity and invalidates P1 |
| P2 release flag=two | I | M(20) | orders X=20 before flag write |
| P1 acquire reads two | I | M(20) | synchronizes; P1 has not read X yet |
| P1 actually reads X, BusRd | S(20) | S(20) | P2 supplies 20, downgrades; memory becomes 20 |

At most one cache can hold M, and when M exists no other valid S exists. Each read miss obtains the newest value from the M owner and downgrades that owner to S. Acquire on flag is not a load of X. The actual X load causes BusRd. Sharing X's cache block with flag would add false-sharing traffic excluded by the premise here.



## Snooping and write invalidation

Snooping caches observe broadcast coherence traffic. Scaling a broadcast domain costs bandwidth. A write-invalidate protocol obtains exclusive ownership before modifying a line.

| MSI state | Meaning |
|---|---|
| I | Invalid |
| S | Readable shared copy, possibly replicated |
| M | Sole valid cached copy; memory may be stale |

A write from S invalidates other copies. A remote read of M requires the owner to supply the newest data. Basic MSI returns both caches to S and makes memory current; a dirty shared owner needs an extended protocol such as MOESI.

## 5.9 Designing a cache controller

A cache controller handles variable-latency memory with a state machine. Idle accepts a request; Compare Tag determines hit or miss; Write-Back preserves a dirty victim; Allocate waits for the lower-level block. While memory has not responded, `Ready=0`. On completion, install the data and tag, set valid, clear the new line's initial dirty bit, and return to Compare Tag. Only the rechecked hit that completes the original read or write asserts `CacheReady`. For a saved write, apply its store data and set dirty before returning to Idle.

Retain request type, store data, fault/retry context, and victim ownership across waits. Before overwriting the victim slot, Write-Back must complete or move its contents into a buffer with guaranteed completion. Allocation cannot bypass that ordering requirement.

Useful invariants are that each valid line's tag identifies its data, dirty contents are preserved before their only storage is overwritten, responses occur only when the requested data is ready, and reset/error handling leaves no half-completed transaction. Assertions or model checking can test these stated invariants.



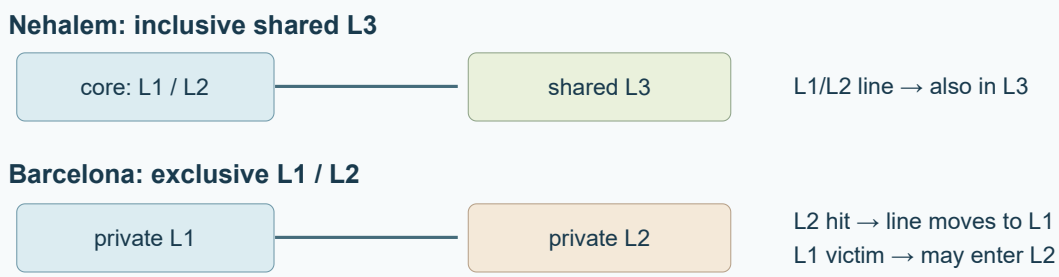
## 5.10 Historical systems

The Barcelona Opteron X4 and Nehalem examples combine on-die caches, TLBs, and memory controllers. Private L1/L2 and shared lower-level storage reduce some off-chip communication while adding coherence, area, and power costs.

The examples illustrate sharing, inclusion, victim relationships, and intercore traffic. They are historical configurations, not current product rankings. An L2 miss can hit a noninclusive L3; noninclusion does not make that impossible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Historical examples: inclusive and exclusive caches




L3 behavior depends on the model; do not assume every level is exclusive.

Intel's Nehalem white paper describes per-core 32 KiB instruction and data caches, a 256 KiB unified L2, and up to 8 MiB of shared inclusive L3. Inclusive means that a valid block in a core's L1/L2 also has a corresponding L3 copy; an L3 miss can therefore help rule out hits in other cores' private caches.

Patterson and Hennessy's Opteron X4 2356 (Barcelona) example instead describes exclusive L1/L2 caches: an L2 hit can move the block to L1 while the displaced L1 block enters L2. Its shared L3 is not always exclusive for blocks shared among cores. Do not extend the L1/L2 rule to every level; eviction, dirty data, and coherence depend on the actual protocol.



## 5.11 Fallacies and pitfalls

For a 32-byte direct-mapped cache with four-byte blocks, byte address 36 denotes block 9 and index $9 \bmod 8 = 1$. Capacity is not the block count.

Do not mix local and global miss denominators, instructions and data references, or time and cycles. A larger L1 can improve hit rate yet lose because its hit path is slower.

Write units and denominators before inserting numerical rates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.12 The common hierarchy model

Locality, block transfer, placement, search, replacement, write propagation, and miss cost connect caches with paging. Their parameter scales differ greatly.

Multiple levels distribute competing objectives. Software blocking, loop ordering, and prefetching can improve locality but require workload measurements. Increasing L3 alone does not resolve bandwidth, coherence, or scheduling costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 5.13 Historical perspective

Limited memory, relocation, and protection motivated virtual memory and caching. Paging, TLBs, set associativity, and coherence became recurring solutions.

Complex implementations still preserve architectural contracts for process addresses and memory effects. Locality, protection, and movement costs remain relevant when technology ratios change.

Historical device values are evidence about a particular design period, not immutable ratios.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## End-of-chapter check

Decode byte addresses, trace hits and dirty victims, derive AMAT and stall CPI, distinguish translation and data faults, and trace an invalidation protocol. State 3 C and consistency-model boundaries.

Use exercises independently, then inspect solutions. Every result needs the placement, timing, and write assumptions that make it valid.

### References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 5. CSO course slides, memory hierarchy. Intel, Next Generation Microarchitecture (Nehalem), 2008, https://www.intel.com/pressroom/archive/reference/whitepaper_Nehalem.pdf

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

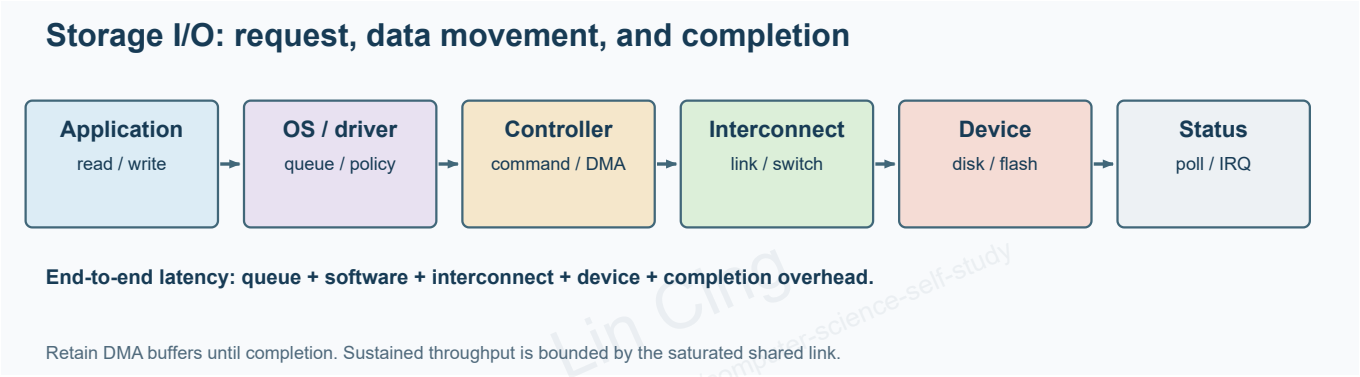
## The complete request path

A file read crosses the file system, driver, controller, media, and completion path. Issuing a command, placing data in memory, and allowing the application to use a buffer are separate events.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The complete request path (continued)




Fix request size, read/write mix, queue depth, cache state, and completion semantics. A write acknowledged from volatile controller cache is not necessarily durable after power loss. Capacity, latency, throughput, and reliability require separate measures.

## Capacity, bandwidth, and IOPS denominators

GB/TB mean $10^9/10^{12}$ bytes; GiB/TiB mean $2^{30}/2^{40}$ bytes. Divide bit/s by eight and account for protocol traffic before claiming payload byte/s.

$$100000\ \mathrm{IOPS} \times 4096\ \mathrm{B} = 409600000\ \mathrm{B/s} = 0.4096\ \mathrm{GB/s}.$$

At 64 KiB per request, the same rate would require 6.5536 GB/s, which the device/link may not sustain. IOPS depends on request size. Response time spans submission to completion; the reciprocal of one request's latency does not establish concurrent throughput.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.1 Workload-specific design objectives

Interactive queries prioritize tail latency; backups need sustained bandwidth and recoverability; transactions also require durable commits and failure atomicity. Identical SSD counts can face different bottlenecks under different access and completion contracts.

Devices, controllers, interconnect, memory, and CPU command processing all impose limits. Added devices consume power, queue entries, interrupt capacity, and recovery time. Peak bandwidth alone guarantees neither low latency nor availability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.2 Failure, reliability, and availability

Failure means service deviates from its specification; restoration returns it to compliance. Redundancy can mask a component fault before it becomes a visible service failure.

Reliability R(t) is the probability of uninterrupted correct service through t. Availability is the probability of correct service at an observation time, or its long-run time fraction, allowing repairs. Fast repair improves availability without erasing an earlier interruption from reliability history.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Deriving availability from renewal cycles

Let each cycle have nonnegative uptime U and repair duration D. Assume iid cycle pairs and $0 < E[U] + E[D] < \infty$. The law of large numbers gives

$$A_k = \frac{\sum_{i=1}^{k} U_i}{\sum_{i=1}^{k} (U_i + D_i)} \longrightarrow \frac{E[U]}{E[U] + E[D]}.$$

Thus $A = MTTF/(MTTF + MTTR)$. Individual durations need not equal their means, and the ratio does not bound failures per year. With 100000 h MTTF and 4 h MTTR, A=0.9999600016. Expected long-run downtime in 8760 h is $8760(1 - A)60 = 21.02$ minutes. Reducing MTTR to 0.5 h gives 2.63 minutes, saving 18.39 minutes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A mean lifetime is not a failure deadline

Only an additional constant-hazard exponential model gives

$$R(t) = e^{-\lambda t}, \qquad MTTF = 1/\lambda, \qquad P(T \le t) = 1 - e^{-t/MTTF}.$$

For 100000 h MTTF, the one-year failure probability is $1 - e^{-8760/100000} = 8.39\%$. This guarantees neither 100000 h of survival nor at most one yearly failure. Aging, shared power, and environmental damage can invalidate constant hazards or independence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.3 Disk geometry and logical capacity

Tracks, sectors, surfaces, and cylinders form a legacy fixed-geometry model:

$$C = \text{cylinders} \times \text{surfaces} \times \text{sectors/track} \times \text{bytes/sector}.$$

10000 cylinders, four surfaces, 500 sectors/track, and 512 bytes/sector give 10240000000 bytes=10.24 GB=9.5367 GiB. If these are already logical addressable sectors, do not arbitrarily deduct raw gaps/ECC again. Modern LBA/controller translation does not expose this formula as physical geometry.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Components of one random read

Ignoring queues,

$$T_s = T_{seek} + T_{rotation} + T_{transfer} + T_{controller}.$$

At 12000 RPM, a rotation is 5 ms. Uniform random arrival angle gives 2.5 ms mean rotation. With 500 sectors/track, one sector takes 0.01 ms. Seek 4.5 ms and controller 0.10 ms give

$$T_s = 4.5 + 2.5 + 0.01 + 0.10 = 7.11 \text{ ms}.$$

One outstanding unreordered request gives about 140.65 IOPS. More concurrency may change locality/scheduling and average service time, so this is not a universal queue-depth-independent bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Sequential requests amortize positioning

A merged 100-sector request stays on one track and pays no extra setup or rotation. Transfer takes 1 ms:

$$4.5 + 2.5 + 1 + 0.10 = 8.10 \text{ ms}.$$

51200 payload bytes yield 6.321 MB/s. A hundred independent random requests instead take 711 ms in this model. The saving pays positioning once; it does not accelerate rotation 100 fold. Real sequential models must include track crossings, prefetch, and controller overlap.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.4 NAND pages and erase blocks

NOR supports fine-grained reads and selected execute-in-place firmware uses. Denser NAND normally reads/programs pages and erases larger blocks. An FTL maps logical updates to new physical pages, invalidates old pages, and garbage-collects erase blocks instead of overwriting arbitrary words like DRAM.

An eight-page erase block with six valid and two invalid pages requires copying six survivors before erase. The eight reclaimed positions minus six survivor positions leave two net host-update positions. Two host pages plus six copied pages cause eight programs, so WA=8/2=4 for this cycle. Erases are not directly added as written bytes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Endurance estimates and their limits

$$WA = \frac{\text{media bytes written}}{\text{host bytes written}}.$$

120 GB host and 384 GB media giveWA 3.2. With 400 GB physical NAND, 3000 P/E cycles, uniform wear, fixed WA, and no other reserves,

$$400(3000)/3.2 = 375000\ \text{GB} = 375\ \text{TB}$$

of host writes is the rough budget. It is not a warranty. Bad-block reserves, temperature, retention, metadata, and wear leveling matter. Overprovisioning usually helps garbage collection, but a simultaneous working-set change confounds its effect on WA.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.5 Links, buses, and bottlenecks

A shared bus serializes competing devices. Point-to-point links/switches allow independent paths but can share upstream links, controllers, and memory channels. Device peaks cannot simply be summed into application throughput.

$$X \le \min(6(80000), 120000, 300000, 150000) = 120000 \text{ IOPS}.$$

The terms represent SSDs, HBA, memory, and CPU/driver. Two extra SSDs leave the HBA bound unchanged. Raising HBA capacity to 240000 exposes the 150000 CPU limit. These are same-size, same-mix capacity bounds; queues, locks, and recovery may lower actual rates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.6 Commands, notifications, and movement

MMIO maps device registers into privileged address space. Some ISAs also provide special I/O instructions. Device cacheability/ordering follows platform rules rather than ordinary RAM assumptions.

Polling/interrupts determine how completion is observed; programmed I/O/DMA determine who moves data. Either completion method can accompany DMA. A doorbell store announces descriptors only after their data becomes device-visible under the DMA/I/O API's required ordering.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Polling and interrupt costs

At 3 GHz and 20000 I/O/s, exactly one 100-cycle poll per request costs 2000000 cycles/s=0.0667% CPU. A 1500-cycle interrupt per request costs 30000000 cycles/s=1%. Merely adding 100 runnable threads does not change this declared one-poll arithmetic.

If each request instead busy-waits 50 microseconds without shared waiting, 20000 requests consume one whole core-second. The earlier 0.0667% no longer applies. A hybrid briefly polls then sleeps; coalescing and completion queues batch work. Compare setup, busy waiting, switches, batch size, and latency distributions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



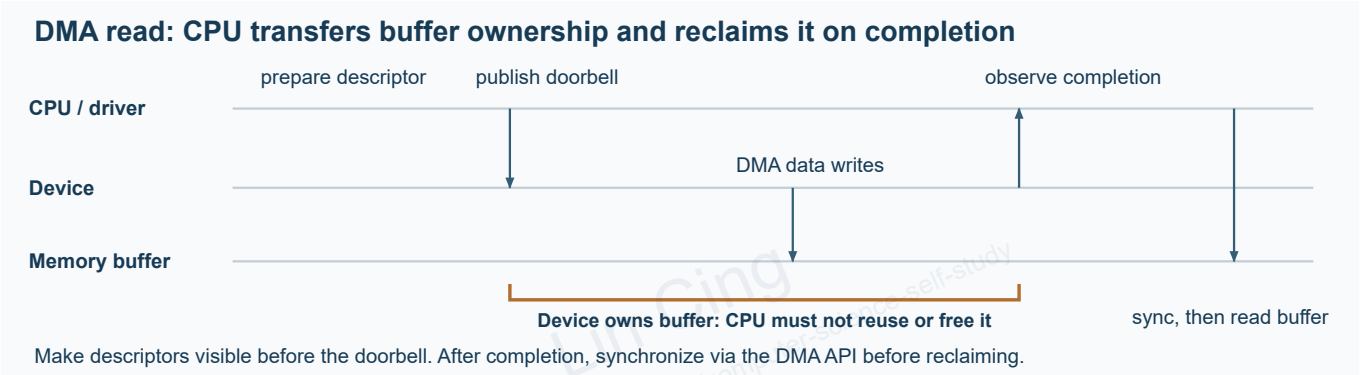
## Interrupt priority and completion processing

The device sets pending state; enables/priorities determine acceptance. The driver identifies the source, reads status, processes completions/errors, acknowledges according to device rules, and wakes waiters. A shared interrupt may belong to another device.

A low-priority handler masking all interrupts for 8 microseconds makes a high-priority event arriving after 1 microsecond wait at least 7 microseconds. Nesting requires separate saved context, protected shared state, and bounded depth. Coalescing lowers handler count but can delay the first completion notification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## DMA buffer ownership




For a device-to-memory read, CPU allocates/maps the buffer and publishes descriptors. While the device owns it, CPU cannot free, reuse, or consume incomplete contents. The device finishes writes and publishes completion; CPU observes completion, performs API-required visibility synchronization, and reacquires ownership.



## DMA buffer ownership (continued)

Noncoherent platforms require properly timed clean/invalidate operations. Cleaning an old dirty CPU line after DMA wrote new memory can overwrite the new data. A timeout does not prove DMA stopped. Before unmapping, ensure the device can issue no more writes to pages that may be reassigned.

## DMA saves core copying work

$$64(2^{20})/0.020 = 3.3554432\ \mathrm{GB/s}.$$

A 500-cycle setup at 3 GHz takes 0.1667 microseconds. Programmed I/O takes 16 copy cycles plus 10 transaction cycles per 16 bytes:

$$(64(2^{20})/16)(26) = 109051904\ \mathrm{cycles},$$

or 36.35 ms of CPU execution before waits. DMA still consumes interconnect, memory, IOMMU, and completion work; setup time is not end-to-end transfer time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.7 Little's law

For stable flow with finite averages, mean requests in the system L, completion rate lambda, and response W satisfy

$$L = \lambda W.$$

The area under the request-count-over-time curve equals the sum of request residence times. Divide by observation length and let residual boundary effects vanish. L includes waiting and service; queue-only measures use Lq/Wq.

100000 IOPS at 80 microseconds mean response implies L=8. Setting queue depth eight does not guarantee that rate, because changing concurrency may change W. The law checks mutually consistent steady-state observations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why M/M/1 response rises near saturation

Assume Poisson arrivals lambda, independent exponential service rate mu, one FCFS server, unlimited waiting room, and rho=lambda/mu<1. Balance gives $p_n\lambda = p_{n+1}\mu$, hence $p_n = p_0\rho^n$. Normalization yields $p_0 = 1 - \rho$. Differentiating the geometric series gives

$$L = (1-\rho)\sum_{n\geq 0} n\rho^n = \frac{\rho}{1-\rho}, \qquad W = L/\lambda = \frac{1}{\mu - \lambda}.$$

Unchanged mean service can coexist with sharply rising queueing. At rho≥1, this finite stationary mean does not exist. A negative denominator cannot be treated as ordinary latency. The model does not assert that actual disk service is exponential.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## One device under different loads

For mu=200 IOPS, mean service is 5 ms.

| Arrival rate | Utilization | Response W | Queue Wq | Requests L |
|---|---|---|---|---|
| 100 | 0.5 | 10 ms | 5 ms | 1 |
| 160 | 0.8 | 25 ms | 20 ms | 4 |
| 190 | 0.95 | 100 ms | 95 ms | 19 |

Going from 160 to 190 increases completions 18.75% while response quadruples. This follows the declared model, not every device. Specify mean/tail latency limits alongside maximum IOPS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.8 Fixed costs and durability

Serial nonoverlapped latency components add; resource throughput bounds take a minimum. CPU/device work may overlap partially, so neither unconditional addition nor perfect overlap is justified.

Compare identical completion endpoints. A RAM acknowledgement and a durable-media acknowledgement are different services. Protected caches or explicit flush completion may define durability. Retry semantics also depend on partial completion:repeating a write may be safe, while repeating an increment can apply it twice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.9 RAID capacity and failures

Assume n equal C-capacity disks, no spares/metadata/reserves, and known whole-disk erasures with correct survivors and working controllers/power.

| Layout | Usable capacity | Recoverable failure |
|---|---|---|
| RAID0 | nC | no redundancy |
| paired RAID1/RAID10 | nC/2, n even | at most one per mirror pair |
| RAID4/5 | (n−1)C | any one disk |
| RAID6 P+Q | (n−2)C | any two under the encoding assumptions |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.9 RAID capacity and failures (continued)

### RAID 0, 5, 10, and 6 physical disk layouts

| RAID 0 | RAID 5 | RAID 10 | RAID 6 |
|---|---|---|---|
| disk0 D0/D4 | disk0 D0/D3 | disk0 D0/D2 | disk0 D0/D2 |
| disk1 D1/D5 | disk1 D1/D4 | disk1 D0/D2 | disk1 D1/P1 |
| disk2 D2/D6 | disk2 D2/P1 | disk2 D1/D3 | disk2 P0/Q1 |
| disk3 D3/D7 | disk3 P0/D5 | disk3 D1/D3 | disk3 Q0/D3 |

**Each RAID box lists disks by row; / separates stripes. RAID10 mirrors span disks; P/Q are independent checks.**

RAID tolerates specified failures and does not replace backup. Count rebuild traffic and exposure separately.

RAID3 fine-grained striping uses synchronized access. RAID4 block striping distributes small reads but concentrates parity writes. RAID5 rotates parity, removing the fixed parity-disk bottleneck without eliminating parity work.



## XOR recovery proof and example

Let $P = D_0 \oplus D_1 \oplus D_2$. Associativity, commutativity,$x \oplus x = 0$, and $x \oplus 0 = x$ give

$$P \oplus D_0 \oplus D_2 = D_1.$$

For 1010, 1100, 0110, parity is 0000. Recovering D1 computes 0000 XOR 1010 XOR 0110=1100. Two unknown blocks leave one equation and generally multiple solutions. Silent wrong values have unknown locations; recovering a known erasure does not locate arbitrary corruption.

## Small writes and full stripes

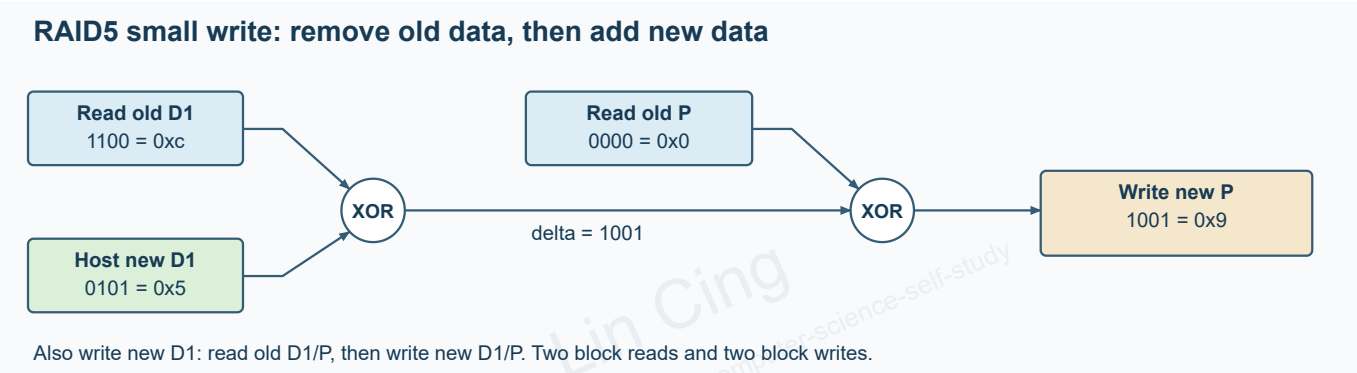
$$P' = P \oplus D_{old} \oplus D_{new}.$$

Reading old data/parity and writing new data/parity costs two reads/two writes. Changing 1100 to 0101 gives parity 0000 XOR 1100 XOR 0101=1001.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Small writes and full stripes (continued)




A full m-data-block stripe gets all new data from the host and writes m+1 blocks without old reads. For k updates, RMW costs 2 k+2 operations; reconstruct-write reads m−k unchanged blocks and writes k+1, total m+1. Compare those workloads with cache state and parallel bottlenecks.

## RAID5 read/write mixture bound

Eight disks at 100 block I/O/s provide 800 aggregate operations/s. A small read costs one; a small RMW write costs four. With read fraction r,

$$\text{operations/request} = r + 4(1-r) = 4 - 3r, \qquad X \le 800/(4-3r).$$

At r=0.75, the bound is 457.14 IOPS. Balanced traffic, no combining, and no rebuild are premises. Locks, controller limits, and one overloaded disk can reduce it. RAID4's parity disk alone needs two operations per small write, limiting writes to half its capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Duplicate parity is not RAID6

If Q merely copies P, the second equation adds no information. In a field GF(2^w), use distinct nonzero coefficients ai:

$$P = \sum_i D_i, \qquad Q = \sum_i a_i D_i.$$

Field addition is XOR. Removing known survivors for erasures i, j gives $X = D_i + D_j$,$Y = a_i D_i + a_j D_j$. Distinct coefficients ensure $a_i + a_j \neq 0$, so

$$D_j = (Y + a_i X)/(a_i + a_j), \qquad D_i = X + D_j.$$

Division uses a field inverse, not ordinary integer division mixed with XOR.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A reproducible P+Q example

Use GF(16) modulo binary 10011, polynomial x 4+x+1, with coefficients 1, 2, 4. Multiply by two by shifting; when the old high bit is one, truncate to four bits and XOR 0011.

$D_0 = 1, D_1 = 2, D_2 = 3$ give $P = 1 \oplus 2 \oplus 3 = 0$ and $Q = 1 \oplus 4 \oplus 12 = 9$. If $D_0, D_1$ disappear, $X = P \oplus D_2 = 3$ and $Y = Q \oplus (4D_2) = 9 \oplus 12 = 5$. Thus $3D_1 = Y \oplus X = 6$. In the stated field modulo binary `10011`, $3 \cdot 14 = 1$, so $3^{-1} = 14$ and $6 \cdot 14 = 2$. Hence $D_1 = 2$ and $D_0 = 3 \oplus 2 = 1$. Actual layouts may use other fields or coefficients; this proves the stated code's recovery.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The parity write hole

Data and parity are separate writes. Power loss after new D1=0101 becomes durable but before old parity 0000 changes leaves P unequal to XOR(data). Correctly readable blocks can still belong to inconsistent versions.

Parity algebra alone supplies no atomic update. Protected write cache, journaling, or stripe transactions need explicit ordering, durability, and restart rules. Redundancy also cannot recover legitimate deletions; it does not replace independent backup.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rebuild traffic denominators

A 12-disk 10 TB RAID5 is 90%full and rebuilds only nine valid TB per disk. Each missing block reads 11 survivors, so reads 99 TB and writes 9 TB. At shared aggregate 1 GB/s,

$$T \geq (99+9)10^{12}/10^{9} = 108000\ \mathrm{s} = 30\ \mathrm{h}.$$

If 1 GB/s instead means replacement output, nine TB takes 2.5 h but needs 11 GB/s surviving reads. Full-disk reconstruction reads 110 TB+writes 10 TB, total 120 TB, and takes at least 33.33 h on the aggregate path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A second failure during rebuild

An eight-disk RAID5 has already lost one disk. Fix repair time r=10 h and independent constant survivor hazards lambda=10^-5/h. All seven survive with probability

$$(e^{-\lambda r})^{7} = e^{-7\lambda r}.$$

$$P(\text{another failure}) = 1 - e^{-7(10^{-5})(10)} = 0.000699755 = 0.06998\%.$$

This conditions on degraded entry and one repair interval. It is not annual array data-loss probability. Uncorrectable reads, correlated failures, and variable repair durations require additional models.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.10 Historical Sun Fire and power boundaries

The book's circa 2008 Sun Fire x4150 is a 1 U case with a maximum configuration of two four-core Xeon sockets, 64 GB DRAM, eight 15000 RPM SAS disks, and RAID controller. These historical specifications are not current product claims. A 19-inch rack has 1 U=1.75 inches; 42 U=73.5 inches, subject also to power/cooling limits.

A separate teaching model uses idle 200 W, full 400 W, and 2000 transactions/s, or 0.2 J/transaction. Half load at 300 W and 1000/s costs 0.3 J/transaction. Consolidating two half-loaded machines saves energy only if capacity, tail latency, fault tolerance, and migration costs remain acceptable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Different workloads on the same historical configuration

A teaching workload, not the book's measurement, uses eight cores each sustaining 10^9 instructions/s. Each 64 KiB I/O consumes 250000 user+150000 OS instructions, giving CPU bound 20000 IOPS.

Eight random-read disks at 4 ms service provide 2000 IOPS. Sequential disks at 112 MB/s each provide $8(112 \times 10^6)/65536 = 13671.875$ IOPS. A 2 GB/s RAID-card payload link provides 30517.578 IOPS.

Ignoring other conflicts, random throughput is bounded by 2000 and sequential by 13671.875, both disk-limited. Faster SSDs exceeding 20000 IOPS can expose the CPU bound. Do not combine a historical configuration figure and this new workload into a product benchmark.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.11 Network transfer and waiting

A 10 Gbit/s link with 80%payload efficiency gives 8 Gbit/s=1 GB/s.

$$64(2^{20})/10^{9} = 67.108864 \text{ ms}.$$

Sixteen 4 MiB batches have 15 inter-batch 0.20 ms gaps when the connection is established, total 3 ms, and overall 70.108864 ms. A hundred streams may hide waits but cannot create 100 GB/s on that shared link.

Propagation, serialization, queueing, and protocol setup are distinct. Bandwidth times RTT is in-flight data needed to sustain a rate, not automatic extra payload per request.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.12 Counterexamples to common claims

DMA still costs setup/completion CPU time and memory bandwidth. Two parity blocks require independent encoding and do not solve write holes. High MTTF is a mean rather than a survival deadline.

Equal IOPS at 4 KiB and 64 KiB imply different bytes/s. More concurrency can increase response time, as the M/M/1 table shows. RAID cannot undo deletion or an attacker overwriting all replicas. Each counterexample identifies its violated premise instead of asserting a universal device rule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.13 Inferring bottlenecks from observations

When throughput stops rising while response grows, measure utilization, queue length, and per-I/O work for each resource. Little's law checks common steady-state accounting but does not localize saturation by itself.

Fast acknowledgements followed by power-loss data loss require a completion/durability investigation. Slow rebuilds require separate survivor-read, replacement-write, and shared-link accounting. Both performance and failure analysis begin with the end-to-end service contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.14 Technology changes, analysis persists

Mechanical positioning, out-of-place NAND updates, and shared links have different costs but still require separate capacity, service, queueing, and failure models. Switches and SSDs change leading costs without removing ownership, completion, or persistence obligations.

Historical cases explain tradeoffs. New comparisons need matching workload and completion data rather than treating 2008 peaks as modern constants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References

Patterson and Hennessy,*Computer Organization and Design: The Hardware/Software Interface*, Revised Fourth Edition, 6.1–6.14, printed pp.570–618, PDF pp.597–645. RAID layouts and small writes correspond to figures 6.13/6.14, printed pp.603–604, PDF pp.630–631. M/M/1 and GF(16) examples develop the stated supplemental models. See exercises and solutions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Why parallel processing

Power density, clock distribution, limited ILP, and memory stalls constrain a faster single core. Parallel hardware aims to increase completed work or shorten a large problem's elapsed time.

A multicore puts cores on one die. A multiprocessor can span sockets. A cluster spans computers and a network. Communication latency, coherence, and failure boundaries differ. Every speedup needs an explicit workload and baseline.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Kinds of parallelism

Independent jobs can run concurrently without dividing one job. A parallel program instead distributes one problem among workers, subject to dependencies.

ILP overlaps operations in one instruction stream. Data parallelism applies similar work to many elements. Task parallelism distributes different functions or requests. These can coexist in one system.

Decide whether the goal is single-request latency or aggregate throughput before choosing a decomposition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.1 Power-aware design

Lower voltage and frequency can reduce dynamic energy per operation while reducing per-core speed. Several slower cores may improve aggregate work per watt only if software exposes useful parallel work.

Replicated services can continue after a node failure, but require detection and recovery. Core count alone does not provide availability. Scalable software must remain correct across counts, cache capacities, and topologies.

An efficiency claim needs voltage/frequency policy, workload, and system power boundaries.

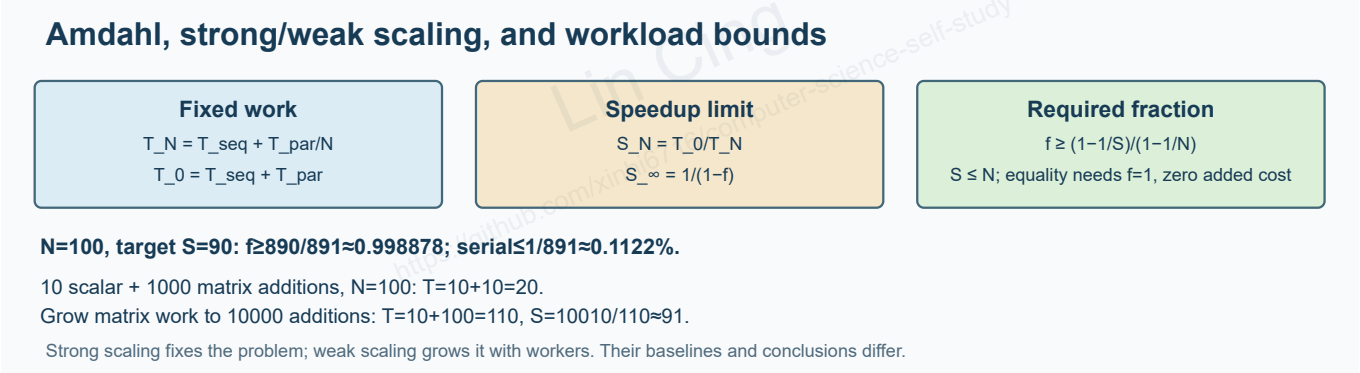


## 7.2 Amdahl's fixed-work model

A fixed workload takes baseline single-thread time $T_0$. Fraction f of that time parallelizes perfectly among N workers; the rest remains serial. With no added costs,

$$T_N = T_0(1-f) + T_0 f/N, \qquad S_N = T_0/T_N = 1/(1-f+f/N).$$

Efficiency is $E_N = S_N/N$, distinct from occupancy. For $N = 8$ and $S_8 = 4.70588$, $E_8 = 0.588235 = 58.8235\%$.



## 7.2 Amdahl's fixed-work model (continued)

$$S_\infty = 1/(1-f).$$

The asymptote holds fixed workload, baseline time fractions, and costs. It is not a measured machine result.

### Inverting a speedup target

From $1/S = 1 - f(1 - 1/N)$, a target $1 < S < N$ requires

$$f \geq \frac{N(S-1)}{S(N-1)}.$$

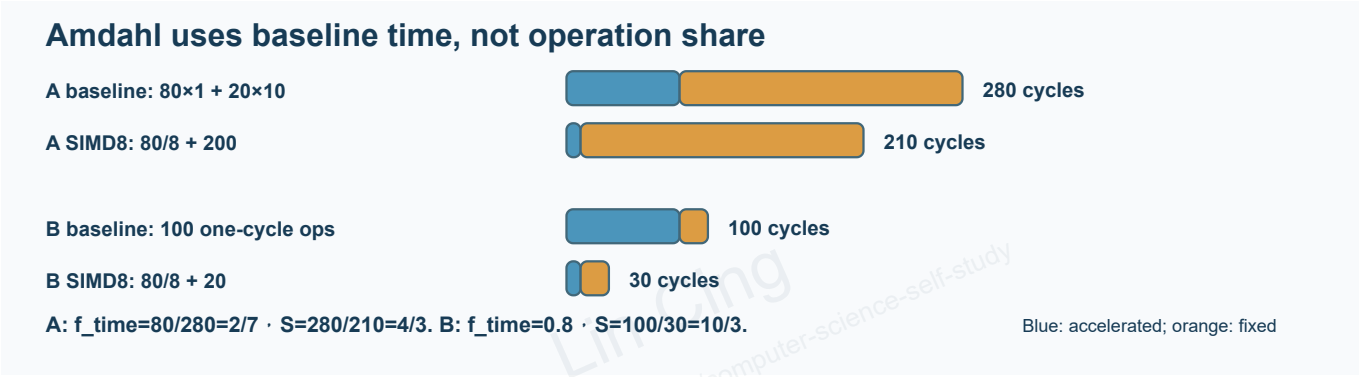
N=100, S=90 requires f≥8900/8910=0.998878 and serial fraction≤0.1122%, not merely 90% parallel work.

$$T_{N,observed} = T_N + T_{critical} + T_{sync} + T_{contention}.$$

Launch, imbalance, synchronization, and shared bandwidth are excluded from the ideal model. Verify which costs are negligible before attributing a measured gap to one cause.



## Operation fractions versus time fractions




Eighty one-cycle operations vectorize; twenty ten-cycle operations do not. Baseline is 80+200=280 cycles. Eightfold acceleration gives 10+200=210 cycles, speedup 4/3. The accelerated baseline time fraction is 80/280=2/7.

## Operation fractions versus time fractions (continued)

| Model | Accelerated baseline time | Other time | Time after eightfold acceleration | Speedup |
|---|---|---|---|---|
| A, unequal costs | 80 | 200 | 210 | 1.3333 |
| B, all 100 operations cost one cycle | 80 | 20 | 30 | 3.3333 |

The same 80% operation share has different time shares. Substitute 0.8 into Amdahl only under equal costs or a measured 80% baseline-time fraction.



## Strong scaling, weak scaling, and efficiency

Strong scaling fixes W and increases P:

$$S_{strong}(P) = T_1(W)/T_P(W), \qquad E(P) = S_{strong}(P)/P.$$

Weak scaling grows work to PW while holding per-worker work fixed:

$$E_{weak}(P) = T_1(W)/T_P(PW).$$

Ideal weak efficiency is 1. This time ratio compares different total problem sizes and is not fixed-work speedup; do not divide it by P again. Only assuming linear single-core cost, $T_1(PW) = PT_1(W)$, permits

$$S_{scaled}(P) = T_1(PW)/T_P(PW) = PE_{weak}(P).$$

Without a larger-problem single-core time or that linear-cost assumption, the same-size speedup is unknown. Fixed total memory M gives about M/P per worker under strong scaling; growing total memory to PM holds about M per worker under weak scaling.

### Two complete addition cases

Each addition costs t; ten additions remain serial.

## Strong scaling, weak scaling, and efficiency (continued)

| Problem | P=10 | P=100 |
|---|---|---|
| Small, 10 serial+100 parallel | T=20 t, S=110/20=5.5 | T=11 t, S=110/11=10 |
| Large, 10 serial+10000 parallel | T=1010 t, S=10010/1010=9.91 | T=110 t, S=10010/110=91 |

The small fixed workload reaches the serial floor quickly. More parallel work dilutes the same serial cost in the large workload. Amdahl and weak scaling are consistent because work and baseline change together. Record problem size, per-core work, workers, baseline, bytes communicated, memory footprint, clock/power policy, and wall time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Load balance and the critical path

Let total parallel work be W, heavy-worker work Wmax, workers P, and fixed overhead C. If other workers evenly divide the remainder,

$$T_{parallel} = \max(W_{max}, (W - W_{max})/(P-1)), \qquad T_{total} = T_{parallel} + C.$$

### Complete example

W=10000, C=10, P=100. A heavy worker with 2% receives 200 units; others each get 9800/99=98.99.

$$T = \max(200, 9800/99) + 10 = 210, \qquad S = 10010/210 = 47.67, \quad E = 47.7\%.$$

At 5% heavy work,

$$T = \max(500, 9500/99) + 10 = 510, \qquad S = 10010/510 = 19.63, \quad E = 19.6\%.$$

Average worker load does not predict completion. The slowest worker or dependency chain controls it. Work stealing adds queue/atomic metadata and migration-related cache/network traffic; tiny tasks may cost more to coordinate than the balance gain saves.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.3 Shared-memory multiprocessors

A shared physical address space permits communication with loads and stores. Coherence maintains the single-location copy/order contract.

UMA approximates equal memory access costs from different processors. NUMA gives local and remote memory different latency and bandwidth. It still provides a shared address space. Page placement and task affinity therefore matter.

Directory protocols can scale coherence beyond a small broadcast snooping domain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-st



## Shared-memory synchronization

Atomicity prevents externally visible intermediate RMW state. Ordering specifies which operations precede others. Visibility determines the value a consumer sees once those orderings exist. Coherence supplies same-location serialization; cross-location publication needs synchronization from the memory model.

### Shared-data publication: atomicity, ordering, visibility

**Release orders data before ready. An acquire that reads that release value synchronizes with it, so the reader's later data load sees 42.**

**Separate relaxed variant: both objects atomic; ready=1 can precede a data load returning 0.**

Conflicting unprotected non-atomic accesses race. Acquire/release orders only accesses covered by its contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Shared-memory synchronization (continued)

### Publication example

```
int data = 0;                // shared object
atomic_int ready = 0;
// Writer
data = 42;
atomic_store_explicit(&ready, 1, memory_order_release);
// Reader
while (!atomic_load_explicit(&ready, memory_order_acquire))
    yield();
int observed = data;
```

Only an acquire that actually reads the writer's released 1 synchronizes with that store. Program order on each side then forms a happens-before chain from data=42 to its load, giving observed=42.



## Shared-memory synchronization (continued)

For a defined counterexample, make both data and ready atomic and use memory_order_relaxed for all four accesses. The writer stores data=42 before ready=1; a reader observing ready=1 may still load data=0 because no cross-object synchronizes-with relation exists. Coherence orders each atomic object separately. Plain non-atomic conflicting accesses without other synchronization instead cause a C/C++ data race and undefined behavior; their possible results cannot be restricted to 0 and 42.

Synchronization covers only the conflicting accesses it orders. A third thread writing outside that protection still races; an atomic flag does not make a multi-location transaction atomic. Spinlocks consume interconnect/power under contention. Blocking mutexes pay OS/context costs. Barriers wait for all participants; a tree can reduce depth to O(logP) while retaining per-node synchronization/wakeup costs.

## 7.4 Clusters and message passing

Cluster nodes have separate physical address spaces and typically separate OS instances. Sends, receives, RPCs, and queues transfer information explicitly.

Encoding, buffering or copying, routing, receiving, and any acknowledgment contribute cost. Isolation does not remove recovery logic. Specialized MPP networks and commodity cluster networks make different price, latency, and bandwidth tradeoffs.

A distributed shared-memory abstraction still needs communication beneath it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 7.5 Hardware multithreading

Hardware threads retain separate PCs, register state, and selected controls while sharing execution and memory resources.

Fine-grained multithreading selects among threads frequently, often every cycle. Coarse-grained switching responds to long stalls. SMT fills superscalar issue slots with ready work from different threads in the same cycle.

All can hide latency, but they add state and competition. Two threads/core do not guarantee twice the throughput, and a thread may take longer when sharing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.6 Hardware and programming classifications

SISD has one instruction and data stream. MIMD has independent instruction/data streams. SIMD broadcasts an operation over multiple elements. SPMD is a programming model in which threads execute the same program on different data.

SPMD can target MIMD clusters or SIMT GPUs. It is not a separate hardware width. SIMD suits regular contiguous operations, while divergent control and irregular dependencies reduce lane use.

Separate software decomposition from hardware execution before assigning a label.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Vector architecture

A vector ISA expresses operations over a vector length, while hardware executes through lanes, pipelines, and chaining. Vector length need not equal physical lane count. Fixed-width multimedia SIMD often requires explicit width-dependent blocks and tail handling.

Let each element require $k$ operations, with $L$ lanes each completing one element/cycle, frequency $f$, and useful utilization $\eta$.

$$throughput = kLf\eta \text{ operations/s}.$$

For lane initiation interval $q$ cycles/element, divide by $q$. Alternatively, vector length $W$ and issue rate $r$ vectors/cycle give $kWrf$. Do not multiply both descriptions together and count work twice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.7 GPU positioning

GPUs use many threads, warps, and high-bandwidth memory for aggregate throughput. CPUs prioritize fewer latency-sensitive streams. GPU scheduling hides latency with other ready work rather than making every thread's operation fast.

Transfers, launches, divergence, coalescing, occupancy, and synchronization determine end-to-end gains. Appendix A develops the execution and memory model.

Core-count ratios are not a scalar-program speedup formula.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

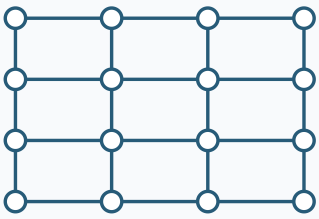


## 7.8 Multiprocessor network topologies

Represent endpoints and links as a graph. Count a full-duplex physical link once; state whether cut bandwidth counts one direction or both.

### A four-by-four mesh

**4×4 mesh: 24 local links, no wraps**



**Mesh metrics**

horizontal = 4×(4−1) = 12
vertical = 4×(4−1) = 12
edges = 24; average degree = 48/16 = 3
corners degree 2; interior degree 4
diameter = (4−1)+(4−1) = 6
bisection links = 4

**No opposite boundaries connect, so this is a mesh rather than a torus.**

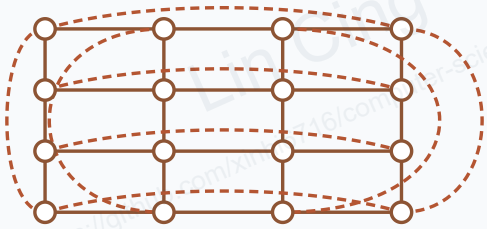
$$|E| = 4(4-1) + 4(4-1) = 12 + 12 = 24.$$

## 7.8 Multiprocessor network topologies (continued)

Degree sum=48, average 3; corners have degree 2, edges 3, interior 4. The longest shortest path is 3 horizontal+3 vertical=6 hops. Splitting columns 0–1 from 2–3 crosses four horizontal links; the minimum balanced cut has four.

### A four-by-four torus

**4×4 torus: 24 local + 8 dashed wrap = 32 links**



**Torus metrics**

degree = 4 for every node
wrap links = 4 horizontal + 4 vertical
total edges = 24 + 8 = 32
diameter = floor(4/2)×2 = 4
bisection links = 8

**H0–H3: (0,r)↔(3,r). V0–V3: (c,0)↔(c,3). Dashed lines are all eight wraps.**

Every row and column forms a four-node cycle. Former mesh corners have degree four in the torus.



## 7.8 Multiprocessor network topologies (continued)

Add four row wraps and four column wraps to the mesh:

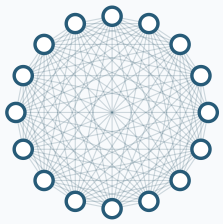
$$|E| = 24 + 4 + 4 = 32, \qquad degree = 4, \qquad diameter = 2 + 2 = 4.$$

Each dimension's shortest wrap distance is at most floor(4/2)=2. The same column cut crosses four local and four wrap links; minimum bisection is 8. Horizontal wraps Hr connect (0, r)↔(3, r), r=0..3. Vertical wraps Vc connect (c, 0)↔(c, 3), c=0..3. Omitting one of these eight changes the endpoint degrees; omitting all leaves a mesh.

## 7.8 Multiprocessor network topologies (continued)

### K16 and a central crossbar

**K16: every pair of 16 endpoints has a direct link**



**K16 metrics**

edges = N(N−1)/2 = 16×15/2 = 120

degree = 15

diameter = 1

bisection = (N/2)² = 8² = 64 links

One-way cut capacity = 64×link rate

**K16 and a central crossbar use different endpoint and switch-resource models.**

For full-duplex links, both cut directions total 2×64×one-way rate. State the counting convention.

$$|E| = 16(15)/2 = 120, \quad degree = 15, \quad diameter = 1.$$

Every balanced 8/8 cut crosses 64 links. At one-way link rate b, one-way bisection bandwidth is 64 b. A central N-port crossbar instead normally connects each endpoint to one port and has $O(N^2)$ internal crosspoints. Its arbitration, wires, endpoint degree, and port limits need a switch model.



## 7.8 Multiprocessor network topologies (continued)

Total link bandwidth sums link capacities; bisection uses the smallest balanced cut. Neither determines application latency by itself. Hotspots, routing, message size, and flow control matter.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.9 Multiprocessor benchmarks

Fix program, data set, correctness validation, compiler, processor count, and scaling rule. Record growing problem size explicitly for weak scaling.

Dense-linear-algebra benchmarks, independent-job rate tests, and shared-memory application suites stress different resources. Peak FLOP/s is not measured application performance. Communication, memory bandwidth, imbalance, synchronization, energy, and tails can dominate.

Report the scaling curve rather than one largest-machine number.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 7.10 Deriving Roofline

A phase performs F FLOPs and transfers Q bytes across a specified memory interface. Arithmetic intensity is I=F/Q. Sustained compute ceiling PC and interface bandwidth B imply

$$T_{compute} = F/P_C, \qquad T_{memory} = Q/B = F/(IB).$$

Even with complete overlap,

$$T \geq \max(F/P_C, F/(IB)), \qquad P = F/T \leq \min(P_C, IB).$$

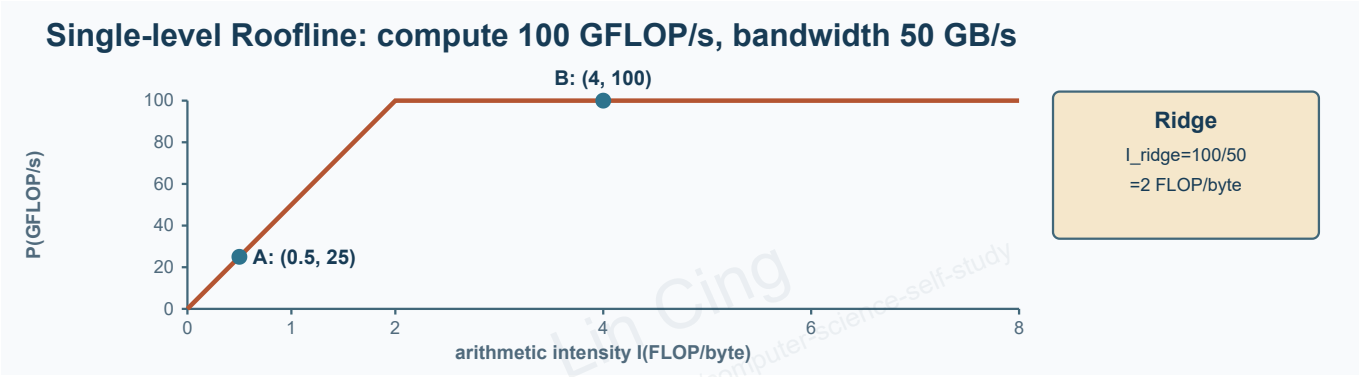
Equating the two roofs gives $I_{ridge} = P_C/B$. Lower intensities are memory-limited; higher ones are compute-limited. The bound is not a promise that dependencies, mix, imbalance, and transaction efficiency permit reaching it.

### Single-level working points

PC=100 GFLOP/s and B=50 GB/s give ridge 2 FLOP/byte. A at I=0.5 has memory roof 25 and bound 25 GFLOP/s. B at I=4 has memory roof 200 but compute caps it at 100 GFLOP/s.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.10 Deriving Roofline (continued)

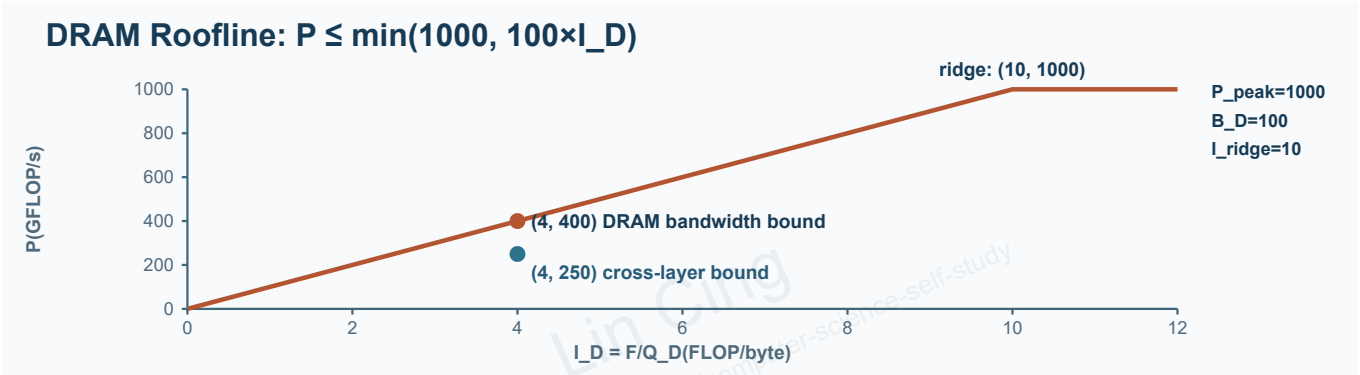



### One phase across two memory interfaces

Fix $F = 10^9$ FLOPs and $P_C = 1000$ GFLOP/s. DRAM transfers $Q_D = 0.25 \times 10^9$ bytes at $B_D = 100$ GB/s; shared memory transfers $Q_S = 4 \times 10^9$ bytes at $B_S = 1000$ GB/s.

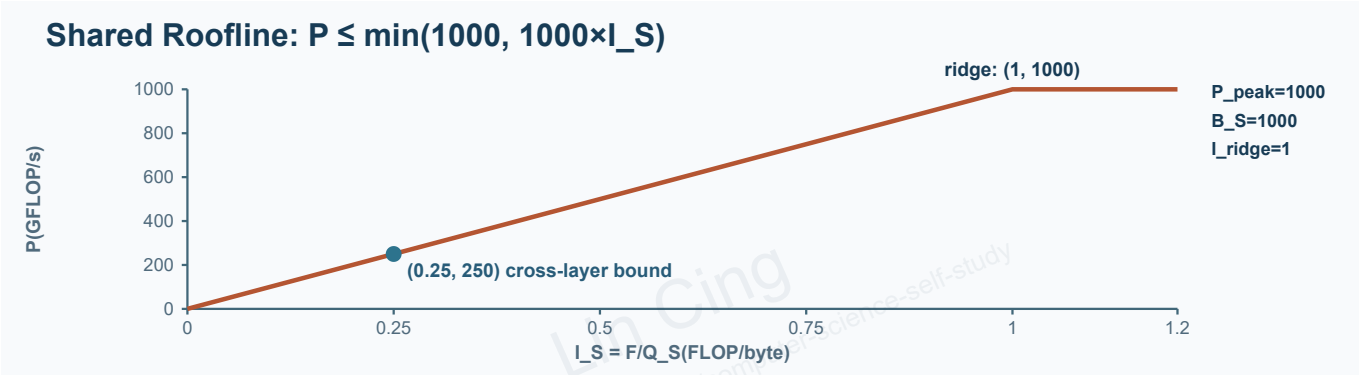


## 7.10 Deriving Roofline (continued)




$$I_D = 10^9/(0.25 \times 10^9) = 4, \qquad P_D \le \min(1000, 100(4)) = 400.$$

## 7.10 Deriving Roofline (continued)




$$I_S = 10^9/(4 \times 10^9) = 0.25, \qquad P_S \le \min(1000, 1000(0.25)) = 250.$$

$$P \le \min(1000, 400, 250) = 250 \text{ GFLOP/s},$$

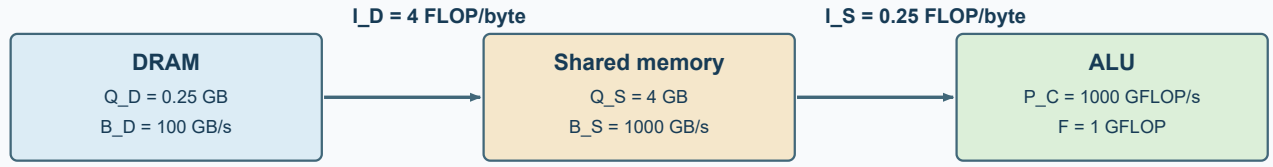
$$T \ge \max\left(\frac{10^9}{10^{12}}, \frac{0.25 \times 10^9}{100 \times 10^9}, \frac{4 \times 10^9}{1000 \times 10^9}\right) \text{ s} = \max(1, 2.5, 4) \text{ ms} = 4 \text{ ms}.$$



## 7.10 Deriving Roofline (continued)

DRAM's independent working point is(4, 400). The shared constraint lowers the cross-layer bound to(4, 250) when plotted using ID. Shared's own point is(0.25, 250), on its slope; its plateau remains 1000, not 250. These are model bounds, not measured rates.

## 7.10 Deriving Roofline (continued)




**T_compute = 1 ms; T_DRAM = 2.5 ms; T_shared = 4 ms.**

T ≥ max(1, 2.5, 4) = 4 ms; P ≤ min(1000, 400, 250) = 250 GFLOP/s.

A cache hit changes which interface carries bytes. Recompute intensity with that interface's bytes and bandwidth; never combine DRAM traffic with cache bandwidth. Separate roofs may be drawn separately, but the final same-phase bound takes the minimum across resources.



## 7.11 Historical multicore comparisons

The book compares four systems using sparse-matrix and structured-grid kernels. Their compute peaks, bandwidth, caches, TLBs, and programming environments differ. The distance from a measured point to its applicable ceiling can suggest a testable optimization opportunity.

Indirect sparse accesses and regular stencils can behave differently even at similar intensity because latency, compiler output, and access regularity differ. Historical results teach a method, not a current hardware ranking.

## 7.12 Parallel fallacies

Weak scaling does not refute a fixed-work Amdahl bound. Peak FLOP/s does not determine the best machine for a memory-bound application. More threads can add oversubscription, cache pressure, synchronization, or power limits.

Coherence does not make racing software race-free. Correctness evidence and scaling evidence answer different questions.

Construct an execution trace or measured counterexample rather than relying on a slogan.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 7.13 Parallel-system synthesis

Hardware supplies resources; software decomposition, balance, communication, synchronization, and locality determine their useful work. Shared-memory and message-passing interfaces expose different costs.

Roofline bounds compute and bandwidth together. Measurements and counters test why a point lies below it. They do not turn a peak number into delivered application performance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## End-of-chapter check

Compute speedup with serial work and overhead. Distinguish strong from weak scaling. Explain SMP/NUMA/cluster addresses, compare thread scheduling, calculate graph cuts and hops, and derive Roofline bounds with consistent units.

Use the separate exercises and solutions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 7.14 Historical perspective

Vector instructions amortize control over regular operations. Shared-memory multiprocessors extend load/store programming while requiring coherence and synchronization. Clusters scale resources through explicit messages and distinct failure boundaries.

Multicore, SMT, and GPUs exploit different task, thread, and data opportunities. None removes a serial critical path or makes remote communication local. Compare historical designs at fixed numerical validation, problem size, capacity, traffic, and power conditions.

In K16, every endpoint has fifteen direct links and a balanced cut has $8 \times 8 = 64$. A sixteen-port central crossbar normally gives each endpoint one switch attachment and has internal quadratic crosspoints. Do not combine their graph metrics.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References

Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Revised Fourth Edition, sections 7.2 and 7.8–7.10, printed pages 635–674 (physical PDF pages 662–701).

Williams, Waterman, and Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM* 52(4), 2009, DOI.

The level-specific traffic convention follows the NERSC Roofline documentation. Each intensity uses bytes at its own interface. Numerical values here are declared model inputs and derived results.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A.1 Mapping parallel work to a GPU

GPUs trade complex low-latency control for many execution units and high bandwidth. An individual GPU thread need not beat a CPU thread, and small workloads may not amortize launch/transfer costs. Programmable graphics and compute share processors, while rasterization, texturing, and ROP operations also use fixed functions. API pipeline stages need not own separate physical SM cores.

CUDA/Tesla terminology gives historical context. The 32-byte-sector, 32-bank, and occupancy examples explicitly use abstract parameters; combining them does not describe one GeForce 8800.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Interpolation in the graphics pipeline

Vertex shaders produce positions/attributes; clipping and rasterization produce fragments; fragment shading, depth/stencil, and blending update a framebuffer. A fragment need not become a visible pixel.

For screen vertices (0, 0),(2, 0),(0, 2), all clip w=1, the point (0.5, 0.5) has barycentric weights (1/2, 1/4, 1/4). They are nonnegative and sum to one. Pure R/G/B attributes interpolate to (0.5, 0.25, 0.25). For different w, correct perspective weights are

$$\frac{\lambda_i/w_i}{\sum_j \lambda_j/w_j}.$$

Screen-linear weights alone no longer suffice. Texture filtering reads neighboring texels; ROP may read old color/depth, so traffic exceeds one final color store.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A.2 Host, device, and end-to-end costs

A discrete GPU has device DRAM and an I/O connection managed by drivers. Integrated/shared-memory designs can reduce copying but still require visibility, permissions, and bandwidth accounting. Unified addresses do not make movement free.

Assume sequential input/output copies without compute overlap. Input/output are 64 MiB each, link 8 GB/s each direction, launch 20 microseconds, kernel 2 ms:

$$T_{copies} = 2(64)(2^{20})/(8 \times 10^9) = 16.777216 \text{ ms}.$$

Total is 18.797216 ms. Against a 25 ms CPU, overall speedup is 1.330, not kernel-only 12.5. Retaining data for ten kernels costs 36.977216 ms with one input/output pair. Actual reuse depends on data dependencies and use cases.



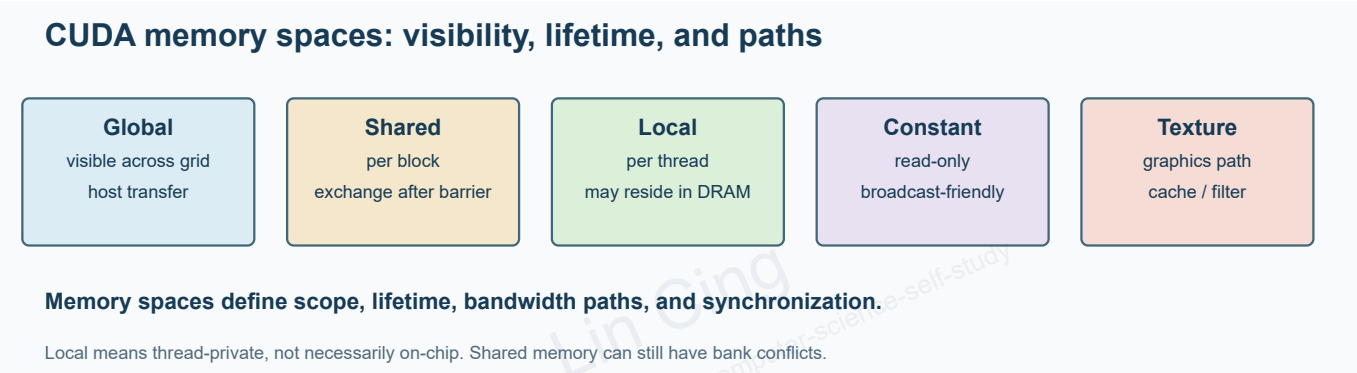
## A.3 Grid, block, and thread indices

The mapping is $i = blockIdx.x\, blockDim.x + threadIdx.x$. N=1000, B=256 needs four blocks; the last has 232 valid outputs 768..999 and 24 inactive threads.

```
i = block_id * B + thread_id
if i < N:
    y[i] = f(x[i])
```

Every valid i has unique quotient/remainder (⌊i/B⌋, i mod B), proving coverage and exclusive writes. Skipping inactive threads is safe here because no barrier follows. With a collective barrier, predicate computation instead of returning required participants early.

## Memory-space visibility and lifetime




Registers and local memory are thread-private; local arrays/spills usually reside in device DRAM. Shared memory is block-private scratch space lasting for the block. Global data can cross blocks but visibility follows synchronization rules. Constant memory can broadcast identical warp addresses. Texture paths have format/filter/cache semantics distinct from arbitrary loads. A memory-space name alone does not guarantee latency.

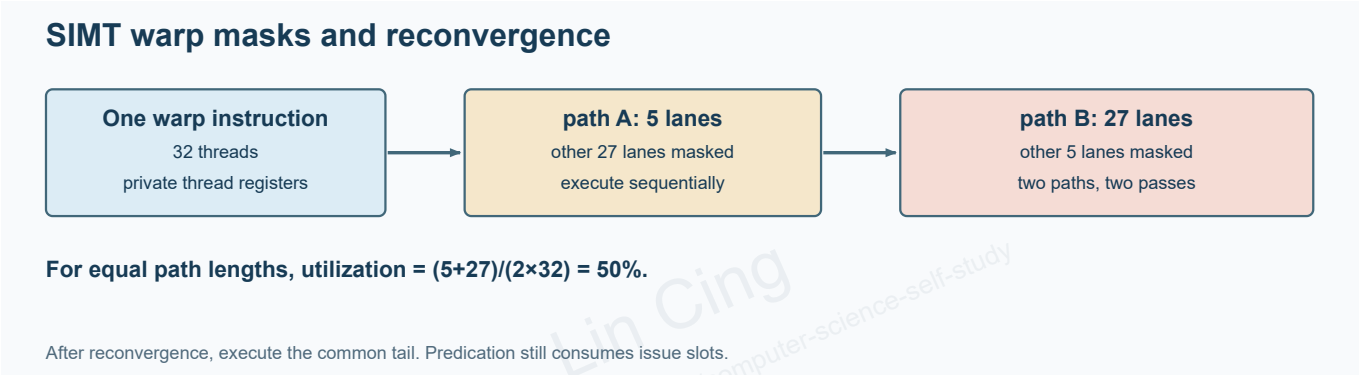


## A barrier is more than a timer

A block barrier requires the specified participants and establishes memory visibility. It is not a general cross-block barrier. Apparent warp lockstep does not waive required synchronization.

For partial tiles, all threads predicate-load valid data or identities, reach a barrier, perform valid-output work, and reach another barrier. The second prevents a fast thread overwriting data still read by a slower one. Blocks need not run concurrently or in order. Resident blocks spinning for unscheduled blocks may occupy every resource and deadlock. Separate kernel phases provide a global completion boundary without that residency assumption.

## A.4 SMs, warps, and scoreboards




A warp issues together while each thread retains registers/logical state. Historical Tesla uses 32-thread warps and eight SPs, serving lanes across four core cycles. This throughput width does not make every result latency four cycles.



## A.4 SMs, warps, and scoreboards (continued)

A scoreboard tracks dependencies; the scheduler issues ready warps. Other ready warps can cover a stalled warp, but high residency does not ensure readiness when all wait on a barrier or memory. PTX is a virtual ISA compiled/JITed for a target. One PTX instruction need not equal one cycle or one fixed-size SASS instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A finite latency-hiding model

With one warp instruction issued every four cycles and a 32-cycle dependent chain per warp, eight ready warps give 32 cycles between same-warp issues. Four warps give 16 cycles and must wait 16 more. A 400-cycle dependency with only one outstanding chain per warp would need 100 warps, possibly beyond capacity. ILP, independent loads, caches, and memory parallelism change this requirement. Occupancy alone cannot guarantee zero stalls.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Weighted divergence in lane slots

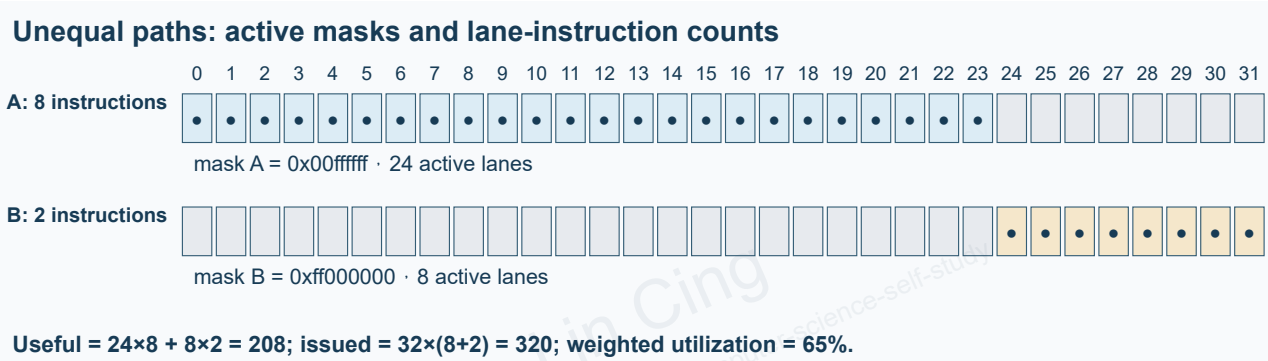
For W lanes, active counts a+b=W, and path lengths LA/LB, when both mutually exclusive paths execute,

$$\eta = \frac{aL_A + bL_B}{W(L_A + L_B)}.$$

Only equal path lengths reduce this to one-half. W=32, a=24, b=8, LA=8, LB=2 give 208 useful lane-instructions/320 slots=65%. This is not kernel speedup or DRAM efficiency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Weighted divergence in lane slots (continued)




A path with no active lane can be omitted from the denominator. Predication still consumes issue capacity; false-predicate memory lanes do not automatically generate DRAM transactions.



## Occupancy is an integer resource problem

Use abstract limits 768 threads, 24 warps, eight blocks, 32768 registers, and 48 KiB shared per SM, warp size 32, ignoring allocation granularity. Blocks have 256 threads, 32 registers/thread, and 16 KiB shared:

$$blocks \le \min(\lfloor 768/256 \rfloor, 8, \lfloor 32768/(256(32)) \rfloor, \lfloor 48/16 \rfloor) = 3.$$

Three blocks give 24 warps, 100% occupancy. With 128 threads but 16 KiB shared/block, three blocks give 12 warps, 50%. Reducing shared to 8 KiB permits six blocks and 24 warps. For the original 256-thread version, 64 registers/thread instead limits to two blocks, 16/24 occupancy. These are residency bounds, not measured utilization. Reducing registers can cause spills and make higher occupancy slower.

## A.5 Counting coalesced sectors

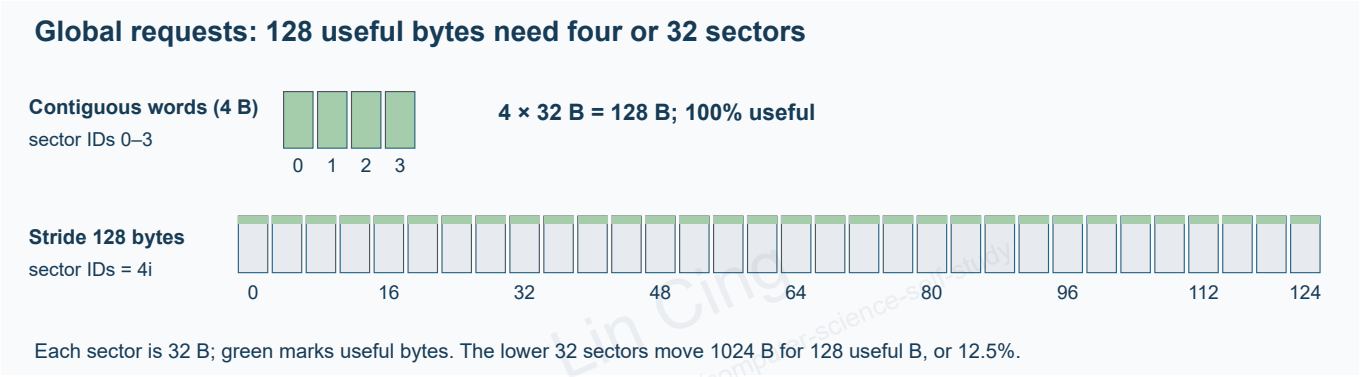
Assume 32 lanes, four-byte words, and 32-byte sectors; count unique sectors for one warp request, ignoring cache/merging. This is not the complete GeForce 8800 rule set. For byte addresses ai, the sector set is $\{\lfloor a_i/32 \rfloor\}$.

Aligned contiguous 128 bytes use four sectors. Starting at +4 uses five, 160 transferred bytes for 128 useful, 80%. Stride 32 or 128 bytes gives 32 sectors, 1024 bytes, 12.5% useful. Equal sector counts need not imply equal channel/row locality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A.5 Counting coalesced sectors (continued)




All lanes reading one word request only four unique bytes, possibly served by one sector plus broadcast. Thirty-two logical uses do not become 128 distinct bytes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Shared banks, broadcast, and padding

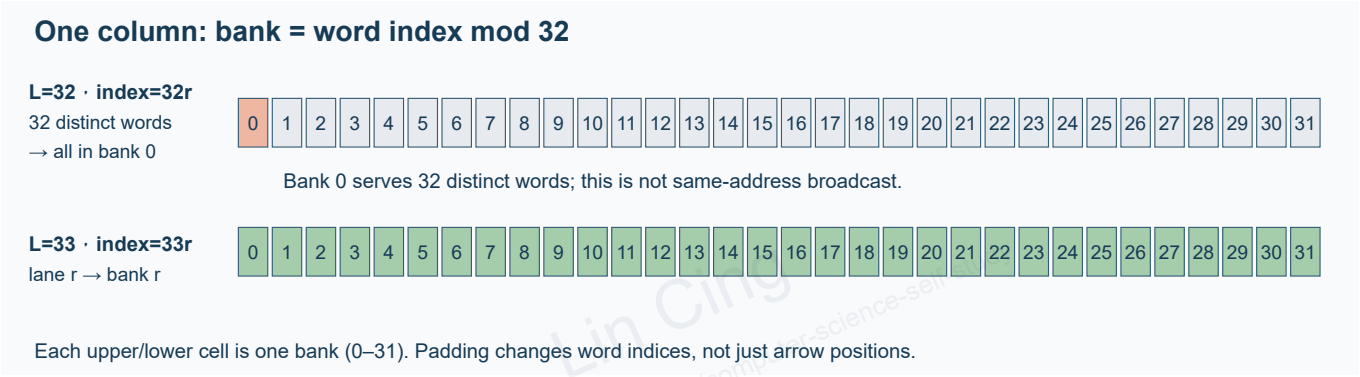
Assume 32 banks, four-byte words, bank=word index mod 32. Different banks serve in parallel; distinct words in one bank serialize. Identical-word reads can broadcast.

Column 0 of a 32 x 32 row-major array addresses 32r, so 32 distinct words collide in bank 0. Leading dimension 33 makes bank=r, using all banks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Shared banks, broadcast, and padding (continued)




Padding grows shared footprint 4096→4224 bytes and may reduce resident blocks at an integer threshold. Historical Tesla's 16-bank/half-warp rules require recounting 16 simultaneous requests; this 32-way result is not its measured behavior.

## A.6 Floating-point order is part of the model

Binary32 has 24 significand bits. Choose representable $a = 1 + 2^{-23}$, $b = 1 - 2^{-23}$, and $c = -1$. The exact product is $ab = 1 - 2^{-46}$. Rounding that product to binary32 gives 1, so adding $c$ then gives 0. A single-round IEEE FMA first computes the exact $ab + c = -2^{-46}$, which is exactly representable, so its result is $-2^{-46}$. Historical G80 MAD must not be assumed to have these FMA semantics. Subnormal, approximate-function, and exception support are version-specific.

Changing a reduction tree can change rounding. Binary32 `(10^8+1)+(-10^8)` gives 0 while `(10^8-10^8)+1` gives 1. An integer associativity proof does not establish identical floating-point bits. Specify error/reproducibility requirements separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A.7 Historical GeForce 8800 structure

The book's figure A.7.1 has eight TPCs, two SMs per TPC, and eight SPs per SM, total 128 SPs. Six 64-bit DRAM partitions provide a 384-bit interface. SM shared/constant/SFU paths differ from texture/ROP functions.

This structure check does not predict current GPU performance. Generations differ in caches, warps, FP, and transaction rules; equal core counts can have different clocks, mixes, and dataflow. ALU/SFU peaks can be counted together only when both architecture and workload permit that simultaneous use.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A.8 CSR SpMV data

$$A = \begin{bmatrix} 2 & 0 & -1 & 0 \\ 0 & 3 & 0 & 4 \\ 5 & 0 & 0 & 6 \end{bmatrix}, \qquad x = (1, 2, 3, 4)^T.$$

Ap=[0, 2, 4, 6], Aj=[0, 2, 1, 3, 0, 3], Av=[2,−1, 3, 4, 5, 6]. Row i occupies `[Ap[i], Ap[i+1])`. Thus $y_0 = 2(1) - 1(3) = -1$, $y_1 = 3(2) + 4(4) = 22$, and $y_2 = 5(1) + 6(4) = 29$. Ap has rows+1 entries ending at nnz; Aj stays within column bounds. This rectangular matrix needs x length 4 and y length 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## One thread per row

```
row = block_id * B + thread_id
if row < rows:
    total = 0
    for k = Ap[row] .. Ap[row+1]-1:
        total += Av[k] * x[Aj[k]]
    y[row] = total
```

One writer owns each row. The invariant says total equals products already processed in that row's interval; termination gives the dot product. Empty rows output zero.

If 31 rows in a warp have one entry and one has 64, the simplified masked loop runs 64 rounds. There are 95 useful lane-iterations/2048 slots=4.64%. A warp-per-row mapping distributes a long row but can waste more lanes on short rows. Caching x helps only when reuse exceeds loading/synchronization costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Inclusive and exclusive scan

For [3, 1, 7, 0, 4, 1, 6, 3], inclusive scan is [3, 4, 11, 11, 15, 16, 22, 25]; exclusive scan is [0, 3, 4, 11, 11, 15, 16, 22]. Hillis–Steele reads each previous-round snapshot at distances 1, 2, 4:

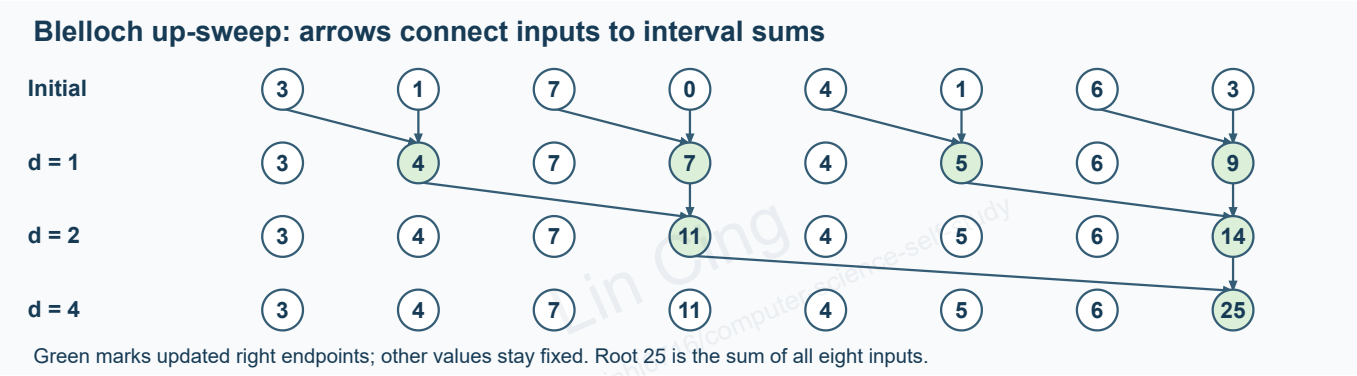
| Distance | Array after round |
|---|---|
| initial | [3, 1, 7, 0, 4, 1, 6, 3] |
| 1 | [3, 4, 8, 7, 4, 5, 7, 9] |
| 2 | [3, 4, 11, 11, 12, 12, 11, 14] |
| 4 | [3, 4, 11, 11, 15, 16, 22, 25] |

Unsynchronized in-place writes may read current-round results. This case uses 7+6+4=17 adds; general work Θ(n log n), span Θ(log n). Small depth alone is not work efficiency.



## Work-efficient up-sweep

Blelloch scan combines disjoint intervals in a binary tree.



## Work-efficient up-sweep (continued)

| d | Array |
|---|---|
| 1 | [3, 4, 7, 7, 4, 5, 6, 9] |
| 2 | [3, 4, 7, 11, 4, 5, 6, 14] |
| 4 | [3, 4, 7, 11, 4, 5, 6, 25] |

Each internal node sums its two disjoint child intervals. Induction on height proves its interval sum. There are $n - 1$ additions and $\log_2 n$ levels.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Down-sweep carries preceding prefixes

Save total 25 and replace root with zero. If a parent stores the sum p before its interval and the left subtree sum is t, the left child gets p, the right gets p+t. These are precisely the prefixes before their respective intervals.

| Step | Array |
|---|---|
| root=0 | [3, 4, 7, 11, 4, 5, 6, 0] |
| d=4 | [3, 4, 7, 0, 4, 5, 6, 11] |
| d=2 | [3, 0, 7, 4, 4, 11, 6, 16] |
| d=1 | [0, 3, 4, 11, 11, 15, 16, 22] |

Induction down the tree gives each leaf the sum of earlier elements, proving exclusive scan. Down-sweep also takes n−1 adds, total 2(n−1) work and 2 log_2 n span, plus swaps/root setup. Pad non-power-of-two sizes with zero; N=0 returns empty output.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Shared-memory scan pseudocode

For power-of-two n, all participants execute every round; disjoint k positions run in parallel.

```
for d = 1, 2, 4, ..., n/2:
    parallel for k = 0, 2*d, 4*d, ...:
        x[k+2*d-1] += x[k+d-1]
    block_barrier()
if thread_id == 0:
    total = x[n-1]
    x[n-1] = 0
block_barrier()
for d = n/2, ..., 2, 1:
    parallel for k = 0, 2*d, 4*d, ...:
        t = x[k+d-1]
        p = x[k+2*d-1]
        x[k+d-1] = p
        x[k+2*d-1] = p + t
    block_barrier()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One designated thread saves total, then publishes it as needed. This in-place tree algorithm does not require Hillis–Steele's double-buffer snapshot; their synchronization rules differ.



## Multiblock scan and global phase boundaries

For N=10, B=4, input [3, 1, 7, 0, 4, 1, 6, 3, 2, 5], zero-padded local scans give block sums [11, 14, 7]. Scanning these gives offsets [0, 11, 25]. Adding offsets yields [0, 3, 4, 11, 11, 15, 16, 22, 25, 27], total 32.

Run local scan/sums, scan sums, and add offsets as ordered phases. Recurse over many sums. Kernel completion supplies global order without simultaneous block residency. Fixed-size blocks produce a geometric series of per-level data sizes, so total work O (N), abstract tree span O (logN). Actual time includes launches, traffic, and finite-SM scheduling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Reduction invariant and values

Merge the back half into the front with strides 4, 2, 1. Each active partial represents a disjoint subset; merging preserves that invariant and ends with the total.

| Stride | Active partials |
|---|---|
| 4 | [3+4, 1+1, 7+6, 0+3]=[7, 2, 13, 3] |
| 2 | [7+13, 2+3]=[20, 5] |
| 1 | [20+5]=[25] |

Seven adds complete the reduction. Multiple blocks can reduce block sums or use supported atomics. Atomic updates prevent lost additions but do not fix interblock floating-point summation order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Stable radix partition destinations

For unsigned three-bit keys [7, 2, 5, 0, 3, 4, 1, 6], bit 0 zero flags are [0, 1, 0, 1, 0, 1, 0, 1], exclusive prefixes [0, 0, 1, 1, 2, 2, 3, 3], total zeros Z=4. Zero destinations are Zi; one destinations are Z+i−Zi. Dest=[4, 0, 5, 1, 6, 2, 7, 3], producing [2, 0, 4, 6, 7, 5, 3, 1]. Prefixes increase in original order within each partition, so destinations are distinct, cover 0..7, and preserve stability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## LSD radix sort induction

| Completed bits | Array |
|---|---|
| bit 0 | [2, 0, 4, 6, 7, 5, 3, 1] |
| bits 0..1 | [0, 4, 5, 1, 2, 6, 7, 3] |
| bits 0..2 | [0, 1, 2, 3, 4, 5, 6, 7] |

Assume the previous b low bits are sorted. Stable partition on bit b separates its values while preserving lower-bit order within each group, so b+1 bits are sorted. k passes of O (N) scan/scatter give O (kN) work. Separate input/output storage prevents overwrite. Signed/FP keys need a specified order-preserving transform; raw encoding need not match unsigned order.



## N-body acceleration and force

$$\mathbf{a}_i = G \sum_{j \neq i} \frac{m_j (\mathbf{x}_j - \mathbf{x}_i)}{(\|\mathbf{x}_j - \mathbf{x}_i\|^2 + \varepsilon^2)^{3/2}}.$$

Target mass belongs in force $m_i \mathbf{a}_i$, not acceleration's numerator. Softening changes the physical model as well as avoiding singularity. For x=[0, 1, 3, 4], unit masses/G, epsilon=0, and omitted self terms:

$$a_0 = 1 + 1/9 + 1/16 = 169/144, \quad a_1 = -1 + 1/4 + 1/9 = -23/36, \quad a_2 = 23/36, \quad a_3 = -169/144.$$

Total force zero checks pairwise signs; finite precision/order can introduce small error.

## N-body tiling and two barriers

Each B-thread block owns B targets and cooperatively loads B sources into shared memory.

```
i = block_id * B + thread_id
my = (i < N) ? body[i] : zero_body
acc = 0
for base = 0; base < N; base += B:
    j = base + thread_id
    sh[thread_id] = (j < N) ? body[j] : zero_body
    block_barrier()
    if i < N:
        for k = 0 .. min(B,N-base)-1:
            if base+k != i:
                acc += acceleration(my, sh[k])
    block_barrier()
if i < N:
    output[i] = acc
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## N-body tiling and two barriers (continued)

The first barrier completes tile publication; the second prevents early reuse. Inactive targets still help load and participate. Each valid target processes every valid source once, matching the direct mathematical sum. Pair work remains $\Theta(N^2)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Which traffic tiling reduces

Assume N divisible by B, no cache reuse, one separate my load per target, and N source loads before skipping self computation. Naive sources=$N^2$; tiled=N/B blocks each loading N sources, total $N^2/B$.

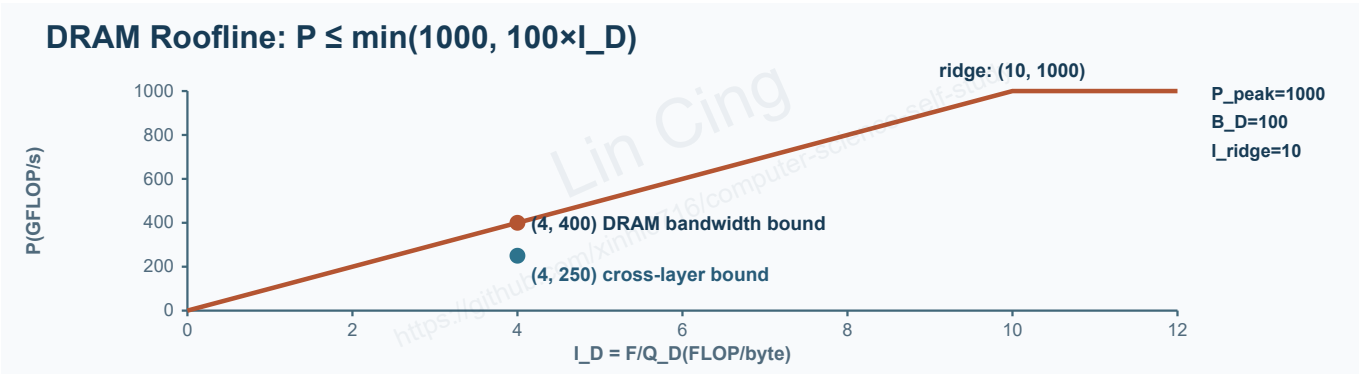
At N=4, B=2, sources fall 16→8, but four my loads make totals 20→12, only a 40% total reduction rather than an exact halving. Compiler/cache reuse and body-field widths require interface-specific byte recounting. Shared sources may still be read by B targets, and output traffic remains. Reduced DRAM traffic does not reduce every hierarchy's bytes equally.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Do not mix traffic boundaries

Fix one phase with $F = 10^9$ FLOPs and sustained compute ceiling $P_C = 1000$ GFLOP/s. It transfers $Q_D = 0.25 \times 10^9$ bytes through DRAM at $B_D = 100$ GB/s and $Q_S = 4 \times 10^9$ bytes through shared memory at $B_S = 1000$ GB/s. These are declared model inputs, not measured specifications of one GPU.

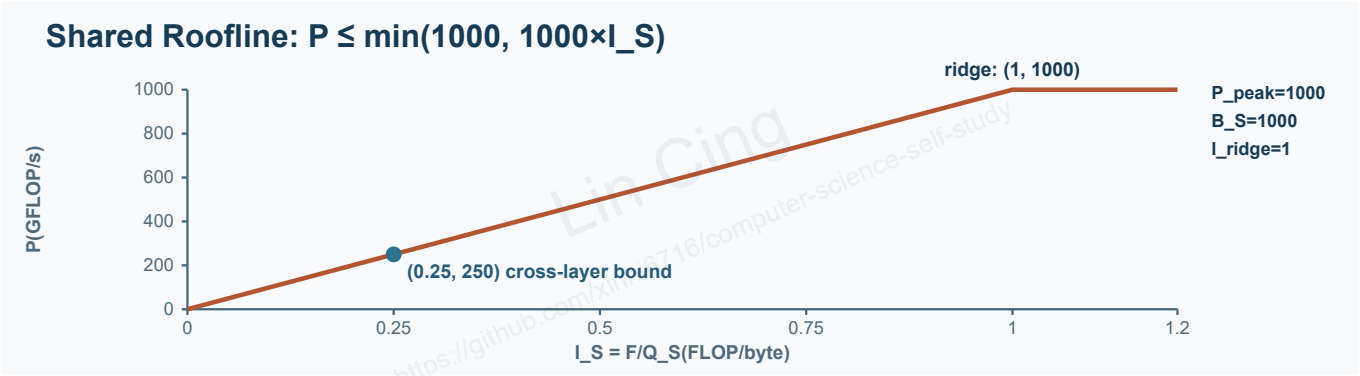



$$I_D = F/Q_D = 10^9/(0.25 \times 10^9) = 4\ \mathrm{FLOP/byte},$$

## Do not mix traffic boundaries (continued)

$$P_D \le \min(P_C, I_D B_D) = \min(1000, 4 \cdot 100) = 400 \text{ GFLOP/s}.$$

The DRAM ridge is $P_C/B_D = 10$ FLOP/byte. Its independent working point $(4, 400)$ lies on the bandwidth slope.



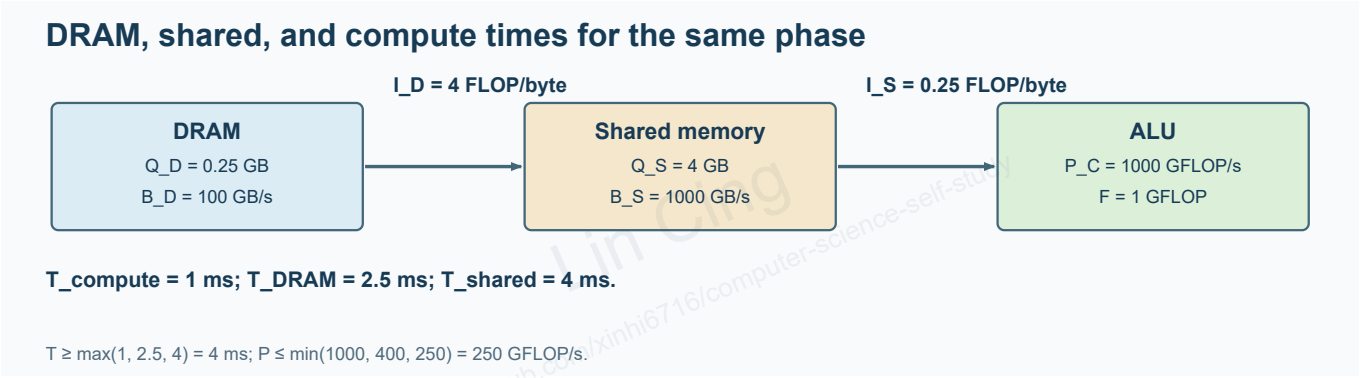

$$I_S = F/Q_S = 10^9/(4 \times 10^9) = 0.25 \text{ FLOP/byte},$$

$$P_S \le \min(P_C, I_S B_S) = \min(1000, 0.25 \cdot 1000) = 250 \text{ GFLOP/s}.$$



## Do not mix traffic boundaries (continued)

The shared ridge is 1 FLOP/byte. Its plateau remains 1000 GFLOP/s; $(0.25, 250)$ lies on the slope. Drawing 250 as the entire horizontal shared roof would change the model.




The three resource times are

$$T_C = 10^9/(1000 \times 10^9) = 0.001 \text{ s},$$

$$T_D = 0.25 \times 10^9/(100 \times 10^9) = 0.0025 \text{ s}, \qquad T_S = 4 \times 10^9/(1000 \times 10^9) = 0.004 \text{ s}.$$

## Do not mix traffic boundaries (continued)

Even if their traffic overlaps completely,

$$T \geq \max(1, 2.5, 4) = 4 \text{ ms}, \qquad P \leq F/T = 10^9/0.004 = 250 \text{ GFLOP/s}.$$

At $(I_D, I_S) = (4, 0.25)$, the combined model bound is 250 GFLOP/s. Bank replay, synchronization, occupancy limits, and extra traffic may lower actual throughput. Use each interface's own byte denominator and sustainable bandwidth for this same phase.

| Resource | Bytes | Intensity | Sustained ceiling or bandwidth | Resource bound |
|---|---|---|---|---|
| Compute | Not applicable | Not applicable | 1000 GFLOP/s | 1000 GFLOP/s |
| DRAM | $0.25 \times 10^9$ | 4 FLOP/byte | 100 GB/s | 400 GFLOP/s |
| Shared | $4 \times 10^9$ | 0.25 FLOP/byte | 1000 GB/s | 250 GFLOP/s |
| Combined | Separate per interface | Separate per interface | Separate per interface | 250 GFLOP/s |



## A.9 Counterexamples to performance claims

Reducing register count can cause spills and additional traffic despite higher occupancy. Different precisions, approximate functions, FMA counting, or transfer endpoints also make equal reported FLOP counts incomparable. High occupancy can coexist with stalls; fewer sectors can coexist with bank/channel imbalance. A roof is an upper bound, not an attained rate. If AI 0.3 and 320 GB/s imply 96 GFLOP/s while 1.2 GFLOP in 5 ms suggests 240, the phase, units, FLOP count, and interface bytes cannot all describe the same experiment. Added divergence cannot explain exceeding an upper bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A.10 Correctness before mapping

Prove unique output ownership, valid indices, collective participation, and phase ordering before comparing mappings. For scan/radix, the interval-prefix and stability proofs justify all legal integer inputs; for N-body, each target/source pair and mass factor must be correct. Then compare measured end-to-end time including transfer, launch, synchronization, traffic, and numerical requirements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A.11 Historical architecture and portable methods

GPUs evolved from fixed graphics stages toward unified programmable processors. CUDA and PTX map many logical threads onto scalable resources. Warp width, banks, caches, and numerical support can change while latency, throughput, capacity, and visibility remain distinct contracts. Historical Tesla/GeForce examples explain design constraints rather than current rankings. Mapping, resource floors, explicit memory boundaries, and complete numerical traces transfer across generations only after updating the architecture parameters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References

Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Revised Fourth Edition, Appendix A. Historical CUDA/Tesla cases and the separately declared sector/bank/occupancy models retain their distinct assumptions. See the unit guide, exercises, and solutions.

The source map covers A.1–A.11, printed A-3–A-77, PDF 732–806. Warp scheduling/divergence appear at A-28–A-30 (PDF 757–759), the 16-bank case at A-40 (PDF 769), GeForce figure A.7.1 at A-47 (PDF 776), and scan/reduction/radix/N-body at A-60–A-69 (PDF 789–798). See Roofline DOI and NERSC interface-byte definitions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

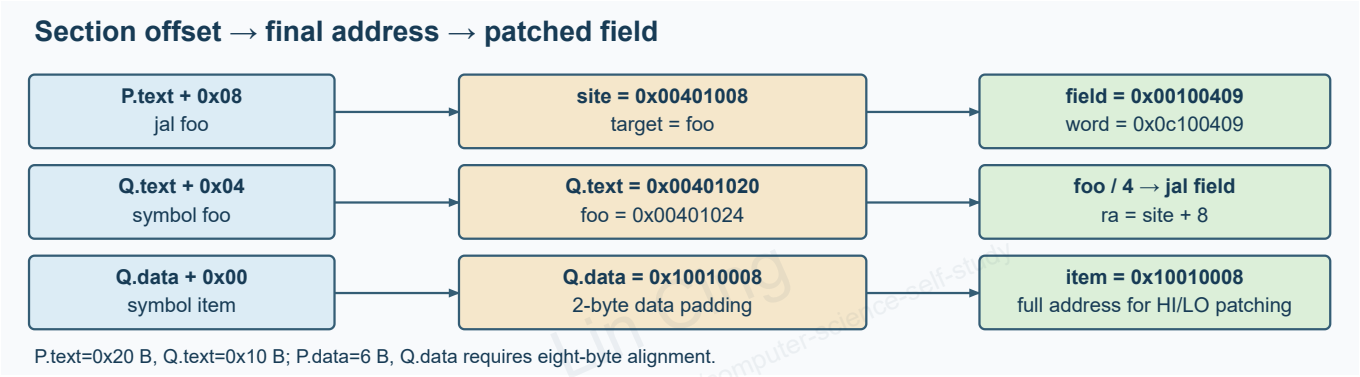
## Five boundaries between source and machine state

Compilers produce assembly or objects. Assemblers encode instructions, directives, and labels into code, symbols, and relocations. Linkers bind cross-object names and assign addresses. Loaders establish process mappings. CPUs execute ISA operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Five boundaries between source and machine state (continued)




A parseable object need not execute correctly. Successful linking does not prove ABI compliance. Check legal fields, unique symbol bindings, correct relocations, segment permissions, and preserved caller/callee state at their respective boundaries.

## The MIPS model

Use pre-Release-6 MIPS32, four-byte machine instructions, one branch/jump delay slot, and jal saving ra=PC+8. Examples show nop slots and assume no load delay. Select matching SPIM delayed-branch settings; its services are not Linux's MIPS syscall ABI.

Addresses are 32-bit; integer results wrap modulo 2^32 when specified. Branches sign-extend imm 16 and multiply by four. J-targets concatenate PC+4's high nibble, the 26-bit field, and 00. This teaching object model states addends explicitly and is not a complete ELF parser.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.1 Instructions, directives, and macros

`addu $t0,$t1,$t2` is a machine instruction. `.text` selects a section, `.word` allocates data, and `.align n` aligns to 2^n bytes in this syntax. The CPU does not execute directives.

`move $s0,$s1` can expand to `addu $s0,$s1,$zero`; `la` may require two or more instructions depending on addressing. Macros expand syntax without runtime call frames. Binary size follows expanded instructions, data, and padding, not source lines times four. Assembly helps boot/exception/ABI boundaries; lower-level spelling alone does not prove speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## B.2 Two-pass assembler data

Pass one keeps a separate location counter per section and records each label's section/offset. Instructions, data, and alignment advance exact sizes. Pass two uses identical expansion and size rules. A known PC-relative reference within the same section can be encoded from its offset difference. A reference that still needs a final section base or external symbol produces a relocation. Different pseudo lengths between passes corrupt later labels; one expanded intermediate representation avoids guessing twice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A complete location-counter trace

All shown instructions are already four-byte machine instructions; initial text offset is zero.

```
.text
main: addiu $t0,$zero,0
loop: addiu $t0,$t0,1
      bne   $t0,$a0,loop
      nop
done: jr    $ra
      nop
.data
msg:  .asciiz "Hi"
.align 2
value: .word 0x1234abcd
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A complete location-counter trace (continued)

| Event | Before | After |
|---|---|---|
| main instruction | text 0 | text 4 |
| loop instruction | text 4 | text 8 |
| bne, nop | text 8 | text 16 |
| done:jr, nop | text 16 | text 24 |
| msg plus NUL | data 0 | data 3 |
| align 2 | data 3 | data 4, one padding byte |
| value | data 4 | data 8 |

Symbols are main=(text, 0), loop=(text, 4), done=(text, 16), msg=(data, 0), value=(data, 4). Linking later supplies section bases.



## Branch encoding and relocation invariance

For text base 0x00400000, bne is 0x00400008 and loop 0x00400004:

$$imm = (target - (PC + 4))/4 = (0x00400004 - 0x0040000c)/4 = -2 = 0xfffe.$$

Opcode 5, rs 8, and rt 4 give

$$(5 \ll 26) \mid (8 \ll 21) \mid (4 \ll 16) \mid 0xfffe = 0x1504fffe.$$

Moving the whole section by delta adds the same delta to target and PC, so their difference is invariant. Layout changes or cross-section targets require recalculation, signed 16-bit range, and alignment checks.

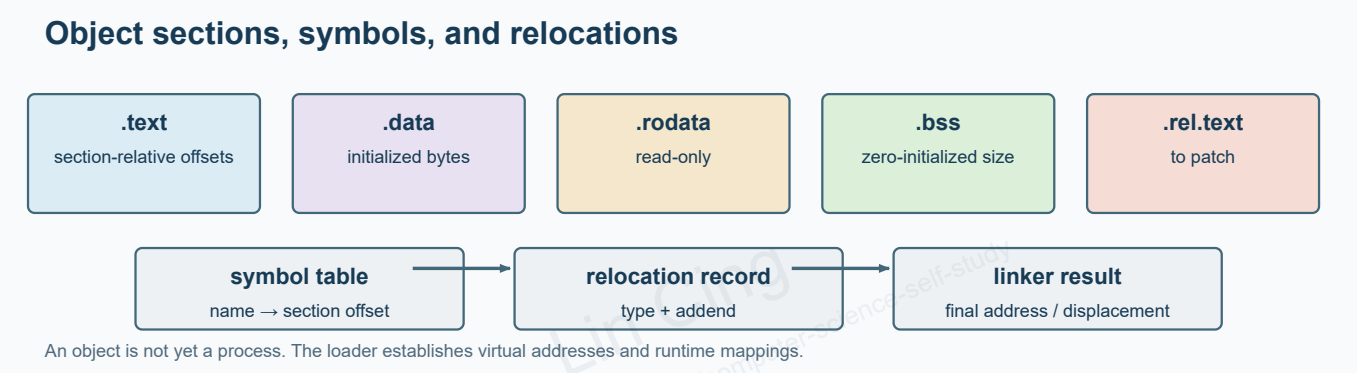
## Macro expansion can clobber its input

An unsafe multiply-by-three macro expands to `addiu $t0,$zero,3; mult $arg,$t0; mflo $v0`. If arg is t0 with original 7, the first instruction overwrites it and produces 9 rather than 21. Specify clobbers and preserve inputs before overwrite, or select nonconflicting temporaries.

Internal labels also need unique expansion names. Conditional assembly selects emitted code at assembly time; it is not a runtime branch. Omitted code has no execution cost, but conditions change layout and symbols.



## An object is not a memory snapshot




Text holds code, data initialized writable values, rodata constants, and bss zero-filled allocation size, usually without equal zero bytes in the file. Symbols also carry section, value, binding, and type. Relocations identify site, symbol, type, and how to obtain the addend, whether record-stored or encoded in place. A zero placeholder alone does not mean no repair is required. Debug information is not ordinary runtime load/store data.

## Symbol-resolution scope

Local loop labels in different objects can coexist. Duplicate strong globals normally cause an error; undefined references need a legal definition, library, or runtime binding. Weak/common visibility follows format/toolchain rules rather than arbitrary selection. These examples use unique strong definitions. Name ambiguity must be resolved before relocation can compute a target.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.3 Merging two objects

P.text size 0x20, Q.text 0x10, P.data 6 bytes, Q.data 4 bytes aligned to eight. Text base 0x00401000, data 0x10010000, input order P then Q.

| Input | Final base | Half-open range |
|---|---|---|
| P.text | 0x00401000 | [0x00401000, 0x00401020) |
| Q.text | 0x00401020 | [0x00401020, 0x00401030) |
| P.data | 0x10010000 | [0x10010000, 0x10010006) |
| padding | 0x10010006 | two bytes |
| Q.data | 0x10010008 | [0x10010008, 0x1001000c) |

Q's foo=(text, 4) becomes 0x00401024; item=(data, 0) becomes 0x10010008. Add a symbol offset to its own input section's final base, not one common executable base.

## Every field of a jal relocation

P.text offset 8 is site 0x00401008. Target foo 0x00401024 is aligned and shares PC+4's high nibble.

$$field = (0x00401024 \gg 2)\ \&\ 0x03ffffff = 0x00100409, \quad word = (3 \ll 26) \mid field = 0x0c100409.$$

CPU concatenation reconstructs 0x00401024. jal writes ra 0x00401010 and executes site+4's nop slot first. A target in another 256 MiB region requires a legal veneer/indirect sequence or a range error; truncating 26 bits cannot fix it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## High/low relocation and sign extension

For V=0x1234abcd, lui/ori uses high 0x1234 and zero-extended low 0xabcd. For lui/addiu, that low is −21555, so

$$H = ((V + 0x8000) \gg 16)\ \&\ 0xffff = 0x1235, \qquad (0x1235 \ll 16) - 21555 = 0x1234abcd.$$

Using 0x1234 with addiu yields 0x1233abcd, off by 0x10000. Real HI16/LO16 relocations also obey ABI addend/pair rules; this final-value arithmetic does not implement every relocation format.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A left-to-right archive scan

Define a linker that always includes objects, then extracts needed archive members repeatedly to a fixed point within that archive, but never automatically revisits earlier archives. main.o references f; F.o defines f and references g; G.o defines g.

`main.o libF.a libG.a` updates unresolved sets {f}→{g}→{} and succeeds. Reversing libraries sees only f while at libG, extracts nothing, and later libF leaves g undefined. Actual linkers can provide group/rescan options; this is the declared model rather than a C-language rule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Dynamic linking and a GOT pointer

Static linking includes selected library code; dynamic linking binds shared-object references at startup or first use. Processes can map one readonly backing page at different VAs without sharing writable globals.

An abstract GOT slot starts unresolved. The loader finds foo 0x70001234 and installs that pointer; later indirect calls read it. Lazy binding can first invoke a resolver then update the slot. Actual stubs/registers follow their ABI. ASLR changes bases; PIC uses relative references/tables to reduce text patching. Matching names alone do not guarantee version/ABI compatibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## B.4 Loader file size and memory size

A writable segment has file offset 0x2000, VA 0x10010000, file size 0x100, memory size 0x180, page size 0x1000. Map the first 0x100 bytes and zero the next 0x80; do not take unrelated following file bytes as bss.

Segments can combine sections with common permissions. Loaders primarily follow executable load metadata rather than section names. Text may be R-X, data RW-, and guard pages unmapped. Runtime setup precedes main; ELF/PIE/interpreter details depend on the OS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.5 Byte order and alignment

At 0x100, the value 0x12345678 has big-endian bytes 12, 34, 56, 78 or little-endian 78, 56, 34, 12. Byte placement changes, not the register value or bit positions within an instruction field.

Strict MIPS lw requires low address bits 00; 0x102 is unaligned. Packet formats need explicit widths, byte order, and padding; native struct dumps are not portable protocols. Typical stack/heap growth sketches are ABI/OS choices rather than universal layouts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

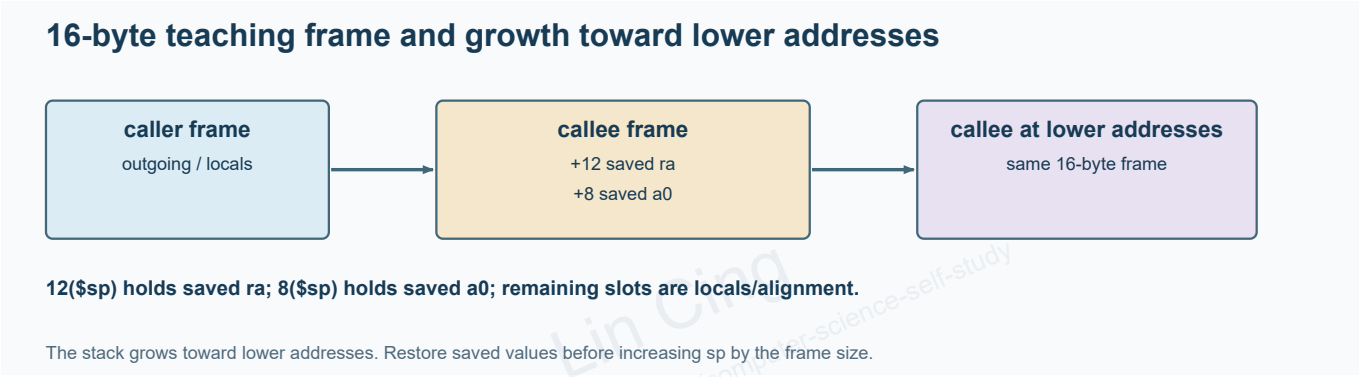
## B.6 Caller and callee state

Arguments often use a0..a3, returns v0/v1, t-registers caller-saved, and s-registers callee-saved. A callee modifying s0 must restore its incoming value. A caller needing old a0 after a call must preserve it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.6 Caller and callee state (continued)




This recursion example uses a teaching frame, not a complete o32 nonleaf layout. Full ABIs can require outgoing argument space, alignment, and other saves. ISA-correct code with the wrong calling convention can still corrupt its caller.

## The complete sum_to procedure

Require n≥0, a representable signed 32-bit sum, aligned entry sp, and enough stack. addu wraps modulo 2^32; the range precondition makes that result equal to the mathematical sum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The complete sum_to procedure (continued)

```
sum_to:
  addiu $sp,$sp,-16
  sw    $ra,12($sp)
  sw    $a0,8($sp)
  beq   $a0,$zero,base
  nop
  addiu $a0,$a0,-1
  jal   sum_to
  nop
  lw    $t0,8($sp)
  addu  $v0,$v0,$t0
  j     done
  nop
base:
  addu  $v0,$zero,$zero
done:
  lw    $ra,12($sp)
  addiu $sp,$sp,16
  jr    $ra
  nop
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## The complete sum_to procedure (continued)

Each transfer has its slot. Reload this activation's n from its frame after recursion; neither a0 nor a pre-call t0 is guaranteed to survive.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Actual recursive frames and results

Start at sp 0x1000 and call sum_to(3); the base case also allocates.

| n | Current sp | Saved n address | Saved ra address | Return v0 |
|---|---|---|---|---|
| 3 | 0x0ff0 | 0x0ff8 | 0x0ffc | 6 |
| 2 | 0x0fe0 | 0x0fe8 | 0x0fec | 3 |
| 1 | 0x0fd0 | 0x0fd8 | 0x0fdc | 1 |
| 0 | 0x0fc0 | 0x0fc8 | 0x0fcc | 0 |

Unwinding computes 0, 0+1=1, 1+2=3, 3+3=6 and restores sp 0x1000. Four simultaneous frames use 64 bytes. Restore with the current base before releasing it. Old storage remains addressable with recalculated offsets or an old-base register, but blindly retaining offsets after moving sp reads the wrong addresses.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Inductive recursion correctness

The contract returns v0=n(n+1)/2 and restores sp/callee-saved state. At n=0, the base returns zero and restores ra/sp. For positive n, save this activation's n/ra in its nonoverlapping frame. By induction, the child returns (n−1) n/2 and restores the parent's sp. Reload n, add it, and obtain n(n+1)/2. The epilogue restores ra/sp. Decreasing nonnegative n terminates.

Stack capacity and representability are premises. Negative inputs, unbounded recursion, and stack overflow lie outside the contract. Running n=3 alone does not prove it generally.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.7 Exceptions are not ordinary ABI calls

Synchronous faults arise from instructions; interrupts arrive externally. A handler can interrupt any point with caller-saved values live, so saving only callee-saved registers is insufficient. Hardware provides recovery PC/status; software saves what it clobbers, identifies cause, handles/acknowledges sources, and returns according to recovery policy. Uncleared device state can immediately retrigger. Nested handlers need separate contexts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## EPC with and without a branch delay slot

A recoverable non-slot fault has BD=0 and EPC at the faulting instruction. Repairing a page then retrying normally returns there. A completed syscall can instead advance under a stated policy.

If a branch is 0x00400020 and its slot load 0x00400024 faults, BD=1 and EPC 0x00400020 point to the branch. EPC+4 alone loses its decision; preserve/reconstruct branch and slot semantics or correctly reexecute them. Precise state preserves older effects and prevents forbidden faulting/younger effects. Interrupt priority does not reorder synchronous faults past older instructions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.8 Polling an MMIO receiver

Define a teaching device with control 0xffff0000, data 0xffff0004, ready bit 0. Reading data consumes a byte and clears ready. MMIO is uncached with the required platform ordering; these addresses are not a universal SPIM guarantee.

```
lui   $t0,0xffff
poll:
lw    $t1,0($t0)
andi  $t1,$t1,1
beq   $t1,$zero,poll
nop
lw    $v0,4($t0)
andi  $v0,$v0,0xff
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Control returns 0, 0, 1. There are three control reads and one data read. Moving the data read into the branch slot executes it even when not ready, changing device state. Keep the nop.

## B.9 SPIM service semantics

SPIM print-int uses v0=1 and a0=integer; print-string uses v0=4 and a0=NUL-terminated address; exit uses v0=10. Service semantics come from the simulator table rather than instruction encoding. A service may clobber/result registers as specified. Real OS ABI numbers and argument rules differ.

```
addiu $v0,$zero,1
addiu $a0,$zero,42
syscall
addiu $v0,$zero,10
syscall
```

Neither syscall occupies a delay slot here. The simulator can implement the service directly rather than execute an identical real-OS handler. Observing output 42 checks this program's declared service behavior; it does not establish that a physical MIPS system uses the same exception vector or kernel ABI.



## B.10 Instruction-semantic checks

| Instructions | Operation and boundary |
|---|---|
| add/addu | same low 32-bit sum; add traps on signed overflow, addu does not |
| addiu | sign-extended immediate, no signed-overflow trap |
| andi/ori | zero-extended immediate |
| slt/sltu | signed/unsigned comparison can differ on identical bits |
| lb/lbu | sign/zero-extend an eight-bit value |
| lw/sw | base+signext immediate, aligned words in this model |
| beq/bne | compare GPRs, target=PC+4+(signext imm<<2), execute slot |
| j/jal | PC+4 high nibble; jal also writes ra=PC+8 |
| jr | register target with a slot |

## B.10 Instruction-semantic checks (continued)

0xffffffff means −1 under slt and 4294967295 under sltu. Signedness follows the operation/data contract, not a tag carried by bits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.11 Trace an incorrect address to its boundary

Wrong-function faults suggest symbol resolution/link maps. A constant off by 0x10000 suggests signed-low/high-pair arithmetic. Wrong post-call registers suggest ABI/slot state. Reversed bytes suggest serialization/endianness.

Assembler listings expose source/offset/word; object dumps expose symbols/relocations; link maps expose final placement; execution traces expose PC/register changes. Check the relevant field instead of assuming a compiler defect.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## B.12 Model boundaries for the exercises

Recompute layouts, negative low halves, archive dependencies, and recovery behavior under explicit versions. Do not memorize one hex output. Full ISAs also include FP, coprocessor, and system operations. A reference entry must be read for semantics, exceptions, alignment, and delay rules as well as opcode.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References

Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Revised Fourth Edition, Appendix B. B.2–B.9, printed B-10–B-44, PDF 815–849; link figure B.3.1 at B-19, PDF 824; instruction reference B.10 at B-45–B-80, PDF 850–885. These object/numeric/recursion examples use their stated teaching models; full platform interoperability requires the actual ABI.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study